

Carbon Accounting in Julia

Scope 1, 2 and 3 · audit trails and security · data parsing and normalisation · calculation engines and emission-factor libraries · integration and connectivity · a neural soft sensor for the annual analyzer campaign · anomaly detection

Pluto notebook printout · 11 notebooks · generated 2026-09-21 11:58 UTC

Everything in this document is the *output of the notebooks as they were executed*: the build runs each notebook headlessly, captures its cells and renders them here.

All data is synthetic and generated deterministically by `scripts/make_data.jl`. Emission factors are published defaults — replace them with a licensed factor set before any real disclosure.

Contents

1. **00_Overview_and_Notation**
2. **01_Data_Parsing_and_Normalization**
3. **02_Integration_and_Connectivity**
4. **03_Emission_Factor_Libraries**
5. **04_Scope1_Direct_Emissions**
6. **05_Scope2_Purchased_Energy**
7. **06_Scope3_Value_Chain**
8. **07_SoftSensor_Neural_Net**
9. **08_Anomaly_Detection**
10. **09_Audit_Trail_and_Security**
11. **10_Inventory_Rollup_Uncertainty_Disclosure**

00_Overview_and_Notation

19 cells · executed in 31.76 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

```
datadir (generic function with 1 method)
```

```
md"""
# Carbon accounting in Julia – platform overview

**Scope 1, 2 and 3 · audit trails and security · parsing and normalisation ·
calculation engines and emission-factor libraries · integration and connectivity ·
a neural soft sensor for the annual analyzer campaign · anomaly detection.**

The repository is a Julia package (`src/CarbonAccounting.jl`) plus eleven Pluto
notebooks that walk through it, plus a build pipeline that turns the notebooks into
the PDF you are reading.

| module | responsibility | notebook |
|---|---|---|
| `notation.jl` | symbolic legend, GWP sets,  $\Sigma_i$  | **00** |
| `ingest.jl` | parsing, normalisation, quarantine, lineage | **01** |
| `connect.jl` | connectors, incremental sync, idempotency | **02** |
| `factors.jl` | versioned emission-factor library | **03** |
| `engines.jl` | Scope 1 / 2 / 3 calculation engines | **04, 05, 06** |
| `softsensor.jl` | neural soft sensor + campaign design | **07** |
| `anomaly.jl` | EWMA, CUSUM,  $T^2$ , SPE, autoencoder | **08** |
| `security.jl` | RBAC, hash-chained audit trail, sealing | **09** |
| `uncertainty.jl`, `report.jl` | GUM + Monte Carlo, DQI, disclosure, exports | **10** |

Three design rules run through every file:

1. **The code carries the equation.** Every quantity keeps its mathematical symbol
   ( $\dot{Q}_i$ ,  $EF_i$ ,  $\hat{\sigma}$ ,  $W^{(1)}$ ,  $\hat{f}$ ,  $h_n$ ), so a cell can be read next to the formula
   it implements.
2. **A number is never reported without its qualification.** Provenance (source
   hash), data-quality class DQ, factor version and uncertainty travel with each row.
3. **Nothing is silently repaired.** Rows that cannot be interpreted are quarantined
   with a reason; every state change lands in a hash-chained ledger.
"""
```

Carbon accounting in Julia — platform overview

Scope 1, 2 and 3 · audit trails and security · parsing and normalisation · calculation engines and emission-factor libraries · integration and connectivity · a neural soft sensor for the annual analyzer campaign · anomaly detection.

The repository is a Julia package (`src/CarbonAccounting.jl`) plus eleven Pluto notebooks that walk through it, plus a build pipeline that turns the notebooks into the PDF you are reading.

module	responsibility	notebook
<code>notation.jl</code>	symbolic legend, GWP sets, Σ_i	00
<code>ingest.jl</code>	parsing, normalisation, quarantine, lineage	01
<code>connect.jl</code>	connectors, incremental sync, idempotency	02
<code>factors.jl</code>	versioned emission-factor library	03
<code>engines.jl</code>	Scope 1 / 2 / 3 calculation engines	04, 05, 06
<code>softsensor.jl</code>	neural soft sensor + campaign design	07
<code>anomaly.jl</code>	EWMA, CUSUM, T^2 , SPE, autoencoder	08
<code>security.jl</code>	RBAC, hash-chained audit trail, sealing	09
<code>uncertainty.jl, report.jl</code>	GUM + Monte Carlo, DQI, disclosure, exports	10

Three design rules run through every file:

1. **The code carries the equation.** Every quantity keeps its mathematical symbol ($\dot{Q}_i$, EF_i , $\hat{\sigma}$, $W^{(1)}$, $\hat{r}_i$, h_n), so a cell can be read next to the formula it implements.
2. **A number is never reported without its qualification.** Provenance (source hash), data-quality class DQ, factor version and uncertainty travel with each row.
3. **Nothing is silently repaired.** Rows that cannot be interpreted are quarantined with a reason; every state change lands in a hash-chained ledger.

```
md"""
## 1. The notation legend is part of the code

`NOTATION` is a table of `(symbol, LaTeX, meaning, unit, area)`; `check_notation()`
asserts that every symbol in it is a legal Julia identifier, so documentation and
source cannot drift apart.
"""
```

1. The notation legend is part of the code

NOTATION is a table of (symbol, LaTeX, meaning, unit, area); check_notation() asserts that every symbol in it is a legal Julia identifier, so documentation and source cannot drift apart.

```
length(NOTATION), NOTATION_AREAS, check_notation()

Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1, ("90", MIME type text/plain)), (2,
(Dict{Symbol, Any}(:prefix => "String", :elements => Tuple{Int64, Any}[(1, ("\"boundary\"", MIME
type text/plain)), (2, ("\"activity\"", MIME type text/plain)), (3, ("\"factors\"", MIME type
text/plain)), (4, ("\"scope1\"", MIME type text/plain)), (5, ("\"scope2\"", MIME type
text/plain)), (6, ("\"scope3\"", MIME type text/plain)), (7, ("\"uncertainty\"", MIME type
text/plain)), (8, ("\"softsensor\"", MIME type text/plain)), (9, ("\"anomaly\"", MIME type
text/plain)), (10, ("\"audit\"", MIME type text/plain))], :type => :Array, :prefix_short => "",
:objectid => "6cc14f54dd119f19"), MIME type application/vnd.pluto.tree+object)), (3,
(Dict{Symbol, Any}(:prefix => "String", :elements => Tuple{Int64, Any}[], :type => :Array,
:prefix_short => "", :objectid => "aa71eab8693bffc"), MIME type
application/vnd.pluto.tree+object))], :type => :Tuple, :objectid => "f0e278b4991cf94")
```

Markdown.parse(nototation_markdown(area="activity"))

Symbol	LaTeX	Meaning	Unit	Area
$\dot{Q}_i$	$\dot{\text{Q}}_i$	activity rate of source i: energy or mass flow per period	GJ, t, kWh	activity
$\dot{m}_i$	$\dot{\text{m}}_i$	mass flow of fuel i ($\dot{m} = \dot{n} \cdot M$: mole flow $\times$ molar mass)	t/ $\mathcal{P}$	activity
$\dot{n}_i$	$\dot{\text{n}}_i$	molar flow of species i	kmol/ $\mathcal{P}$	activity
$\dot{V}_i$	$\dot{\text{V}}_i$	gas volumetric flow, normal conditions (0 °C, 1.01325 bar)	m ³ (n)/ $\mathcal{P}$	activity
OF _i	OF _i	oxidation factor: carbon actually oxidised / carbon input (IPCC)	—	activity
η_b	η_b	boiler or combustion efficiency	—	activity
DRE	DRE	destruction & removal efficiency of a flare / abatement device	—	activity
F _i	F _i	load or utilisation factor of equipment i	—	activity
h _{op}	h _{op}	operating hours of a source inside $\mathcal{P}$	h/ $\mathcal{P}$	activity
LHV _i	LHV _i	lower heating value of fuel i	GJ/t	activity
CC _i	CC _i	carbon content of fuel i	tC/t	activity
ΔP_i	ΔP_i	leak-driving pressure differential of source i	bar	activity

Markdown.parse(nototation_markdown(area="softsensor"))

Symbol	LaTeX	Meaning	Unit	Area
$\mathbf{x}$	$\vec{\text{x}}$	feature vector of continuous DCS/SCADA signals	mixed	softsensor
y	y	target variable, measured only during the analyzer campaign	mixed	softsensor
$\hat{y}$	$\hat{\text{y}}$	soft-sensor prediction, $\hat{y} = f_{\theta}(\mathbf{x})$	mixed	softsensor
θ	$\vec{\text{theta}}$	all trainable parameters $\{W^{(l)}, b^{(l)}\}$	—	softsensor
$W^{(l)}$	$W^{(l)}$	weight matrix of layer l, $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$	—	softsensor
$b^{(l)}$	$b^{(l)}$	bias vector of layer l	—	softsensor
$z^{(l)}$	$z^{(l)}$	pre-activation, $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$	—	softsensor
$a^{(l)}$	$a^{(l)}$	activation, $a^{(l)} = g(z^{(l)})$	—	softsensor
$\delta^{(l)}$	$\delta^{(l)}$	error signal, $\delta^{(l)} = \partial L / \partial z^{(l)}$ (back-propagation)	—	softsensor
η	η	learning rate of the optimiser	—	softsensor
β_1, β_2	β_1, β_2	Adam decay rates for the 1st / 2nd moment	—	softsensor
$\hat{m}, \hat{v}$	$\hat{\text{m}}, \hat{\text{v}}$	bias-corrected Adam moment estimates	—	softsensor
λ_{L2}	λ_{L2}	L2 weight-decay coefficient (ridge penalty $\Sigma \ W\ ^2$)	—	softsensor
L	L	loss, e.g. Gaussian negative log-likelihood	—	softsensor
$\hat{\sigma}$	$\hat{\sigma}$	estimated residual standard deviation (aleatoric noise)	mixed	softsensor
N_{camp}	N_{camp}	campaign samples, 7 d $\cdot$ 24 h = 168 per source	—	softsensor
D^2_M	D^2_M	Mahalanobis distance of a sample from the campaign cloud	—	softsensor
ε	ε	numerical floor (division by zero / log 0 protection)	—	softsensor

```
md"""
## 2. Units are types, so `E = Q · EF` cannot be wrong by accident

A factor is converted into the unit the activity is measured in – and refused when
the dimension does not match:

| activity | factor | result |
|---|---|---|
| `1250 MWh` | `0.42 kgCO2e/kWh` | 525 tCO2e |
| `1250 MWh` | `0.42 kgCO2e/GJ` | 1.89 tCO2e – MWh → GJ, faithfully converted |
| `1250 MWh` | `0.42 kgCO2e/t` | DimensionMismatch – energy vs mass |

Canonical units: mass `t`, CO2e `tCO2e`, energy `GJ`, volume `m³` (normal
conditions), distance `km`, transport work `t·km`, money `USD2024`, temperature `K`.
"""
```

2. Units are types, so $E = Q \cdot EF$ cannot be wrong by accident

A factor is converted into the unit the activity is measured in — and refused when the *dimension* does not match:

activity	factor	result
1250 MWh	0.42 kgCO ₂ e/kWh	525 tCO ₂ e
1250 MWh	0.42 kgCO ₂ e/GJ	1.89 tCO ₂ e — MWh → GJ, faithfully converted
1250 MWh	0.42 kgCO ₂ e/t	DimensionMismatch — energy vs mass

Canonical units: mass t, CO₂e tCO₂e, energy GJ, volume m³ (normal conditions), distance km, transport work t·km, money USD2024, temperature K.

```
begin
    inputs = ["1,250 MWh", "1,250 kWh", "3,412 Btu", "1 toe", "2,204.6 lb", "1,000 scf"]
    q = parse_quantity(inputs)
    DataFrame(as_written=inputs, value_canonical=val_canon(q),
              canonical_unit=[CANONICAL[dimension(x)] for x in q])
end

Dict{Symbol, Any}(:rows => Any[(1, Any[("\1,250 MWh\\"", MIME type text/plain), ("4500.0", MIME
type text/plain), ("\"GJ\\"", MIME type text/plain)]), (2, Any[("\1,250 kWh\\"", MIME type
text/plain), ("4.5", MIME type text/plain), ("\"GJ\\"", MIME type text/plain)]), (3, Any[("\3,412
Btu\\"", MIME type text/plain), ("0.00359985", MIME type text/plain), ("\"GJ\\"", MIME type
text/plain)]), (4, Any[("\1 toe\\"", MIME type text/plain), ("41.868", MIME type text/plain),
(\"GJ\\"", MIME type text/plain)]), (5, Any[("\2,204.6 lb\\"", MIME type text/plain), ("0.99999",
MIME type text/plain), ("\"t\\"", MIME type text/plain)]), (6, Any[("\1,000 scf\\"", MIME type
text/plain), ("28.3168", MIME type text/plain), ("\"m³\\"", MIME type text/plain)]), :objectid =>
"2eece099adac274a", :schema => Dict{Symbol, Any}(:names => Any["as_written", "value_canonical",
"canonical_unit"], :types => ["String", "Float64", "String"])]
```

```
begin
     $\dot{Q}_i$  = q"1250 MWh"                # the activity as the utility reports it
    EFi = q"0.42 kgCO2e/kWh"            # the factor as the library holds it
    (Ei = emission( $\dot{Q}_i$ , EFi),
     same_energy_in_GJ = emission(q"1250 MWh", q"0.42 kgCO2e/GJ"),
     refused = try
         emission(q"1250 MWh", q"0.42 kgCO2e/t")    # energy × mass factor: refused
     catch e
         sprint(showerror, e)
     end)
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("Ei", ("525.0 tCO2e", MIME type text/plain)),
("same_energy_in_GJ", ("1.89 tCO2e", MIME type text/plain)), ("refused", ("\"DimensionMismatch:
activity «MWh» (energy) does not match factor denominator t (mass)\", MIME type text/plain))],
:type => :NamedTuple, :objectid => "1defe5d6855fde05")
```

```
md"""
## 3. The GWP conversion

`E_CO2e = E_CO2 + GWP_CH4 · E_CH4 + GWP_N2O · E_N2O`. The set used is part of the
disclosure: AR6 separates fossil from biogenic methane (29.8 vs 27.0 over
100 years), so a biogenic slip valued with the fossil factor is 10 % too high.
"""
```

3. The GWP conversion

$E_{CO_2e} = E_{CO_2} + GWP_{CH_4} \cdot E_{CH_4} + GWP_{N_2O} \cdot E_{N_2O}$. The set used is part of the disclosure: AR6 separates **fossil** from **biogenic** methane (29.8 vs 27.0 over 100 years), so a biogenic slip valued with the fossil factor is 10 % too high.

```
begin
    t = 1.0                # one tonne of each gas
    DataFrame(gas=["CH4 fossil", "CH4 biogenic", "N2O", "CO2"],
              AR4=[co2e(; CH4=t, gwp=GWP_AR4), co2e(; CH4=t, gwp=GWP_AR4, fossil=false),
                   co2e(; N2O=t, gwp=GWP_AR4), t],
              AR5=[co2e(; CH4=t, gwp=GWP_AR5), co2e(; CH4=t, gwp=GWP_AR5, fossil=false),
                   co2e(; N2O=t, gwp=GWP_AR5), t],
              AR6=[co2e(; CH4=t), co2e(; CH4=t, fossil=false), co2e(; N2O=t), t])
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\"CH4 fossil\"", MIME type text/plain), ("25.0", MIME
type text/plain), ("30.0", MIME type text/plain), ("29.8", MIME type text/plain)]), (2,
Any[("\"CH4 biogenic\"", MIME type text/plain), ("25.0", MIME type text/plain), ("28.0", MIME
type text/plain), ("27.0", MIME type text/plain)]), (3, Any[("\"N2O\"", MIME type text/plain),
("298.0", MIME type text/plain), ("265.0", MIME type text/plain), ("273.0", MIME type
text/plain)]), (4, Any[("\"CO2\"", MIME type text/plain), ("1.0", MIME type text/plain), ("1.0",
MIME type text/plain), ("1.0", MIME type text/plain)]), :objectid => "699b28e5d1fe869f", :schema
=> Dict{Symbol, Any}(:names => Any["gas", "AR4", "AR5", "AR6"], :types => ["String", "Float64",
"Float64", "Float64"])]
```

```
md"""
## 4. The platform end to end

The remaining cells run the whole chain on the repository's synthetic portfolio –
six sites in five countries, one reporting year – in a few seconds, so that the
following notebooks can go deep on one link at a time:

    raw exports → ingest → normalise → factors → engines → uncertainty → disclosure
                  ↘ soft sensor ↘ anomaly detection ↘ audit trail ↗

"""
```

4. The platform end to end

The remaining cells run the whole chain on the repository's synthetic portfolio — six sites in five countries, one reporting year — in a few seconds, so that the following notebooks can go deep on one link at a time:

```
raw exports → ingest → normalise → factors → engines → uncertainty → disclosure
                  ↘ soft sensor ↘ anomaly detection ↘ audit trail ↗
```

```
begin
    paths = [datadir("raw", f) for f in
              ["activity_erp_2024.csv", "electricity_utility_2024.csv",
               "procurement_2024.csv",
               "logistics_travel_2024.csv", "waste_2024.csv"]]
    records = ingest_files(paths; entity_default="AcmeIndustrial_SA").records
    inventory = compute_inventory(records; consolidation=:operational, date=Date(2024, 12,
31))
    inventory
end
```

```
InventoryResult [operational]
 E_Scope1      = 174781.393 tCO2e
 E_Scope2 loc  =  73433.265 tCO2e
 E_Scope2 mkt  =  86683.228 tCO2e
 E_Scope3      =  12509.686 tCO2e
 E_total       = 260724.344 tCO2e (445 rows)
```

```
contribution_table(inventory)
```

```
Dict{Symbol, Any}(:rows => Any[[1, Any[("\Scope 1\\"", MIME type text/plain), ("1.74781e5", MIME
type text/plain), ("0.670369", MIME type text/plain)]], (2, Any[("\Scope 2 (location based)\\"",
MIME type text/plain), ("73433.3", MIME type text/plain), ("0.281651", MIME type text/plain)]],
(3, Any[("\Scope 2 (market based)\\"", MIME type text/plain), ("86683.2", MIME type text/plain),
("0.332471", MIME type text/plain)]], (4, Any[("\Scope 3\\"", MIME type text/plain), ("12509.7",
MIME type text/plain), ("0.0479805", MIME type text/plain)]], :objectid => "7c0a3d7a3571335e",
:schema => Dict{Symbol, Any}(:names => Any["component", "E", "share"], :types => ["String",
"Float64", "Float64"])])
```

```
begin
    contrib = contribution_table(inventory)
    plt = bar(contrib.component, contrib.E,
              title="Greenhouse-gas inventory 2024 (operational control)",
              ylabel="tCO2e", xrotation=10, legend=false,
              color=[:steelblue :seagreen :darkseagreen :darkorange])
    save_figure(plt, "00_contribution"; saver=savefig)
    plt
end
```



```
md"""
## 5. How to read the rest

* every notebook activates the same environment and reads the same files, so each can
  be run on its own;
* cells keep the symbols of the equations: where a cell reads `E_Scope1 = Σi(...)`, the
  expression between `Σi` and the closing parenthesis is the summand of the published
  formula;
* the last cell of each notebook states what was verified, not merely what was
  computed – the same sentence appears in the PDF printout.

The PDF in `docs/` is produced by `julia --project=. scripts/build_pdf.jl`, which runs
these notebooks headlessly, renders the cell outputs to HTML and prints that to PDF
with headless Chrome.
"""
```

5. How to read the rest

- every notebook activates the same environment and reads the same files, so each can be run on its own;
- cells keep the symbols of the equations: where a cell reads $E_{\text{Scope1}} = \Sigma_i(\dots)$, the expression between Σ_i and the closing parenthesis is the summand of the published formula;
- the last cell of each notebook states what was **verified**, not merely what was computed — the same sentence appears in the PDF printout.

The PDF in docs/ is produced by `julia --project=. scripts/build_pdf.jl`, which runs these notebooks headlessly, renders the cell outputs to HTML and prints that to PDF with headless Chrome.

```
md"""
---
*Notebook 00 of the Carbon Accounting platform · full documentation in `README.md`.*
"""
```

Notebook 00 of the Carbon Accounting platform · full documentation in README.md.

```

begin
    # last cell: state what was *verified*, not merely what was computed
    println("notation symbols checked : ", length(NOTATION), " (invalid: ", check_notation(),
    ")")
    println("activity records          : ", length(records))
    println("inventory total          : ", round(inventory.E_total; digits=1), " tCO2e over
    ",
            length(inventory.rows), " calculated rows")
    println("Notebook 00 – the legend and the source code are consistent, and the platform
    runs end to end.")
end

```

01_Data_Parsing_and_Normalization

20 cells · executed in 20.81 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 01 · Data parsing and normalisation

Raw activity data arrives from meters, ERP extracts, fuel invoices and utility
portals: mixed delimiters, units glued to the value (`"1,234.5 MWh"`), decimal
commas, several date conventions, multi-language headers, duplicated rows.

`src/ingest.jl` turns that into typed `ActivityRecord`s carrying a canonical
unit per dimension (`t`, `GJ`, `m³`, `kWh`, `USD2024`, `t·km`), an explicit period
 $\mathcal{P} = [\tau_0, \tau_1]$ , a data-quality class  $DQ \in \{1,2,3,4\}$  and full lineage — and it
quarantines every row it cannot interpret, with the reason, instead of silently
dropping or coercing it.
"""
```

01 · Data parsing and normalisation

Raw activity data arrives from meters, ERP extracts, fuel invoices and utility portals: mixed delimiters, units glued to the value ("1,234.5 MWh"), decimal commas, several date conventions, multi-language headers, duplicated rows.

src/ingest.jl turns that into typed ActivityRecords carrying a **canonical unit** per dimension (t, GJ, m³, kWh, USD2024, t·km), an explicit period $\mathcal{P} = [\tau_0, \tau_1]$, a **data-quality class** $DQ \in \{1,2,3,4\}$ and full **lineage** — and it **quarantines** every row it cannot interpret, with the reason, instead of silently dropping or coercing it.

```
md"""### The raw export, exactly as it arrived – semicolons, decimal commas, German headers"
```

The raw export, exactly as it arrived — semicolons, decimal commas, German headers

```
begin
    f = datadir("raw", "activity_erp_2024.csv")
    println.(first(eachline(f), 8))
    f
end
```

"C:\\Users\\inter\\ca_acct_repo\\data\\raw\\activity_erp_2024.csv"

```
md"""
### Scalar parsers

`parse_number` accepts `1.234,5` as well as `1,234.5`; `parse_period` resolves
`2024-Q1`, `2024-Q3`, `2024` and `01.03.2024-31.03.2024` into  $\mathcal{P} = [\tau_0, \tau_1]$ ; the
`dayfirst` flag resolves `01/02/2024` explicitly instead of guessing.
"""
```

Scalar parsers

`parse_number` accepts `1.234,5` as well as `1,234.5`; `parse_period` resolves `2024-Q1`, `2024-Q3`, `2024` and `01.03.2024-31.03.2024` into $\mathcal{P} = [\tau_0, \tau_1]$; the `dayfirst` flag resolves `01/02/2024` explicitly instead of guessing.

```
begin
    nums = ["1.234,5", "1,234.5", "(1 250,75)", "12,5", "3.141,59"]
    dates = ["2024-03-31", "31/03/2024", "31.03.2024", "2024-03"]
    (numeric=[parse_number(s) for s in nums],
     dates=[parse_date(s) for s in dates],
     periods=[string(parse_period(s)) for s in
               ["2024-Q1", "2024-Q3", "2024", "01.03.2024-31.03.2024"]])
end
```

```
Dict{Symbol, Any}(:elements => Tuple{("numeric", (Dict{Symbol, Any}(:prefix => "Float64",
:elements => Tuple{Int64, Any}[(1, ("1234.5", MIME type text/plain)), (2, ("1234.5", MIME type
text/plain)), (3, ("-1250.75", MIME type text/plain)), (4, ("12.5", MIME type text/plain)), (5,
("3141.59", MIME type text/plain))], :type => :Array, :prefix_short => "", :objectid =>
"6ef660c6d29c68ff"), MIME type application/vnd.pluto.tree+object)), ("dates", (Dict{Symbol, Any}
(:prefix => "Dates.Date", :elements => Tuple{Int64, Any}[(1, ("2024-03-31", MIME type
text/plain)), (2, ("2024-03-31", MIME type text/plain)), (3, ("2024-03-31", MIME type
text/plain)), (4, ("2024-03-01", MIME type text/plain))], :type => :Array, :prefix_short => "",
:objectid => "324c5416de847464"), MIME type application/vnd.pluto.tree+object)), ("periods",
(Dict{Symbol, Any}(:prefix => "String", :elements => Tuple{Int64, Any}[(1, ("\"
(Dates.Date(\\\\"2024-01-01\\\\"), Dates.Date(\\\\"2024-03-31\\\\"))\\\"", MIME type text/plain)), (2,
("\\\"(Dates.Date(\\\\"2024-03-01\\\\"), Dates.Date(\\\\"2024-03-31\\\\"))\\\"", MIME type text/plain)),
(3, ("\"(Dates.Date(\\\\"2024-01-01\\\\"), Dates.Date(\\\\"2024-12-31\\\\"))\\\"", MIME type
text/plain)), (4, ("\"(Dates.Date(\\\\"2024-03-01\\\\"), Dates.Date(\\\\"2024-03-31\\\\"))\\\"", MIME
type text/plain))], :type => :Array, :prefix_short => "", :objectid => "8d377a57f2ee3cd6"), MIME
type application/vnd.pluto.tree+object))), :type => :NamedTuple, :objectid => "19f516221c2eb154")
```

```
md"""
### Ingest the five exports
```

```
Every file is hashed (SHA-256) and – because a ledger and an actor are passed – the
ingestion itself is written to the audit trail, so the *provenance* of every figure
is provable rather than asserted.
"""
```

Ingest the five exports

Every file is hashed (SHA-256) and — because a ledger and an actor are passed — the ingestion itself is written to the audit trail, so the *provenance* of every figure is provable rather than asserted.

```
begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("notebook-01")))
    analyst = Actor("u001", "A. Analyst", :analyst)
    files = ["activity_erp_2024.csv", "electricity_utility_2024.csv", "procurement_2024.csv",
            "logistics_travel_2024.csv", "waste_2024.csv"]
    report = ingest_files([datadir("raw", f) for f in files];
                        entity_default="AcmeIndustrial_SA", ledger=ledger, actor=analyst,
                        context="notebook 01 - ingest run")

    report
end
```

```
IngestReport: 373 records, 2 quarantined, 5 file(s)
  activity_erp_2024.csv sha256=153a1d4a953e... rows=177
  electricity_utility_2024.csv sha256=5493b5ea9f79... rows=72
  procurement_2024.csv sha256=827f861091b4... rows=42
  logistics_travel_2024.csv sha256=a10fc2eb64d6... rows=66
  waste_2024.csv sha256=675305a28ae5... rows=18
```

```
md"""
### What was quarantined, and why

The quarantine is the audit-friendly alternative to silent repair: the rows of the
demo that cannot be interpreted appear here with their reason.
"""
```

What was quarantined, and why

The quarantine is the audit-friendly alternative to silent repair: the rows of the demo that cannot be interpreted appear here with their reason.

```
begin
    q = quarantine_dataframe(report.quarantined)
    isempty(q) ? q : combine(groupby(q, :reason), nrow => :rows)
end
```

```
Dict{Symbol, Any}(:rows => Any[[1, Any[("\\"unknown source type «Wundermittel» (no matching
SourceRule)\\"", MIME type text/plain), ("1", MIME type text/plain)]], (2, Any[("\\"unknown unit
«Epstein-Barrel» - register it in _register_units! (src/units.jl)\\"", MIME type text/plain),
("1", MIME type text/plain)]], :objectid => "5768619159c2b382", :schema => Dict{Symbol, Any}
(:names => Any["reason", "rows"], :types => ["String", "Int64"]]))
```

```

begin
    records = report.records
    df = records_dataframe(records)
    select(df, :rec_id, :site, :source_type, :item, :t_o, :value, :unit, :Q_canon,
           :unit_canon, :DQ, :source, :source_hash)
end

Dict{Symbol, Any}(:rows => Any[(1, Any[("\activity_erp_2024.csv:2\"", MIME type text/plain),
("\SAO-01\"", MIME type text/plain), ("\natural_gas\"", MIME type text/plain), ("\Erdgas
Kessel 1\"", MIME type text/plain), ("2024-01-01", MIME type text/plain), ("30184.0", MIME type
text/plain), ("\GJ\"", MIME type text/plain), ("30184.0", MIME type text/plain), ("\GJ\"", MIME
type text/plain), ("2", MIME type text/plain), ("\activity_erp_2024.csv\"", MIME type
text/plain), ("\153a1d4a953e\"", MIME type text/plain)]), (2,
Any[("\activity_erp_2024.csv:3\"", MIME type text/plain), ("\SAO-01\"", MIME type text/plain),
("\diesel_fleet\"", MIME type text/plain), ("\Diesel Gerador\"", MIME type text/plain), ("2024-
01-01", MIME type text/plain), ("5595.0", MIME type text/plain), ("\L\"", MIME type text/plain),
("5.595", MIME type text/plain), ("\m³\"", MIME type text/plain), ("2", MIME type text/plain),
("\activity_erp_2024.csv\"", MIME type text/plain), ("\153a1d4a953e\"", MIME type
text/plain)]), (3, Any[("\activity_erp_2024.csv:4\"", MIME type text/plain), ("\SAO-01\"", MIME
type text/plain), ("\natural_gas\"", MIME type text/plain), ("\Erdgas Kessel 1\"", MIME type
text/plain), ("2024-02-01", MIME type text/plain), ("27259.0", MIME type text/plain), ("\GJ\"",
MIME type text/plain), ("27259.0", MIME type text/plain), ("\GJ\"", MIME type text/plain), ("2",
MIME type text/plain), ("\activity_erp_2024.csv\"", MIME type text/plain), ("\153a1d4a953e\"",
MIME type text/plain)]), (4, Any[("\activity_erp_2024.csv:5\"", MIME type text/plain), ("\SAO-
01\"", MIME type text/plain), ("\diesel_fleet\"", MIME type text/plain), ("\Diesel Gerador\"",
MIME type text/plain), ("2024-02-01", MIME type text/plain), ("6416.0", MIME type text/plain),
("\L\"", MIME type text/plain), ("6.416", MIME type text/plain), ("\m³\"", MIME type
text/plain), ("2", MIME type text/plain), ("\activity_erp_2024.csv\"", MIME type text/plain),
("\153a1d4a953e\"", MIME type text/plain)]), (5, Any[("\activity_erp_2024.csv:6\"", MIME type
text/plain), ("\SAO-01\"", MIME type text/plain), ("\natural_gas\"", MIME type text/plain),
("\Erdgas Kessel 1\"", MIME type text/plain), ("2024-03-01", MIME type text/plain), ("27010.0",
MIME type text/plain), ("\GJ\"", MIME type text/plain), ("27010.0", MIME type text/plain),
("\GJ\"", MIME type text/plain), ("2", MIME type text/plain), ("\activity_erp_2024.csv\"", MIME
type text/plain), ("\153a1d4a953e\"", MIME type text/plain)]), (6,
Any[("\activity_erp_2024.csv:7\"", MIME type text/plain), ("\SAO-01\"", MIME type text/plain),
("\diesel_fleet\"", MIME type text/plain), ("\Diesel Gerador\"", MIME type text/plain), ("2024-
03-01", MIME type text/plain), ("5481.0", MIME type text/plain), ("\L\"", MIME type text/plain),
("5.481", MIME type text/plain), ("\m³\"", MIME type text/plain), ("2", MIME type text/plain),
("\activity_erp_2024.csv\"", MIME type text/plain), ("\153a1d4a953e\"", MIME type
text/plain)]), (7, Any[("\activity_erp_2024.csv:8\"", MIME type text/plain), ("\SAO-01\"", MIME
type text/plain), ("\natural_gas\"", MIME type text/plain), ("\Erdgas Kessel 1\"", MIME type
text/plain), ("2024-04-01", MIME type text/plain), ("22205.0", MIME type text/plain), ("\GJ\"",
MIME type text/plain), ("22205.0", MIME type text/plain), ("\GJ\"", MIME type text/plain), ("2",
MIME type text/plain), ("\activity_erp_2024.csv\"", MIME type text/plain), ("\153a1d4a953e\"",
MIME type text/plain)]), (8, Any[("\activity_erp_2024.csv:9\"", MIME type text/plain), ("\SAO-
01\"", MIME type text/plain), ("\diesel_fleet\"", MIME type text/plain), ("\Diesel Gerador\"",
MIME type text/plain), ("2024-04-01", MIME type text/plain), ("5874.0", MIME type text/plain),
("\L\"", MIME type text/plain), ("5.874", MIME type text/plain), ("\m³\"", MIME type
text/plain), ("2", MIME type text/plain), ("\activity_erp_2024.csv\"", MIME type text/plain),
("\153a1d4a953e\"", MIME type text/plain)]), (9, Any[("\activity_erp_2024.csv:10\"", MIME type
text/plain), ("\SAO-01\"", MIME type text/plain), ("\natural_gas\"", MIME type text/plain),
("\Erdgas Kessel 1\"", MIME type text/plain), ("2024-05-01", MIME type text/plain), ("18750.0",
MIME type text/plain), ("\GJ\"", MIME type text/plain), ("18750.0", MIME type text/plain),
("\GJ\"", MIME type text/plain), ("2", MIME type text/plain), ("\activity_erp_2024.csv\"", MIME
type text/plain), ("\153a1d4a953e\"", MIME type text/plain)]), (10,
Any[("\activity_erp_2024.csv:11\"", MIME type text/plain), ("\SAO-01\"", MIME type text/plain),
("\diesel_fleet\"", MIME type text/plain), ("\Diesel Gerador\"", MIME type text/plain), ("2024-

```

```
05-01", MIME type text/plain), ("5944.0", MIME type text/plain), ("\"L\"", MIME type text/plain),
("5.944", MIME type text/plain), ("\"m\"", MIME type text/plain), ("2", MIME type text/plain),
("\"activity_erp_2024.csv\"", MIME type text/plain), ("\"153a1d4a953e\"", MIME type
text/plain)]]), "more", (373, Any[("waste_2024.csv:19\"", MIME type text/plain), ("\"SHA-03\"",
MIME type text/plain), ("waste\"", MIME type text/plain), ("incineration\"", MIME type
text/plain), ("2024-01-01", MIME type text/plain), ("268.0", MIME type text/plain), ("\"t\"",
MIME type text/plain), ("268.0", MIME type text/plain), ("\"t\"", MIME type text/plain), ("3",
MIME type text/plain), ("waste_2024.csv\"", MIME type text/plain), ("\"675305a28ae5\"", MIME
type text/plain)]]], :objectid => "7f2146b7c55226f2", :schema => Dict{Symbol, Any}(:names =>
Any["rec_id", "site", "source_type", "item", "τ", "value", "unit", "Q̇_canon", "unit_canon",
"DQ", "source", "source_hash"], :types => ["String", "String", "String", "String", "Date",
"Float64", "String", "Float64", "String", "Int64", "String", "String"])))
```

```
md"""
```

```
### Normalisation is a dimensional step, not a cosmetic one
```

```
`1.000 L` of diesel, `4,200 kg` of the same fuel and `57 GJ` of heat all land in the
canonical unit of their dimension. The engines downstream only ever see canonical
units, which is why a unit mix-up cannot survive into the total.
"""
```

Normalisation is a *dimensional* step, not a cosmetic one

1.000 L of diesel, 4,200 kg of the same fuel and 57 GJ of heat all land in the canonical unit of their dimension. The engines downstream only ever see canonical units, which is why a unit mix-up cannot survive into the total.

```
combine(groupby(records_dataframe(records), [:scope, :unit, :unit_canon]),
        nrow => :records, :Q_canon => sum => :activity)
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("scope1", MIME type text/plain), ("GJ", MIME type
text/plain), ("GJ", MIME type text/plain), ("73", MIME type text/plain), ("1.55818e6", MIME
type text/plain)]), (2, Any[("scope1", MIME type text/plain), ("L", MIME type
text/plain), ("m³", MIME type text/plain), ("72", MIME type text/plain), ("368.14", MIME type
text/plain)]), (3, Any[("scope1", MIME type text/plain), ("t", MIME type text/plain),
("t", MIME type text/plain), ("12", MIME type text/plain), ("326208.0", MIME type
text/plain)]), (4, Any[("scope1", MIME type text/plain), ("kg", MIME type text/plain),
("t", MIME type text/plain), ("6", MIME type text/plain), ("0.868", MIME type text/plain)]),
(5, Any[("scope1", MIME type text/plain), ("m³(n)", MIME type text/plain), ("m³",
MIME type text/plain), ("6", MIME type text/plain), ("1.73591e7", MIME type text/plain)]), (6,
Any[("scope1", MIME type text/plain), ("component", MIME type text/plain), ("unit",
MIME type text/plain), ("6", MIME type text/plain), ("13382.0", MIME type text/plain)]), (7,
Any[("scope2", MIME type text/plain), ("MWh", MIME type text/plain), ("GJ", MIME type
text/plain), ("39", MIME type text/plain), ("331272.0", MIME type text/plain)]), (8,
Any[("scope2", MIME type text/plain), ("kWh", MIME type text/plain), ("GJ", MIME type
text/plain), ("33", MIME type text/plain), ("2.69459e5", MIME type text/plain)]), (9,
Any[("scope3", MIME type text/plain), ("BRL", MIME type text/plain), ("USD2024", MIME
type text/plain), ("15", MIME type text/plain), ("809969.0", MIME type text/plain)]), (10,
Any[("scope3", MIME type text/plain), ("EUR", MIME type text/plain), ("USD2024", MIME
type text/plain), ("19", MIME type text/plain), ("6.09814e6", MIME type text/plain)]), (11,
Any[("scope3", MIME type text/plain), ("CNY", MIME type text/plain), ("USD2024", MIME
type text/plain), ("5", MIME type text/plain), ("2.56258e5", MIME type text/plain)]), (12,
Any[("scope3", MIME type text/plain), ("ZAR", MIME type text/plain), ("USD2024", MIME
type text/plain), ("3", MIME type text/plain), ("82537.9", MIME type text/plain)]), (13,
Any[("scope3", MIME type text/plain), ("t.km", MIME type text/plain), ("t.km", MIME
type text/plain), ("30", MIME type text/plain), ("6.20511e7", MIME type text/plain)]), (14,
Any[("scope3", MIME type text/plain), ("p.km", MIME type text/plain), ("t.km", MIME
type text/plain), ("36", MIME type text/plain), ("3.71639e7", MIME type text/plain)]), (15,
Any[("scope3", MIME type text/plain), ("t", MIME type text/plain), ("t", MIME type
text/plain), ("18", MIME type text/plain), ("7332.0", MIME type text/plain)]]], :objectid =>
"3f6c2e9b46763553", :schema => Dict{Symbol, Any}(:names => Any["scope", "unit", "unit_canon",
"records", "activity"], :types => ["String", "String", "String", "Int64", "Float64"])
```

```
md"""
```

```
### Reporting controls before any calculation
```

```
`validate_records` applies the checks an auditor asks for first: non-positive
activity, missing entity, periods outside the reporting year, proxy-quality rows,
implausible magnitudes. `:error` rows must be corrected; `:info` rows are those to
refine with supplier data.
"""
```

Reporting controls before any calculation

`validate_records` applies the checks an auditor asks for first: non-positive activity, missing entity, periods outside the reporting year, proxy-quality rows, implausible magnitudes. `:error` rows must be corrected; `:info` rows are those to refine with supplier data.

```
begin
    issues = validate_records(records; year=2024)
    (by_severity=combine(groupby(DataFrame(issues), :severity), nrow => :n),
     examples=[(i.rec_id, i.severity, i.message) for i in first(issues, 4)])
end

Dict{Symbol, Any}(:elements => Tuple[("by_severity", (Dict{Symbol, Any}(:rows => Any[(1,
Any[(":info", MIME type text/plain), ("66", MIME type text/plain)]), (2, Any[(":error", MIME type
text/plain), ("6", MIME type text/plain)])), :objectid => "6bc48875548b723", :schema =>
Dict{Symbol, Any}(:names => Any["severity", "n"], :types => ["Symbol", "Int64"])), MIME type
application/vnd.pluto.table+object)), ("examples", (Dict{Symbol, Any}(:prefix => "Tuple{String,
Symbol, String}", :elements => Tuple{Int64, Any}[(1, (Dict{Symbol, Any}(:elements => Tuple{Int64,
Tuple}[(1, ("\"activity_erp_2024.csv:30\"", MIME type text/plain)), (2, (":info", MIME type
text/plain)), (3, ("\"proxy/spend-based estimate (DQ 4) - refine with supplier data\"", MIME type
text/plain)]), :type => :Tuple, :objectid => "448b0bd7afeb875"), MIME type
application/vnd.pluto.tree+object)), (2, (Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1,
("\"activity_erp_2024.csv:59\"", MIME type text/plain)), (2, (":info", MIME type text/plain)),
(3, ("\"proxy/spend-based estimate (DQ 4) - refine with supplier data\"", MIME type
text/plain)]), :type => :Tuple, :objectid => "46d91f178cb12c8b"), MIME type
application/vnd.pluto.tree+object)), (3, (Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1,
("\"activity_erp_2024.csv:88\"", MIME type text/plain)), (2, (":info", MIME type text/plain)),
(3, ("\"proxy/spend-based estimate (DQ 4) - refine with supplier data\"", MIME type
text/plain)]), :type => :Tuple, :objectid => "2a51369629996fe9"), MIME type
application/vnd.pluto.tree+object)), (4, (Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1,
("\"activity_erp_2024.csv:117\"", MIME type text/plain)), (2, (":info", MIME type text/plain)),
(3, ("\"proxy/spend-based estimate (DQ 4) - refine with supplier data\"", MIME type
text/plain)]), :type => :Tuple, :objectid => "b06eea4fb125d977"), MIME type
application/vnd.pluto.tree+object))), :type => :Array, :prefix_short => "", :objectid =>
"cd22d1d07b90f4d6"), MIME type application/vnd.pluto.tree+object))), :type => :NamedTuple,
:objectid => "5a644feffcfaf775")
```

```
md"""
### Data quality weighted by emissions, and the lineage of every file

DQ 1 is supplier- or meter-specific, DQ 4 a proxy or spend-based estimate. The
weighted table shows where the inventory's soft spots are, ranked by the tonnes they
carry - and `report.files` holds the SHA-256 of every file ingested.
"""
```

Data quality weighted by emissions, and the lineage of every file

DQ 1 is supplier- or meter-specific, DQ 4 a proxy or spend-based estimate. The weighted table shows where the inventory's soft spots are, ranked by the tonnes they carry — and `report.files` holds the SHA-256 of every file ingested.

```

begin
    inventory = compute_inventory(records; consolidation=:operational, date=Date(2024, 12,
31))
    dqi_table(inventory.rows)
end

Dict{Symbol, Any}(:rows => Any[(1, Any[("\electricity\\"", MIME type text/plain), ("1.60116e5",
MIME type text/plain), ("0.460889", MIME type text/plain), ("2.0", MIME type text/plain),
("0.11709", MIME type text/plain), ("\B - good, average data with documented source\\"", MIME
type text/plain)]), (2, Any[("\natural_gas\\"", MIME type text/plain), ("97225.2", MIME type
text/plain), ("0.279859", MIME type text/plain), ("2.0", MIME type text/plain), ("0.708378", MIME
type text/plain), ("\B - good, average data with documented source\\"", MIME type text/plain)]),
(3, Any[("\flaring\\"", MIME type text/plain), ("38701.1", MIME type text/plain), ("0.1114", MIME
type text/plain), ("2.0", MIME type text/plain), ("0.253574", MIME type text/plain), ("\B -
good, average data with documented source\\"", MIME type text/plain)]), (4,
Any[("\process_nitric_acid\\"", MIME type text/plain), ("34892.4", MIME type text/plain),
("0.100437", MIME type text/plain), ("2.0", MIME type text/plain), ("0.402244", MIME type
text/plain), ("\B - good, average data with documented source\\"", MIME type text/plain)]), (5,
Any[("\purchased_goods_spend\\"", MIME type text/plain), ("4162.14", MIME type text/plain),
("0.0119806", MIME type text/plain), ("4.0", MIME type text/plain), ("0.474342", MIME type
text/plain), ("\D - low, screening estimate to be refined\\"", MIME type text/plain)]), (6,
Any[("\employee_commuting\\"", MIME type text/plain), ("4089.19", MIME type text/plain),
("0.0117706", MIME type text/plain), ("3.0", MIME type text/plain), ("0.23065", MIME type
text/plain), ("\C - fair, industry average or proxy\\"", MIME type text/plain)]), (7,
Any[("\refrigerant\\"", MIME type text/plain), ("1812.38", MIME type text/plain), ("0.00521688",
MIME type text/plain), ("2.0", MIME type text/plain), ("0.108628", MIME type text/plain), ("\B -
good, average data with documented source\\"", MIME type text/plain)]), (8, Any[("\waste\\"", MIME
type text/plain), ("1724.34", MIME type text/plain), ("0.00496344", MIME type text/plain),
("3.0", MIME type text/plain), ("0.359212", MIME type text/plain), ("\C - fair, industry average
or proxy\\"", MIME type text/plain)]), (9, Any[("\upstream_transport\\"", MIME type text/plain),
("1349.18", MIME type text/plain), ("0.00388358", MIME type text/plain), ("3.0", MIME type
text/plain), ("0.206128", MIME type text/plain), ("\C - fair, industry average or proxy\\"", MIME
type text/plain)]), (10, Any[("\diesel_fleet\\"", MIME type text/plain), ("993.904", MIME type
text/plain), ("0.00286092", MIME type text/plain), ("2.0", MIME type text/plain), ("0.0519615",
MIME type text/plain), ("\B - good, average data with documented source\\"", MIME type
text/plain)]), (11, Any[("\process_cement\\"", MIME type text/plain), ("956.923", MIME type
text/plain), ("0.00275447", MIME type text/plain), ("2.0", MIME type text/plain), ("0.0583095",
MIME type text/plain), ("\B - good, average data with documented source\\"", MIME type
text/plain)]), (12, Any[("\downstream_transport\\"", MIME type text/plain), ("602.539", MIME type
text/plain), ("0.00173439", MIME type text/plain), ("3.0", MIME type text/plain), ("0.203151",
MIME type text/plain), ("\C - fair, industry average or proxy\\"", MIME type text/plain)]), (13,
Any[("\business_travel_air\\"", MIME type text/plain), ("327.079", MIME type text/plain),
("0.000941484", MIME type text/plain), ("3.0", MIME type text/plain), ("0.211896", MIME type
text/plain), ("\C - fair, industry average or proxy\\"", MIME type text/plain)]), (14,
Any[("\business_travel_land\\"", MIME type text/plain), ("255.221", MIME type text/plain),
("0.000734644", MIME type text/plain), ("3.0", MIME type text/plain), ("0.165529", MIME type
text/plain), ("\C - fair, industry average or proxy\\"", MIME type text/plain)]), (15,
Any[("\ldar\\"", MIME type text/plain), ("199.392", MIME type text/plain), ("0.000573942", MIME
type text/plain), ("4.0", MIME type text/plain), ("0.636396", MIME type text/plain), ("\D - low,
screening estimate to be refined\\"", MIME type text/plain)]), :objectid => "a3f1120d6a87d0ed",
:schema => Dict{Symbol, Any}(:names => Any["source_type", "E", "share", "DQ", "u_rel",
"quality"], :types => ["String", "Float64", "Float64", "Float64", "Float64", "String"])))

```

```
DataFrame(file=basename.([string(f["path"]) for f in report.files]),
  rows=[f["rows"] for f in report.files],
  delimiter=[f["delim"] for f in report.files],
  sha256=[first(string(f["sha256"]), 16) for f in report.files])
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\activity_erp_2024.csv\"", MIME type text/plain),
("177", MIME type text/plain), ("\";\\"", MIME type text/plain), ("\"153a1d4a953e9b24\"", MIME
type text/plain)]), (2, Any[("\electricity_utility_2024.csv\"", MIME type text/plain), ("72",
MIME type text/plain), ("\",\\"", MIME type text/plain), ("\"5493b5ea9f79c11e\"", MIME type
text/plain)]), (3, Any[("\procurement_2024.csv\"", MIME type text/plain), ("42", MIME type
text/plain), ("\",\\"", MIME type text/plain), ("\"827f861091b4e9e7\"", MIME type text/plain)]),
(4, Any[("\logistics_travel_2024.csv\"", MIME type text/plain), ("66", MIME type text/plain),
("\",\\"", MIME type text/plain), ("\"a10fc2eb64d6d6e7\"", MIME type text/plain)]), (5,
Any[("\waste_2024.csv\"", MIME type text/plain), ("18", MIME type text/plain), ("\",\\"", MIME
type text/plain), ("\"675305a28ae5a95f\"", MIME type text/plain)]), :objectid =>
"c47b720eefd58b", :schema => Dict{Symbol, Any}(:names => Any["file", "rows", "delimiter",
"sha256"], :types => ["String", "Int64", "String", "String"])
```

```
begin
  println("files ingested      : ", length(report.files))
  println("records accepted   : ", length(records))
  println("rows quarantined    : ", length(report.quarantined), " (reasons above)")
  println("validation findings : ", length(issues))
  println("audit trail          : ", chain_statement(ledger))
  println("Notebook 01 - every row is typed, dimension-checked, quality-tagged and
traceable.")
end
```

02_Integration_and_Connectivity

14 cells · executed in 18.07 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 02 · Integration and connectivity

Emissions data does not appear in a spreadsheet by magic: it is pulled from
meters, historians, ERPs, utility portals and supplier APIs. `src/connect.jl`
defines a small connector contract with the properties an audited inventory needs:

| property | mechanism |
|---|---|
| incremental sync | only rows newer than the stored high-water mark τ return (`since=...`) |
| idempotency | every row carries a deterministic key — a replayed page cannot double-count |
| resilience | transient failures retried with exponential back-off and jitter |
| provenance | the connector returns a `source_hash` and the next watermark |

Three connectors are implemented: a folder of exports, a REST/JSON API, and
in-memory rows. A mock ERP serves the demo API in-process, so this notebook runs
without a network.
"""
```

02 · Integration and connectivity

Emissions data does not appear in a spreadsheet by magic: it is pulled from meters, historians, ERPs, utility portals and supplier APIs. `src/connect.jl` defines a small connector contract with the properties an audited inventory needs:

property	mechanism
incremental sync	only rows newer than the stored high-water mark τ return (<code>since=...</code>)
idempotency	every row carries a deterministic key — a replayed page cannot double-count
resilience	transient failures retried with exponential back-off and jitter
provenance	the connector returns a <code>source_hash</code> and the next watermark

Three connectors are implemented: a **folder** of exports, a **REST/JSON API**, and in-memory rows. A mock ERP serves the demo API in-process, so this notebook runs without a network.

```
md"""### A folder connector — the 'our utility sends a CSV every month' interface"
```

A folder connector — the 'our utility sends a CSV every month' interface

```
begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("notebook-02")))
    service = Actor("svc-etl", "ETL Service", :service)
    folder = CSVFolderConnector(datadir("raw"), r"electricity.*\.csv$i")
    pull(folder; ledger=ledger, actor=service, context="nightly folder sync")
end
```

```
PullResult(72 rows, 1 page(s), 1 attempt(s), watermark=2026-09-21T11:06:26.346,
sha256=b3cf18d7457f...)
```

```
md"""
### A REST connector against an API that misbehaves

`MockERP` serves paginated JSON with `since` filtering and injects two `503`s, so the
retry logic is demonstrated rather than asserted. The pull reports how many HTTP
attempts it needed – evidence for the audit trail.
"""
```

A REST connector against an API that misbehaves

MockERP serves paginated JSON with since filtering and injects two 503s, so the retry logic is demonstrated rather than asserted. The pull reports how many HTTP attempts it needed — evidence for the audit trail.

```
begin
    function erp_rows()
        rows = Dict{String,String}[]
        for r in CSV.File(datadir("raw", "activity_erp_2024.csv"); delim=';',
                           types=String, missingstring=String[])
            d = Dict{String,String}(string(k) => string(v) for (k, v) in pairs(r))
            d["timestamp"] = "2024-03-01T00:00:00"
            push!(rows, d)
        end
        rows
    end
    erp = MockERP(erp_rows(); port=8877, page_size=25, transients=2)
    start!(erp)
    erp
end
```

```
MockERP(port=8877, rows=177, 1 hit(s), running)
```

```
begin
    conn = RESTConnector(erp_url(erp); page_size=25, token="demo-token")
    state = SyncState()
    records, quarantined, pulled = sync!(conn, state; ledger=ledger, actor=service,
                                         context="first incremental sync")
    (pull=pulled, accepted=length(records), quarantined=length(quarantined),
     watermark=state.watermark)
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("pull", ("PullResult(177 rows, 8 page(s), 8 attempt(s),
watermark=1970-01-01T00:00:00, sha256=ef8212fa3cc7...)", MIME type text/plain)), ("accepted",
("175", MIME type text/plain)), ("quarantined", ("2", MIME type text/plain)), ("watermark",
("1970-01-01T00:00:00", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"dd788f155bfb911")
```

```
md"""
### Idempotency: run the same sync again
```

The second run returns **zero new records**: every row's idempotency key is already in the sync state. This is what protects the inventory from a replayed page after a timeout, the most common way a connector silently doubles emissions.

```
"""
```

Idempotency: run the same sync again

The second run returns **zero new records**: every row's idempotency key is already in the sync state. This is what protects the inventory from a replayed page after a timeout, the most common way a connector silently doubles emissions.

```
begin
    again, _, pulled2 = sync!(conn, state; ledger=ledger, actor=service,
                             context="replay after timeout")
    (replayed_rows_accepted=length(again), attempts=pulled2.attempts,
     seen_keys=length(state.seen_keys))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("replayed_rows_accepted", ("0", MIME type text/plain)),
("attempts", ("8", MIME type text/plain)), ("seen_keys", ("177", MIME type text/plain))], :type
=> :NamedTuple, :objectid => "7e667642dbcf7272")
```

```
begin
    # the idempotency key is a hash of the row content: stable across runs, machines and
    order
    k1 = idempotency_key("rest:127.0.0.1", Dict("Kategorie" => "Erdgas", "Verbrauch" =>
"18000"))
    k2 = idempotency_key("rest:127.0.0.1", Dict("Verbrauch" => "18000", "Kategorie" =>
"Erdgas"))
    (description="key is content-addressed, so field order does not matter",
     key=first(k1, 24), equal=(k1 == k2))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("description", ("\"key is content-addressed, so field order
does not matter\"", MIME type text/plain)), ("key", ("\"f27d7a2085b24302f35e0de5\"", MIME type
text/plain)), ("equal", ("true", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"31b2a2f37b7cd5f0")
```

```
md#### The synchronisation history – one row per pull, with attempts and payload hash"
```

The synchronisation history — one row per pull, with attempts and payload hash

```
sync_history(state)
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("2026-09-21T11:53:08.806", MIME type text/plain),
("\"rest:127.0.0.1:8877\"", MIME type text/plain), ("175", MIME type text/plain), ("8", MIME type
text/plain), ("\"ef8212fa3cc7a892\"", MIME type text/plain)]), (2, Any[("2026-09-
21T11:53:09.130", MIME type text/plain), ("\"rest:127.0.0.1:8877\"", MIME type text/plain), ("0",
MIME type text/plain), ("8", MIME type text/plain), ("\"ef8212fa3cc7a892\"", MIME type
text/plain)]], :objectid => "85535f8f91503ca0", :schema => Dict{Symbol, Any}(:names => Any["ts",
"source", "rows", "attempts", "hash"], :types => ["DateTime", "String", "Int64", "Int64",
"String"])))
```

```
begin
    stop!(erp)
    println("mock API hits          : ", erp.hits)
    println("records accepted       : ", length(records))
    println("watermark                : ", state.watermark)
    println("audit trail                 : ", chain_statement(ledger))
    println("Notebook 02 - ingestion is incremental, replayed safely and logged with its
payload hash.")
end
```

03_Emission_Factor_Libraries

16 cells · executed in 17.64 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    datadir(parts...) = CarbonAccounting.datadir(parts...)
    figdir(parts...) = CarbonAccounting.figdir(parts...)
end
```

figdir (generic function with 1 method)

```
md"""
# 03 · Emission-factor libraries

A factor is not a number, it is a claim: it holds for a given activity, region,
gas and time window, somebody published it, and it carries an uncertainty. Every
record is therefore stored as an immutable `EFRecord` with

    EFi = (value, unit, gas, region, GWP set, validity window, DQ, σ/μ, source, version)

and resolved for an activity by (region → GLOBAL, newest version, item match).
Changing a factor creates a new version; it never overwrites history.
"""
```

03 · Emission-factor libraries

A factor is not a number, it is a **claim**: it holds for a given activity, region, gas and time window, somebody published it, and it carries an uncertainty. Every record is therefore stored as an immutable `EFRecord` with

`EFi = (value, unit, gas, region, GWP set, validity window, DQ, σ/μ, source, version)`

and resolved for an activity by (region → GLOBAL, newest version, item match). Changing a factor creates a **new version**; it never overwrites history.

```
begin
    lib = default_library()
    (count=length(lib), name=lib.name, version=lib.version,
     categories=length(unique(f.category for f in lib.records)),
     sources=length(unique(f.source for f in lib.records)))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("count", ("216", MIME type text/plain)), ("name",
("\CarbonAccounting built-in\\"", MIME type text/plain)), ("version", ("\2026.1\\"", MIME type
text/plain)), ("categories", ("39", MIME type text/plain)), ("sources", ("60", MIME type
text/plain))], :type => :NamedTuple, :objectid => "be8e1d1b92fb451a")
```

```
begin
    df = factors_dataframe(lib)
    first(select(df, :category, :item, :gas, :value, :unit, :region, :version, :DQ,
:uncertainty), 8)
end

Dict{Symbol, Any}(:rows => Any[(1, Any[("\natural_gas\\"", MIME type text/plain),
("\natural_gas\\"", MIME type text/plain), ("\CO2\\"", MIME type text/plain), ("56.1", MIME type
text/plain), ("\kgCO₂/GJ\\"", MIME type text/plain), ("\GLOBAL\\"", MIME type text/plain),
("\2026.1\\"", MIME type text/plain), ("2", MIME type text/plain), ("0.03", MIME type
text/plain)]), (2, Any[("\natural_gas\\"", MIME type text/plain), ("\natural_gas\\"", MIME type
text/plain), ("\CH4\\"", MIME type text/plain), ("0.001", MIME type text/plain), ("\kgCH₄/GJ\\"",
MIME type text/plain), ("\GLOBAL\\"", MIME type text/plain), ("\2026.1\\"", MIME type
text/plain), ("2", MIME type text/plain), ("0.5", MIME type text/plain)]), (3,
Any[("\natural_gas\\"", MIME type text/plain), ("\natural_gas\\"", MIME type text/plain),
("\N2O\\"", MIME type text/plain), ("0.0001", MIME type text/plain), ("\kgN₂O/GJ\\"", MIME type
text/plain), ("\GLOBAL\\"", MIME type text/plain), ("\2026.1\\"", MIME type text/plain), ("2",
MIME type text/plain), ("0.5", MIME type text/plain)]), (4, Any[("\fuel_oil\\"", MIME type
text/plain), ("\fuel_oil\\"", MIME type text/plain), ("\CO2\\"", MIME type text/plain), ("77.4",
MIME type text/plain), ("\kgCO₂/GJ\\"", MIME type text/plain), ("\GLOBAL\\"", MIME type
text/plain), ("\2026.1\\"", MIME type text/plain), ("2", MIME type text/plain), ("0.03", MIME
type text/plain)]), (5, Any[("\fuel_oil\\"", MIME type text/plain), ("\fuel_oil\\"", MIME type
text/plain), ("\CH4\\"", MIME type text/plain), ("0.003", MIME type text/plain), ("\kgCH₄/GJ\\"",
MIME type text/plain), ("\GLOBAL\\"", MIME type text/plain), ("\2026.1\\"", MIME type
text/plain), ("2", MIME type text/plain), ("0.5", MIME type text/plain)]), (6,
Any[("\fuel_oil\\"", MIME type text/plain), ("\fuel_oil\\"", MIME type text/plain), ("\N2O\\"",
MIME type text/plain), ("0.0006", MIME type text/plain), ("\kgN₂O/GJ\\"", MIME type text/plain),
("\GLOBAL\\"", MIME type text/plain), ("\2026.1\\"", MIME type text/plain), ("2", MIME type
text/plain), ("0.5", MIME type text/plain)]), (7, Any[("\diesel_stationary\\"", MIME type
text/plain), ("\diesel_stationary\\"", MIME type text/plain), ("\CO2\\"", MIME type text/plain),
("74.1", MIME type text/plain), ("\kgCO₂/GJ\\"", MIME type text/plain), ("\GLOBAL\\"", MIME type
text/plain), ("\2026.1\\"", MIME type text/plain), ("2", MIME type text/plain), ("0.02", MIME
type text/plain)]), (8, Any[("\diesel_stationary\\"", MIME type text/plain),
("\diesel_stationary\\"", MIME type text/plain), ("\CH4\\"", MIME type text/plain), ("0.003",
MIME type text/plain), ("\kgCH₄/GJ\\"", MIME type text/plain), ("\GLOBAL\\"", MIME type
text/plain), ("\2026.1\\"", MIME type text/plain), ("2", MIME type text/plain), ("0.5", MIME type
text/plain)]], :objectid => "14a39e99f4fd7f39", :schema => Dict{Symbol, Any}(:names =>
Any["category", "item", "gas", "value", "unit", "region", "version", "DQ", "uncertainty"], :types
=> ["String", "String", "String", "Float64", "String", "String", "String", "Int64", "Float64"])]
```

```
md"""
### Precedence: the same factor id, several claims

`lookup_factor` returns the candidates in precedence order – the requested region
first, then `GLOBAL`, then any other region, newest version inside each group. The
order is explicit, so a report can state which factor set it used.
"""
```

Precedence: the same factor id, several claims

`lookup_factor` returns the candidates in precedence order — the requested region first, then GLOBAL, then any other region, newest version inside each group. The order is explicit, so a report can state *which* factor set it used.

```

begin
    # a factor that exists in several regions and versions
    extras = [EFRecord("grid_DEU_2025.1", "Utility disclosure 2025", "2025.1", "electricity",
        "grid_average", "CO2e", 0.352, "kgCO2e/kWh", "DEU", "AR5",
        Date(2025, 1, 1), Date(2026, 12, 31), 2, 0.04, "utility residual
disclosure", ""),
        EFRecord("grid_BRA_2025.1", "Utility disclosure 2025", "2025.1", "electricity",
        "grid_average", "CO2e", 0.098, "kgCO2e/kWh", "BRA", "AR5",
        Date(2025, 1, 1), Date(2026, 12, 31), 2, 0.06, "utility residual
disclosure", "")]
    lib2 = FactorLibrary(lib.name, "2026.2", vcat(lib.records, extras))
    (rows=[(f.region, f.version, f.value, f.source) for f in
        lookup_factor(lib2, "electricity", "grid_average"; region="DEU", date=Date(2025,
6, 1))],
        note="DEU 2025.1 wins; the 2026.1 GLOBAL default follows")
end

```

```

Dict{Symbol, Any}(:elements => Tuple{("rows", (Dict{Symbol, Any}(:prefix => "Tuple{String,
String, Float64, String}", :elements => Any[(1, (Dict{Symbol, Any}(:elements => Tuple{Int64,
Tuple}[(1, ("\"DEU\"", MIME type text/plain)), (2, ("\"2025.1\"", MIME type text/plain)), (3,
("0.352", MIME type text/plain)), (4, ("\"utility residual disclosure\"", MIME type
text/plain))], :type => :Tuple, :objectid => "50e3848555182ca3"), MIME type
application/vnd.pluto.tree+object)), (2, (Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1,
("\"DEU\"", MIME type text/plain)), (2, ("\"2026.1\"", MIME type text/plain)), (3, ("0.38", MIME
type text/plain)), (4, ("\"IEA/Ember 2023 grid intensity\"", MIME type text/plain))], :type =>
:Tuple, :objectid => "cbe17d05f107a9e1"), MIME type application/vnd.pluto.tree+object)), (3,
(Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1, ("\"GLOBAL\"", MIME type text/plain)),
(2, ("\"2026.1\"", MIME type text/plain)), (3, ("0.436", MIME type text/plain)), (4,
("\"IEA/Ember 2023 grid intensity\"", MIME type text/plain))], :type => :Tuple, :objectid =>
"cf50de236e48b26"), MIME type application/vnd.pluto.tree+object)), (4, (Dict{Symbol, Any}
(:elements => Tuple{Int64, Tuple}[(1, ("\"BRA\"", MIME type text/plain)), (2, ("\"2025.1\"", MIME
type text/plain)), (3, ("0.098", MIME type text/plain)), (4, ("\"utility residual disclosure\"",
MIME type text/plain))], :type => :Tuple, :objectid => "f0758088d76dae0e"), MIME type
application/vnd.pluto.tree+object)), (5, (Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1,
("\"BRA\"", MIME type text/plain)), (2, ("\"2026.1\"", MIME type text/plain)), (3, ("0.087", MIME
type text/plain)), (4, ("\"IEA/Ember 2023 grid intensity\"", MIME type text/plain))], :type =>
:Tuple, :objectid => "e2be2c17f965d61"), MIME type application/vnd.pluto.tree+object)), (6,
(Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1, ("\"ZAF\"", MIME type text/plain)), (2,
("\"2026.1\"", MIME type text/plain)), (3, ("0.95", MIME type text/plain)), (4, ("\"IEA/Ember
2023 grid intensity\"", MIME type text/plain))], :type => :Tuple, :objectid =>
"438df107d2b20a7e"), MIME type application/vnd.pluto.tree+object)), (7, (Dict{Symbol, Any}
(:elements => Tuple{Int64, Tuple}[(1, ("\"USA\"", MIME type text/plain)), (2, ("\"2026.1\"", MIME
type text/plain)), (3, ("0.369", MIME type text/plain)), (4, ("\"IEA/Ember 2023 grid
intensity\"", MIME type text/plain))], :type => :Tuple, :objectid => "314759cb4f8d21b7"), MIME
type application/vnd.pluto.tree+object)), (8, (Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}
[(1, ("\"CHN\"", MIME type text/plain)), (2, ("\"2026.1\"", MIME type text/plain)), (3, ("0.58",
MIME type text/plain)), (4, ("\"IEA/Ember 2023 grid intensity\"", MIME type text/plain))], :type
=> :Tuple, :objectid => "c8acb45d1f727eb9"), MIME type application/vnd.pluto.tree+object)), (9,
(Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1, ("\"NLD\"", MIME type text/plain)), (2,
("\"2026.1\"", MIME type text/plain)), (3, ("0.328", MIME type text/plain)), (4, ("\"IEA/Ember
2023 grid intensity\"", MIME type text/plain))], :type => :Tuple, :objectid =>
"c62b09ad081826bb"), MIME type application/vnd.pluto.tree+object)), "more", (13, (Dict{Symbol,
Any}(:elements => Tuple{Int64, Tuple}[(1, ("\"POL\"", MIME type text/plain)), (2, ("\"2026.1\"",
MIME type text/plain)), (3, ("0.662", MIME type text/plain)), (4, ("\"IEA/Ember 2023 grid
intensity\"", MIME type text/plain))], :type => :Tuple, :objectid => "669778506b8f18f5"), MIME
type application/vnd.pluto.tree+object)), :type => :Array, :prefix_short => "", :objectid =>
"c1d619f4b9aa4b74"), MIME type application/vnd.pluto.tree+object)), ("note", ("\"DEU 2025.1 wins;

```

```
the 2026.1 GLOBAL default follows\"", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"481c141f3958b3fb")
```

```
md"""
### Dimensional fit is part of factor selection

`compatibility(f, activity_unit)` refuses to pair a factor with an activity of the
wrong dimension – so a `kgCO2e/kWh` factor never meets a `GJ` activity.
"""
```

Dimensional fit is part of factor selection

`compatibility(f, activity_unit)` refuses to pair a factor with an activity of the wrong dimension — so a `kgCO2e/kWh` factor never meets a `GJ` activity.

```
begin
    f = lookup_factor(lib, "natural_gas", "natural_gas"; region="GLOBAL")[1]
    describe_factor(f, compatibility(f, "GJ"), compatibility(f, "MWh"), compatibility(f,
"t")
end
```

```
Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1, ("\"natural_gas = 56.1 kgCO2/GJ
[CarbonAccounting built-in 2026.1, GLOBAL, DQ2, u_r=3%, IPCC 2006 Vol.2 Tab.2.2]\"", MIME type
text/plain)), (2, ("true", MIME type text/plain)), (3, ("true", MIME type text/plain)), (4,
("false", MIME type text/plain))], :type => :Tuple, :objectid => "1b48c33f55149019")
```

```
md"""
### Resolution: free-text activity → factor

`resolve_factor` walks the rule: exact item, then the rule's default item, then an
EEIO sector hint from the procurement text, and finally the mobile-combustion set.
The same entry point serves Scope 1, 2 and 3.
"""
```

Resolution: free-text activity → factor

`resolve_factor` walks the rule: exact item, then the rule's default item, then an EEIO sector hint from the procurement text, and finally the mobile-combustion set. The same entry point serves Scope 1, 2 and 3.

```

begin
    rules = ["natural_gas", "refrigerant", "ldar", "flaring", "process_cement",
             "electricity", "waste", "upstream_transport"]
    items = Dict("refrigerant" => "R410A", "ldar" => nothing, "flaring" => nothing,
                 "process_cement" => "clinker", "electricity" => nothing,
                 "waste" => "recycling", "upstream_transport" => "sea_container",
                 "natural_gas" => nothing)
    DataFrame(source_type=rules,
              chosen_factor=[resolve_factor(lib, SOURCE_RULES[r];
                                           item=get(items, r, nothing), region="GLOBAL",
                                           date=Date(2024, 12, 31)) |> f -> f === nothing ? "-" : f.ef_id
                           for r in rules])
end

```

```

Dict{Symbol, Any}(:rows => Any[(1, Any[("\natural_gas\\"", MIME type text/plain),
    (\"mobile_combustion_diesel_fleet_CO2e_GLOBAL_2026.1\\"", MIME type text/plain)]), (2,
Any[("\refrigerant\\"", MIME type text/plain), (\"refrigerant_R410A_CO2e_GLOBAL_2026.1\\"", MIME
type text/plain)]), (3, Any[("\ldar\\"", MIME type text/plain),
    (\"ldar_component_leak_CH4_GLOBAL_2026.1\\"", MIME type text/plain)]), (4, Any[("\flaring\\"",
MIME type text/plain), (\"flaring_flare_gas_CO2e_GLOBAL_2026.1\\"", MIME type text/plain)]), (5,
Any[("\process_cement\\"", MIME type text/plain), (\"-\\"", MIME type text/plain)]), (6,
Any[("\electricity\\"", MIME type text/plain),
    (\"electricity_grid_average_CO2e_GLOBAL_2026.1\\"", MIME type text/plain)]), (7,
Any[("\waste\\"", MIME type text/plain), (\"waste_recycling_CO2e_GLOBAL_2026.1\\"", MIME type
text/plain)]), (8, Any[("\upstream_transport\\"", MIME type text/plain),
    (\"upstream_transport_sea_container_CO2e_GLOBAL_2026.1\\"", MIME type text/plain)]), :objectid
=> "81a773d439bcd1c", :schema => Dict{Symbol, Any}(:names => Any["source_type",
    "chosen_factor"], :types => ["String", "String"]))

```

```

md"""
### Spend-based factors: from purchase-order text to an EEIO sector

`purchase → sector_hint → EF_EEIO`, with the spend deflated to the base year of the
factor set.
"""

```

Spend-based factors: from purchase-order text to an EEIO sector

`purchase → sector_hint → EF_EEIO`, with the spend deflated to the base year of the factor set.

```
begin
    texts = ["stainless steel plate", "facility management", "corrugated packaging",
             "cold-chain logistics", "agricultural fertiliser"]
    idx = PriceIndex(2024, Dict(2020 => 100.0, 2021 => 104.7, 2022 => 113.2, 2023 => 117.9,
                               2024 => 121.9))
    DataFrame(text=texts, sector=sector_hint(texts),
              spend_2023_deflated=[round(deflate(1_000_000, 2023, idx); digits=0) for _ in
text])
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\stainless steel plate\\"", MIME type text/plain),
("\steel\\"", MIME type text/plain), ("1.03393e6", MIME type text/plain)]), (2, Any[("\sfacility
management\\"", MIME type text/plain), ("\sservices\\"", MIME type text/plain), ("1.03393e6", MIME
type text/plain)]), (3, Any[("\scorrugated packaging\\"", MIME type text/plain), ("\sservices\\"",
MIME type text/plain), ("1.03393e6", MIME type text/plain)]), (4, Any[("\scold-chain
logistics\\"", MIME type text/plain), ("\stransport_services\\"", MIME type text/plain),
("1.03393e6", MIME type text/plain)]), (5, Any[("\sagricultural fertiliser\\"", MIME type
text/plain), ("\sagriculture\\"", MIME type text/plain), ("1.03393e6", MIME type text/plain)]),
:objectid => "217b87e60dafa785", :schema => Dict{Symbol, Any}(:names => Any["text", "sector",
"spend_2023_deflated"], :types => ["String", "String", "Float64"])]
```

```
md"""
### Every factor carries its own uncertainty and quality

That is what lets the uncertainty engine (notebook 10) propagate a defensible
budget instead of a blanket percentage.
"""
```

Every factor carries its own uncertainty and quality

That is what lets the uncertainty engine (notebook 10) propagate a *defensible* budget instead of a blanket percentage.

```
combine(groupby(factors_dataframe(lib), :category),
         :uncertainty => (x -> round(100 * mean(x); digits=1)) => :mean_u_r_pct, nrow => :records)

Dict{Symbol, Any}(:rows => Any[(1, Any[("\snatural_gas\\"", MIME type text/plain), ("34.3", MIME
type text/plain), ("3", MIME type text/plain)]), (2, Any[("\sfuel_oil\\"", MIME type text/plain),
("34.3", MIME type text/plain), ("3", MIME type text/plain)]), (3, Any[("\sdiesel_stationary\\"",
MIME type text/plain), ("34.0", MIME type text/plain), ("3", MIME type text/plain)]), (4,
Any[("\slpg\\"", MIME type text/plain), ("34.3", MIME type text/plain), ("3", MIME type
text/plain)]), (5, Any[("\scoal\\"", MIME type text/plain), ("21.0", MIME type text/plain), ("3",
MIME type text/plain)]), (6, Any[("\sbiomass\\"", MIME type text/plain), ("35.0", MIME type
text/plain), ("3", MIME type text/plain)]), (7, Any[("\sblast_furnace_gas\\"", MIME type
text/plain), ("35.0", MIME type text/plain), ("3", MIME type text/plain)]), (8,
Any[("\smobile_combustion\\"", MIME type text/plain), ("3.6", MIME type text/plain), ("5", MIME
type text/plain)]), (9, Any[("\selectricity\\"", MIME type text/plain), ("10.3", MIME type
text/plain), ("20", MIME type text/plain)]), (10, Any[("\sppa_electricity\\"", MIME type
text/plain), ("21.7", MIME type text/plain), ("3", MIME type text/plain)]), "more", (39,
Any[("\sinvestments\\"", MIME type text/plain), ("45.0", MIME type text/plain), ("17", MIME type
text/plain)]), :objectid => "f702b36910175450", :schema => Dict{Symbol, Any}(:names =>
Any["category", "mean_u_r_pct", "records"], :types => ["String", "Float64", "Int64"])]
```

```
begin
    out = joinpath(CarbonAccounting.builddir("notebook03"), "factors_snapshot.csv")
    export_factors(lib, out)
    reloaded = load_factors(out; name="snapshot")
    println("exported ", length(lib), " factors to ", out)
    println("reloaded ", length(reloaded), " factors, version ", reloaded.version)
    println("Notebook 03 – the factor set is versioned, provenance-documented, dimension-
checked and exportable.")
end
```

04_Scope1_Direct_Emissions

17 cells · executed in 20.59 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 04 · Scope 1 – direct emissions

Five formulas, each with its own physics, all returning `EmissionRow`s:


$$E_{\text{stat}} = \sum_i \dot{Q}_i / \eta_{\text{b}} \cdot (EF_{\text{CO}_2, i} \cdot OF_i + GWP_{\text{CH}_4} \cdot EF_{\text{CH}_4, i} + GWP_{\text{N}_2\text{O}} \cdot EF_{\text{N}_2\text{O}, i})$$


$$E_{\text{flare}} = \dot{V} \cdot EF \cdot DRE \cdot \eta_{\text{comb}} + GWP_{\text{CH}_4} \cdot \dot{V} \cdot \rho_{\text{CH}_4} \cdot x_{\text{CH}_4} \cdot (1 - DRE \cdot \eta_{\text{comb}})$$


$$E_{\text{fug}} = (C_{\text{start}} + C_{\text{purchased}} - C_{\text{end}} - C_{\text{recovered}} - C_{\text{disposed}}) \cdot GWP_{\text{CH}_4}$$


$$E_{\text{LDAR}} = N \cdot \text{leak\_rate} \cdot GWP_{\text{CH}_4}$$


$$E_{\text{proc}} = P \cdot EF \quad (\text{stoichiometry per tonne of product})$$


The engines never return a bare number: each row carries its factor id, version,
share @i, data-quality class and uncertainty.
"""
```

04 · Scope 1 — direct emissions

Five formulas, each with its own physics, all returning EmissionRows:

```
E_stat =  $\sum_i \dot{Q}_i / \eta_{\text{b}} \cdot (EF_{\text{CO}_2, i} \cdot OF_i + GWP_{\text{CH}_4} \cdot EF_{\text{CH}_4, i} + GWP_{\text{N}_2\text{O}} \cdot EF_{\text{N}_2\text{O}, i})$ 
E_flare =  $\dot{V} \cdot EF \cdot DRE \cdot \eta_{\text{comb}} + GWP_{\text{CH}_4} \cdot \dot{V} \cdot \rho_{\text{CH}_4} \cdot x_{\text{CH}_4} \cdot (1 - DRE \cdot \eta_{\text{comb}})$ 
E_fug =  $(C_{\text{start}} + C_{\text{purchased}} - C_{\text{end}} - C_{\text{recovered}} - C_{\text{disposed}}) \cdot GWP_{\text{CH}_4}$ 
E_LDAR =  $N \cdot \text{leak\_rate} \cdot GWP_{\text{CH}_4}$ 
E_proc =  $P \cdot EF$  (stoichiometry per tonne of product)
```

The engines never return a bare number: each row carries its factor id, version, share @_i, data-quality class and uncertainty.

```
md"""### Stationary combustion, gas by gas – oxidation factor and boiler efficiency"
```

Stationary combustion, gas by gas — oxidation factor and boiler efficiency

```
begin
     $\dot{Q}_i = 1.0e5$  # GJ of fuel input
    rows = [(fuel, combustion_emissions( $\dot{Q}_i$ ; EF_CO2=56.1, EF_CH4=1e-3, EF_N2O=1e-4,
                                         OF=1.0,  $\eta_b$ =0.9, fossil=(fuel != "biomass"))
            for fuel in ["natural_gas", "fuel_oil", "coal"]]
    DataFrame(fuel=[r[1] for r in rows], E=[r[2].E for r in rows],
             E_CO2=[r[2].E_CO2 for r in rows], E_CH4=[r[2].E_CH4 for r in rows],
             E_N2O=[r[2].E_N2O for r in rows])
end

Dict{Symbol, Any}(:rows => Any[(1, Any[("\natural_gas", MIME type text/plain), ("6239.68",
MIME type text/plain), ("6233.33", MIME type text/plain), ("0.111111", MIME type text/plain),
("0.0111111", MIME type text/plain))], (2, Any[("\fuel_oil", MIME type text/plain),
("6239.68", MIME type text/plain), ("6233.33", MIME type text/plain), ("0.111111", MIME type
text/plain), ("0.0111111", MIME type text/plain))], (3, Any[("\coal", MIME type text/plain),
("6239.68", MIME type text/plain), ("6233.33", MIME type text/plain), ("0.111111", MIME type
text/plain), ("0.0111111", MIME type text/plain))]), :objectid => "7c4a1e825afe889a", :schema =>
Dict{Symbol, Any}(:names => Any["fuel", "E", "E_CO2", "E_CH4", "E_N2O"], :types => ["String",
"Float64", "Float64", "Float64", "Float64"])]
```

```
md"""
### Flaring is not one multiplication

Only the fraction  $\phi = \text{DRE} \cdot \eta_{\text{comb}}$  of the gas is destroyed; the rest leaves as
**methane**, which at  $\text{GWP}_{100} = 29.8$  dominates the result.
"""
```

Flaring is not one multiplication

Only the fraction $\phi = \text{DRE} \cdot \eta_{\text{comb}}$ of the gas is destroyed; the rest leaves as **methane**, which at $\text{GWP}_{100} = 29.8$ dominates the result.

```
begin
     $\dot{V} = 1.0e6$  # m3(n) of flare gas in the year
    r = flare_emissions( $\dot{V}$ ; EF_flare=1.85, DRE=0.98,  $\eta_{\text{comb}}$ =0.995, x_CH4=0.8)
    (E=r.E, E_combusted=r.E_combusted, E_slip=r.E_slip,  $\phi$ =r. $\phi$ ,
     m_CH4_slip_kg=r.m_CH4_slip, note="slip is $(round(100*r.E_slip/r.E; digits=1)) % of the
total")
end

Dict{Symbol, Any}(:elements => Tuple[("E", ("2229.44", MIME type text/plain)), ("E_combusted",
("1803.93", MIME type text/plain)), ("E_slip", ("425.504", MIME type text/plain)), (" $\phi$ ",
("0.9751", MIME type text/plain)), ("m_CH4_slip_kg", ("14278.7", MIME type text/plain)), ("note",
("\slip is 19.1 % of the total", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"568cf91b926820bb")
```

```
md"""### Fugitive: refrigerant mass balance and the LDAR component count"
```

Fugitive: refrigerant mass balance and the LDAR component count

```
begin
    ref = fugitive_refrigerant(; C_start=1_200, C_purchased=140, C_end=1_305,
                                C_recovered=25, C_disposed=0, GWP_100=2088) # R410A
    ldar = fugitive_ldar(1_850; leak_rate=0.5)
    (refrigerant=ref, ldar=ldar,
     note="the mass balance finds $(ref.m_lost) kg lost; the LDAR estimate is an independent
view")
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("refrigerant", (Dict{Symbol, Any}(:elements => Tuple[("E",
("20.88", MIME type text/plain)), ("m_lost", ("10", MIME type text/plain))), :type =>
:NamedTuple, :objectid => "a72373558bbad30b"), MIME type application/vnd.pluto.tree+object)),
("ldar", (Dict{Symbol, Any}(:elements => Tuple[("E", ("27.565", MIME type text/plain)), ("m_CH4",
("925.0", MIME type text/plain))], :type => :NamedTuple, :objectid => "24936de7fcde8520"), MIME
type application/vnd.pluto.tree+object)), ("note", ("\"the mass balance finds 10 kg lost; the
LDAR estimate is an independent view\"", MIME type text/plain))], :type => :NamedTuple, :objectid
=> "2585f8caa942fef2")
```

```
md"### Process emissions: stoichiometry per tonne of product"
```

Process emissions: stoichiometry per tonne of product

```
begin
    cases = [("clinker", 42_000.0, 0.525, "CO2", "tCO₂/t"),
              ("lime", 12_000.0, 0.785, "CO2", "tCO₂/t"),
              ("nitric_acid", 3_100.0, 5.7, "N2O", "kgN₂O/t"),
              ("ammonia", 8_000.0, 1.90, "CO2e", "tCO₂e/t")]
    DataFrame(product=[c[1] for c in cases], production_t=[c[2] for c in cases],
              EF=[c[3] for c in cases], unit=[c[5] for c in cases],
              E_tCO₂e=[round(process_emissions(c[2]; EF=c[3], gas=c[4], unit=c[5]).E;
digits=1) for c in cases])
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\"clinker\"", MIME type text/plain), ("42000.0", MIME
type text/plain), ("0.525", MIME type text/plain), ("\"tCO₂/t\"", MIME type text/plain),
("22050.0", MIME type text/plain)]), (2, Any[("\"lime\"", MIME type text/plain), ("12000.0", MIME
type text/plain), ("0.785", MIME type text/plain), ("\"tCO₂/t\"", MIME type text/plain),
("9420.0", MIME type text/plain)]), (3, Any[("\"nitric_acid\"", MIME type text/plain), ("3100.0",
MIME type text/plain), ("5.7", MIME type text/plain), ("\"kgN₂O/t\"", MIME type text/plain),
("4823.9", MIME type text/plain)]), (4, Any[("\"ammonia\"", MIME type text/plain), ("8000.0",
MIME type text/plain), ("1.9", MIME type text/plain), ("\"tCO₂e/t\"", MIME type text/plain),
("15200.0", MIME type text/plain)]), :objectid => "2d989b5cb76b7798", :schema => Dict{Symbol,
Any}(:names => Any["product", "production_t", "EF", "unit", "E_tCO₂e"], :types => ["String",
"Float64", "Float64", "String", "Float64"])))
```

```
md"""
### The portfolio

`compute_scope1` routes every activity record to the right formula of the table
above (see the docstring of `scope1_row`), applies the consolidation share  $\Theta_i$  and
keeps `E_stat`, `E_mob`, `E_fug`, `E_flare`, `E_proc` apart.
"""
```

The portfolio

`compute_scope1` routes every activity record to the right formula of the table above (see the docstring of `scope1_row`), applies the consolidation share Θ_i and keeps `E_stat`, `E_mob`, `E_fug`, `E_flare`, `E_proc` apart.

```
begin
    records = ingest_files([datadir("raw", "activity_erp_2024.csv")];
                        entity_default="AcmeIndustrial_SA").records
    scope1 = compute_scope1(records; consolidation=:operational, date=Date(2024, 12, 31))
    scope1
end
```

```
Dict{Symbol, Any}(:prefix => "Scope1Result", :elements => Any[(:rows, (Dict{Symbol, Any}(:prefix
=> "CarbonAccounting.EmissionRow", :elements => Any[(1, ("activity_erp_2024.csv:2 | natural_gas
| Erdgas Kessel 1 | 30184.000 GJ | IPCC tier 1 (gas-specific, OF,  $\eta_b$ ) | 1883.3843
tCO2e | DQ2", MIME type text/plain)), (2, ("activity_erp_2024.csv:3 | diesel_fleet
Diesel Gerador | 5.595 m3 | average factor (CO2e) | 15.1054 tCO2e | DQ2", MIME
type text/plain)), (3, ("activity_erp_2024.csv:4 | natural_gas | Erdgas Kessel 1
| 27259.000 GJ | IPCC tier 1 (gas-specific, OF,  $\eta_b$ ) | 1700.8738 tCO2e | DQ2", MIME type
text/plain)), (4, ("activity_erp_2024.csv:5 | diesel_fleet | Diesel Gerador
6.416 m3 | average factor (CO2e) | 17.3219 tCO2e | DQ2", MIME type text/plain)), (5,
("activity_erp_2024.csv:6 | natural_gas | Erdgas Kessel 1 | 27010.000 GJ |
IPCC tier 1 (gas-specific, OF,  $\eta_b$ ) | 1685.3370 tCO2e | DQ2", MIME type text/plain)), (6,
("activity_erp_2024.csv:7 | diesel_fleet | Diesel Gerador | 5.481 m3 |
average factor (CO2e) | 14.7976 tCO2e | DQ2", MIME type text/plain)), (7,
("activity_erp_2024.csv:8 | natural_gas | Erdgas Kessel 1 | 22205.000 GJ |
IPCC tier 1 (gas-specific, OF,  $\eta_b$ ) | 1385.5205 tCO2e | DQ2", MIME type text/plain)), (8,
("activity_erp_2024.csv:9 | diesel_fleet | Diesel Gerador | 5.874 m3 |
average factor (CO2e) | 15.8586 tCO2e | DQ2", MIME type text/plain)), (9,
("activity_erp_2024.csv:10 | natural_gas | Erdgas Kessel 1 | 18750.000 GJ |
IPCC tier 1 (gas-specific, OF,  $\eta_b$ ) | 1169.9396 tCO2e | DQ2", MIME type text/plain)), "more",
(175, ("activity_erp_2024.csv:177 | natural_gas | Erdgas Kessel 1 | -8.000
GJ | IPCC tier 1 (gas-specific, OF,  $\eta_b$ ) | -0.4992 tCO2e | DQ2", MIME type text/plain))],
:type => :Array, :prefix_short => "", :objectid => "99b37eb3d67872f9"), MIME type
application/vnd.pluto.tree+object)), (:by_source, (Dict{Symbol, Any}(:prefix => "Dict{String,
Float64}", :elements => Any[(("E_mob", MIME type text/plain), ("993.904", MIME type
text/plain)), (("E_fug", MIME type text/plain), ("2011.78", MIME type text/plain)),
(("E_stat", MIME type text/plain), ("97225.2", MIME type text/plain)), (("E_proc", MIME
type text/plain), ("35849.4", MIME type text/plain)), (("E_flare", MIME type text/plain),
("38701.1", MIME type text/plain))], :type => :Dict, :prefix_short => "Dict", :objectid =>
"a57937df09245553"), MIME type application/vnd.pluto.tree+object)), (:E_Scope1,
("174781.39337479963", MIME type text/plain)), (:memo_biogenic, ("0.0", MIME type text/plain))],
:type => :struct, :prefix_short => "Scope1Result", :objectid => "91ede311025d6dfa")
```

```
scope1_detail(scope1) |> df -> df[1:min(8, nrow(df)), :]
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\activity_erp_2024.csv:2\"", MIME type text/plain),
("\SAO-01\"", MIME type text/plain), ("\natural_gas\"", MIME type text/plain), ("\E_stat\"",
MIME type text/plain), ("\Erdgas Kessel 1\"", MIME type text/plain), ("\IPCC tier 1 (gas-
specific, OF, η_b)\"", MIME type text/plain), ("30184.0", MIME type text/plain), ("\GJ\"", MIME
type text/plain), "more"]), (2, Any[("\activity_erp_2024.csv:3\"", MIME type text/plain),
("\SAO-01\"", MIME type text/plain), ("\diesel_fleet\"", MIME type text/plain), ("\E_mob\"",
MIME type text/plain), ("\Diesel Gerador\"", MIME type text/plain), ("\average factor
(CO₂e)\"", MIME type text/plain), ("5.595", MIME type text/plain), ("\m³\"", MIME type
text/plain), "more"]), (3, Any[("\activity_erp_2024.csv:4\"", MIME type text/plain), ("\SAO-
01\"", MIME type text/plain), ("\natural_gas\"", MIME type text/plain), ("\E_stat\"", MIME type
text/plain), ("\Erdgas Kessel 1\"", MIME type text/plain), ("\IPCC tier 1 (gas-specific, OF,
η_b)\"", MIME type text/plain), ("27259.0", MIME type text/plain), ("\GJ\"", MIME type
text/plain), "more"]), (4, Any[("\activity_erp_2024.csv:5\"", MIME type text/plain), ("\SAO-
01\"", MIME type text/plain), ("\diesel_fleet\"", MIME type text/plain), ("\E_mob\"", MIME type
text/plain), ("\Diesel Gerador\"", MIME type text/plain), ("\average factor (CO₂e)\"", MIME
type text/plain), ("6.416", MIME type text/plain), ("\m³\"", MIME type text/plain), "more"]),
(5, Any[("\activity_erp_2024.csv:6\"", MIME type text/plain), ("\SAO-01\"", MIME type
text/plain), ("\natural_gas\"", MIME type text/plain), ("\E_stat\"", MIME type text/plain),
("\Erdgas Kessel 1\"", MIME type text/plain), ("\IPCC tier 1 (gas-specific, OF, η_b)\"", MIME
type text/plain), ("27010.0", MIME type text/plain), ("\GJ\"", MIME type text/plain), "more"]),
(6, Any[("\activity_erp_2024.csv:7\"", MIME type text/plain), ("\SAO-01\"", MIME type
text/plain), ("\diesel_fleet\"", MIME type text/plain), ("\E_mob\"", MIME type text/plain),
("\Diesel Gerador\"", MIME type text/plain), ("\average factor (CO₂e)\"", MIME type
text/plain), ("5.481", MIME type text/plain), ("\m³\"", MIME type text/plain), "more"]), (7,
Any[("\activity_erp_2024.csv:8\"", MIME type text/plain), ("\SAO-01\"", MIME type text/plain),
("\natural_gas\"", MIME type text/plain), ("\E_stat\"", MIME type text/plain), ("\Erdgas
Kessel 1\"", MIME type text/plain), ("\IPCC tier 1 (gas-specific, OF, η_b)\"", MIME type
text/plain), ("22205.0", MIME type text/plain), ("\GJ\"", MIME type text/plain), "more"]), (8,
Any[("\activity_erp_2024.csv:9\"", MIME type text/plain), ("\SAO-01\"", MIME type text/plain),
("\diesel_fleet\"", MIME type text/plain), ("\E_mob\"", MIME type text/plain), ("\Diesel
Gerador\"", MIME type text/plain), ("\average factor (CO₂e)\"", MIME type text/plain), ("5.874",
MIME type text/plain), ("\m³\"", MIME type text/plain), "more"]]), :objectid =>
"13480ae094eb906c", :schema => Dict{Symbol, Any}(:names => Any["rec_id", "site", "source_type",
"bucket", "item", "method", "Q", "unit", "more"], :types => ["String", "String", "String",
"String", "String", "String", "Float64", "String", "more"]])
```

```
begin
    parts = sort(collect(scope1.by_source))
    plt = bar([p[1] for p in parts], [p[2] for p in parts], legend=false,
              title="Scope 1 by source category", ylabel="tCO₂e", color=:darkorange)
    save_figure(plt, "04_scope1_breakdown"; saver=savefig)
    plt
end
```



```

begin
    # data quality and uncertainty of each contributing sub-total
    df = emissions_dataframe(scope1.rows)
    sort(combine(groupby(df, :source_type),
                  :E => sum => :E, :DQ => (x -> round(mean(x); digits=2)) => :mean_DQ,
                  :u_rel => (x -> round(mean(x); digits=3)) => :mean_u_r), :E, rev=true)
end

```

```

Dict{Symbol, Any}(:rows => Any[(1, Any[("\natural_gas\\"", MIME type text/plain), ("97225.2",
MIME type text/plain), ("2.0", MIME type text/plain), ("0.708", MIME type text/plain)]), (2,
Any[("\flaring\\"", MIME type text/plain), ("38701.1", MIME type text/plain), ("2.0", MIME type
text/plain), ("0.254", MIME type text/plain)]), (3, Any[("\process_nitric_acid\\"", MIME type
text/plain), ("34892.4", MIME type text/plain), ("2.0", MIME type text/plain), ("0.402", MIME
type text/plain)]), (4, Any[("\refrigerant\\"", MIME type text/plain), ("1812.38", MIME type
text/plain), ("2.0", MIME type text/plain), ("0.109", MIME type text/plain)]), (5,
Any[("\diesel_fleet\\"", MIME type text/plain), ("993.904", MIME type text/plain), ("2.0", MIME
type text/plain), ("0.052", MIME type text/plain)]), (6, Any[("\process_cement\\"", MIME type
text/plain), ("956.923", MIME type text/plain), ("2.0", MIME type text/plain), ("0.058", MIME
type text/plain)]), (7, Any[("\ldar\\"", MIME type text/plain), ("199.392", MIME type
text/plain), ("4.0", MIME type text/plain), ("0.636", MIME type text/plain)]), :objectid =>
"6a3d95d55db1c08d", :schema => Dict{Symbol, Any}(:names => Any["source_type", "E", "mean_DQ",
"mean_u_r"], :types => ["String", "Float64", "Float64", "Float64"]))

```

```

begin
    println("E_Scope1          : ", round(scope1.E_Scope1; digits=1), " tCO₂e")
    println("biogenic memo    : ", round(scope1.memo_biogenic; digits=1), " tCO₂e (excluded
from the total)")
    println("rows              : ", length(scope1.rows))
    println("Notebook 04 - combustion, flaring, fugitive and process emissions are computed
by their own physics, not by one generic factor.")
end

```

05_Scope2_Purchased_Energy

15 cells · executed in 20.76 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 05 · Scope 2 – purchased electricity, steam, heat and cooling

The GHG Protocol requires the dual report:

    E_loc =  $\sum_i (C_i \cdot EF_{grid,y})$                                 location based
    E_mkt =  $\sum_i (C_i \cdot EF_{contract}) + C_{unclaimed} \cdot EF_{residual}$     market based

`compute_scope2` emits two rows per purchase – ‘location-based’ and
‘market-based’ – with the same activity, share  $\Theta_i$  and uncertainty, so both figures
decompose identically. The market factor comes from the contractual instrument
declared on the record (PPA, green tariff), otherwise the residual mix applies;
 $\lambda_{cov}$  states how much of the consumption instruments cover at all.
"""
```

05 · Scope 2 — purchased electricity, steam, heat and cooling

The GHG Protocol requires the **dual report**:

$$E_{loc} = \sum_i (C_i \cdot EF_{grid,y}) \quad \text{location based}$$
$$E_{mkt} = \sum_i (C_i \cdot EF_{contract}) + C_{unclaimed} \cdot EF_{residual} \quad \text{market based}$$

`compute_scope2` emits *two rows per purchase* — location-based and market-based — with the same activity, share Θ_i and uncertainty, so both figures decompose identically. The market factor comes from the contractual instrument declared on the record (PPA, green tariff), otherwise the residual mix applies; λ_{cov} states how much of the consumption instruments cover at all.

```
md"""### Where the market-based factor comes from – the contractual hierarchy"
```

Where the market-based factor comes from — the contractual hierarchy

```
begin
    lib = default_library()
    ppa = resolve_factor(lib, SOURCE_RULES["ppa_electricity"]; item="ppa_renewable",
        region="GLOBAL", date=Date(2024, 12, 31)).value
    tariff = resolve_factor(lib, SOURCE_RULES["ppa_electricity"]; item="green_tariff",
        region="GLOBAL", date=Date(2024, 12, 31)).value
    resid = resolve_factor(lib, SOURCE_RULES["electricity"]; region="GLOBAL",
        date=Date(2024, 12, 31), market=:market).value
    DataFrame(instrument=["PPA, renewable", "Green tariff with certificates",
        "No instrument → residual mix"],
        factor=[ppa, tariff, resid], unit="kgCO₂e/kWh")
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\PPA, renewable\\"", MIME type text/plain), ("0.0", MIME
type text/plain), ("\kgCO₂e/kWh\\"", MIME type text/plain)]), (2, Any[("\Green tariff with
certificates\\"", MIME type text/plain), ("0.05", MIME type text/plain), ("\kgCO₂e/kWh\\"", MIME
type text/plain)]), (3, Any[("\No instrument → residual mix\\"", MIME type text/plain), ("0.62",
MIME type text/plain), ("\kgCO₂e/kWh\\"", MIME type text/plain)]), :objectid =>
"761fbc287e03942", :schema => Dict{Symbol, Any}(:names => Any["instrument", "factor", "unit"],
:types => ["String", "Float64", "String"])]
```

```
md"### Location-based grid intensity versus residual mix, by country"
```

Location-based grid intensity versus residual mix, by country

```
begin
    countries = ["BRA", "DEU", "ZAF", "CHN", "GLOBAL"]
    loc = [resolve_factor(lib, SOURCE_RULES["electricity"]; region=c,
        date=Date(2024, 12, 31)).value for c in countries]
    res = [resolve_factor(lib, SOURCE_RULES["electricity"]; region=c,
        date=Date(2024, 12, 31), market=:market).value for c in countries]
    plt_grid = bar(countries, [loc res], bar_position=:dodge,
        label=["location based" "residual mix (market)"],
        title="Grid intensity vs residual mix", ylabel="kgCO₂e/kWh")
    save_figure(plt_grid, "05_grid_vs_residual"; saver=savefig)
    DataFrame(country=countries, EF_grid=loc, EF_residual=res,
        premium_pct=[round(100 * (r / l - 1); digits=1) for (l, r) in zip(loc, res)])
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\BRA\\"", MIME type text/plain), ("0.087", MIME type
text/plain), ("0.15", MIME type text/plain), ("72.4", MIME type text/plain)]), (2,
Any[("\DEU\\"", MIME type text/plain), ("0.38", MIME type text/plain), ("0.569", MIME type
text/plain), ("49.7", MIME type text/plain)]), (3, Any[("\ZAF\\"", MIME type text/plain),
("0.95", MIME type text/plain), ("1.1", MIME type text/plain), ("15.8", MIME type text/plain)]),
(4, Any[("\CHN\\"", MIME type text/plain), ("0.58", MIME type text/plain), ("0.79", MIME type
text/plain), ("36.2", MIME type text/plain)]), (5, Any[("\GLOBAL\\"", MIME type text/plain),
("0.436", MIME type text/plain), ("0.62", MIME type text/plain), ("42.2", MIME type
text/plain)]), :objectid => "bb5b2e6fad4036e7", :schema => Dict{Symbol, Any}(:names =>
Any["country", "EF_grid", "EF_residual", "premium_pct"], :types => ["String", "Float64",
"Float64", "Float64"])]
```

```
md"### The purchases of the portfolio – mixed units in one file, instruments per month"
```

The purchases of the portfolio — mixed units in one file, instruments per month

```
begin
    records = ingest_files([datadir("raw", "electricity_utility_2024.csv")];
                          entity_default="AcmeIndustrial_SA").records
    df = records_dataframe(records)
    (purchases=nrow(df), by_unit=combine(groupby(df, [:unit, :unit_canon]), nrow => :n),
     instruments=unique([r.meta["instrument"] for r in records]))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("purchases", ("72", MIME type text/plain)), ("by_unit",
(Dict{Symbol, Any}(:rows => Any[(1, Any[("\MWh\\"", MIME type text/plain), ("\GJ\\"", MIME type
text/plain), ("39", MIME type text/plain)]), (2, Any[("\kWh\\"", MIME type text/plain),
("\GJ\\"", MIME type text/plain), ("33", MIME type text/plain)])], :objectid =>
"bf43ad96a382e471", :schema => Dict{Symbol, Any}(:names => Any["unit", "unit_canon", "n"], :types
=> ["String", "String", "Int64"])), MIME type application/vnd.pluto.table+object)),
("instruments", (Dict{Symbol, Any}(:prefix => "String", :elements => Tuple{Int64, Any}[(1,
("\", MIME type text/plain)), (2, ("\ppa_renewable\\"", MIME type text/plain)), (3,
("\green_tariff\\"", MIME type text/plain)]), :type => :Array, :prefix_short => "", :objectid =>
"637b32ea26351581"), MIME type application/vnd.pluto.tree+object))), :type => :NamedTuple,
:objectid => "dad53e192e3e3732")
```

```
md"### The dual report"
```

The dual report

```
compute_scope2(records; date=Date(2024, 12, 31), consolidation=:operational)
```

```
Dict{Symbol, Any}(:prefix => "Scope2Result", :elements => Any[(:rows, (Dict{Symbol, Any}(:prefix
=> "CarbonAccounting.EmissionRow", :elements => Any[(1, ("electricity_utility_2024.csv:2 |
electricity | purchased electricity | 11052.000 GJ | location-based | 267.0900
tCO₂e | DQ2", MIME type text/plain)), (2, ("electricity_utility_2024.csv:2 | electricity
| purchased electricity | 11052.000 GJ | market-based | 460.5000 tCO₂e | DQ2", MIME type
text/plain)), (3, ("electricity_utility_2024.csv:3 | electricity | purchased
electricity | 11786.400 GJ | location-based | 284.8380 tCO₂e | DQ2", MIME type
text/plain)), (4, ("electricity_utility_2024.csv:3 | electricity | purchased
electricity | 11786.400 GJ | market-based | 491.1000 tCO₂e | DQ2", MIME type
text/plain)), (5, ("electricity_utility_2024.csv:4 | electricity | purchased
electricity | 9636.566 GJ | location-based | 232.8837 tCO₂e | DQ2", MIME type
text/plain)), (6, ("electricity_utility_2024.csv:4 | electricity | purchased
electricity | 9636.566 GJ | market-based | 401.5236 tCO₂e | DQ2", MIME type
text/plain)), (7, ("electricity_utility_2024.csv:5 | electricity | purchased
electricity | 8963.233 GJ | location-based | 216.6115 tCO₂e | DQ2", MIME type
text/plain)), (8, ("electricity_utility_2024.csv:5 | electricity | purchased
electricity | 8963.233 GJ | market-based | 0.0000 tCO₂e | DQ2", MIME type
text/plain)), (9, ("electricity_utility_2024.csv:6 | electricity | purchased
electricity | 7944.642 GJ | location-based | 191.9955 tCO₂e | DQ2", MIME type
text/plain)), "more", (144, ("electricity_utility_2024.csv:73 | electricity
| purchased electricity | 12679.200 GJ | market-based | 2782.3800 tCO₂e | DQ2", MIME type
text/plain))], :type => :Array, :prefix_short => "", :objectid => "457170456a9d5af4"), MIME type
application/vnd.pluto.tree+object)), (:E_loc, ("73433.264921", MIME type text/plain)), (:E_mkt,
("86683.22791700001", MIME type text/plain)), (:C_elec, ("166869.63499999998", MIME type
text/plain)), (:λ_cov, ("0.19551940051885416", MIME type text/plain)), (:by_site, (Dict{Symbol,
Any}(:rows => Any[(1, Any[("\SAO-01\\"", MIME type text/plain), ("\BRA\\"", MIME type
text/plain), ("1.06196e5", MIME type text/plain), ("2566.4", MIME type text/plain), ("1691.82",
MIME type text/plain)]), (2, Any[("\SAO-02\\"", MIME type text/plain), ("\BRA\\"", MIME type
text/plain), ("1.04235e5", MIME type text/plain), ("2519.02", MIME type text/plain), ("4343.14",
MIME type text/plain)]), (3, Any[("\SHA-03\\"", MIME type text/plain), ("\CHN\\"", MIME type
text/plain), ("1.18108e5", MIME type text/plain), ("19028.5", MIME type text/plain), ("25918.1",
MIME type text/plain)]), (4, Any[("\HAM-01\\"", MIME type text/plain), ("\DEU\\"", MIME type
text/plain), ("62531.5", MIME type text/plain), ("6600.54", MIME type text/plain), ("3032.26",
MIME type text/plain)]), (5, Any[("\ROT-02\\"", MIME type text/plain), ("\NLD\\"", MIME type
text/plain), ("72973.6", MIME type text/plain), ("6648.71", MIME type text/plain), ("9932.52",
MIME type text/plain)]), (6, Any[("\JNB-01\\"", MIME type text/plain), ("\ZAF\\"", MIME type
text/plain), ("1.36687e5", MIME type text/plain), ("36070.1", MIME type text/plain), ("41765.4",
MIME type text/plain)]), :objectid => "c2d5f5784bc5e164", :schema => Dict{Symbol, Any}(:names =>
Any["site", "country", "activity", "E_loc", "E_mkt"], :types => ["String", "String", "Float64",
"Float64", "Float64"])), MIME type application/vnd.pluto.table+object))), :type => :struct,
:prefix_short => "Scope2Result", :objectid => "5883b03c17a1f70f")
```

```

begin
    scope2 = compute_scope2(records; date=Date(2024, 12, 31), consolidation=:operational)
    detail = scope2_detail(scope2)
    first(select(detail, :site, :item, :method, :Q, :unit, :ef_id, :ef_value, :ef_unit, :E,
:DQ), 8)
end

```

```

Dict{Symbol, Any}(:rows => Any[[1, Any[("\SAO-01\\"", MIME type text/plain), ("\purchased
electricity\\"", MIME type text/plain), ("\location-based\\"", MIME type text/plain), ("11052.0",
MIME type text/plain), ("\GJ\\"", MIME type text/plain),
("\electricity_grid_average_CO2e_BRA_2026.1\\"", MIME type text/plain), ("0.087", MIME type
text/plain), ("\kgCO2e/kWh\\"", MIME type text/plain), ("267.09", MIME type text/plain), ("2",
MIME type text/plain)]], (2, Any[("\SAO-01\\"", MIME type text/plain), ("\purchased
electricity\\"", MIME type text/plain), ("\market-based\\"", MIME type text/plain), ("11052.0",
MIME type text/plain), ("\GJ\\"", MIME type text/plain),
("\electricity_residual_mix_CO2e_BRA_2026.1\\"", MIME type text/plain), ("0.15", MIME type
text/plain), ("\kgCO2e/kWh\\"", MIME type text/plain), ("460.5", MIME type text/plain), ("2",
MIME type text/plain)]], (3, Any[("\SAO-01\\"", MIME type text/plain), ("\purchased
electricity\\"", MIME type text/plain), ("\location-based\\"", MIME type text/plain), ("11786.4",
MIME type text/plain), ("\GJ\\"", MIME type text/plain),
("\electricity_grid_average_CO2e_BRA_2026.1\\"", MIME type text/plain), ("0.087", MIME type
text/plain), ("\kgCO2e/kWh\\"", MIME type text/plain), ("284.838", MIME type text/plain), ("2",
MIME type text/plain)]], (4, Any[("\SAO-01\\"", MIME type text/plain), ("\purchased
electricity\\"", MIME type text/plain), ("\market-based\\"", MIME type text/plain), ("11786.4",
MIME type text/plain), ("\GJ\\"", MIME type text/plain),
("\electricity_residual_mix_CO2e_BRA_2026.1\\"", MIME type text/plain), ("0.15", MIME type
text/plain), ("\kgCO2e/kWh\\"", MIME type text/plain), ("491.1", MIME type text/plain), ("2",
MIME type text/plain)]], (5, Any[("\SAO-01\\"", MIME type text/plain), ("\purchased
electricity\\"", MIME type text/plain), ("\location-based\\"", MIME type text/plain), ("9636.57",
MIME type text/plain), ("\GJ\\"", MIME type text/plain),
("\electricity_grid_average_CO2e_BRA_2026.1\\"", MIME type text/plain), ("0.087", MIME type
text/plain), ("\kgCO2e/kWh\\"", MIME type text/plain), ("232.884", MIME type text/plain), ("2",
MIME type text/plain)]], (6, Any[("\SAO-01\\"", MIME type text/plain), ("\purchased
electricity\\"", MIME type text/plain), ("\market-based\\"", MIME type text/plain), ("9636.57",
MIME type text/plain), ("\GJ\\"", MIME type text/plain),
("\electricity_residual_mix_CO2e_BRA_2026.1\\"", MIME type text/plain), ("0.15", MIME type
text/plain), ("\kgCO2e/kWh\\"", MIME type text/plain), ("401.524", MIME type text/plain), ("2",
MIME type text/plain)]], (7, Any[("\SAO-01\\"", MIME type text/plain), ("\purchased
electricity\\"", MIME type text/plain), ("\location-based\\"", MIME type text/plain), ("8963.23",
MIME type text/plain), ("\GJ\\"", MIME type text/plain),
("\electricity_grid_average_CO2e_BRA_2026.1\\"", MIME type text/plain), ("0.087", MIME type
text/plain), ("\kgCO2e/kWh\\"", MIME type text/plain), ("216.611", MIME type text/plain), ("2",
MIME type text/plain)]], (8, Any[("\SAO-01\\"", MIME type text/plain), ("\purchased
electricity\\"", MIME type text/plain), ("\market-based\\"", MIME type text/plain), ("8963.23",
MIME type text/plain), ("\GJ\\"", MIME type text/plain),
("\ppa_electricity_ppa_renewable_CO2e_GLOBAL_2026.1\\"", MIME type text/plain), ("0.0", MIME type
text/plain), ("\kgCO2e/kWh\\"", MIME type text/plain), ("0.0", MIME type text/plain), ("2", MIME
type text/plain)]]], :objectid => "ee392691d4545a3d", :schema => Dict{Symbol, Any}(:names =>
Any["site", "item", "method", "Q", "unit", "ef_id", "ef_value", "ef_unit", "E", "DQ"], :types =>
["String", "String", "String", "Float64", "String", "String", "Float64", "String", "Float64",
"Int64"])]))

```

```
begin
    per_site = combine(groupby(scope2_detail(scope2), [:site, :method]),
                        :Q => sum => :activity_GJ, :E => sum => :E)
    sort!(per_site, [:site, :method])
    per_site
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\HAM-01\\"", MIME type text/plain), (\"location-
based\\"", MIME type text/plain), (\"62531.5\", MIME type text/plain), (\"6600.54\", MIME type
text/plain)]), (2, Any[("\HAM-01\\"", MIME type text/plain), (\"market-based\\"", MIME type
text/plain), (\"62531.5\", MIME type text/plain), (\"3032.26\", MIME type text/plain)]), (3,
Any[("\JNB-01\\"", MIME type text/plain), (\"location-based\\"", MIME type text/plain),
(\"1.36687e5\", MIME type text/plain), (\"36070.1\", MIME type text/plain)]), (4, Any[("\JNB-01\\"",
MIME type text/plain), (\"market-based\\"", MIME type text/plain), (\"1.36687e5\", MIME type
text/plain), (\"41765.4\", MIME type text/plain)]), (5, Any[("\ROT-02\\"", MIME type text/plain),
(\"location-based\\"", MIME type text/plain), (\"72973.6\", MIME type text/plain), (\"6648.71\", MIME
type text/plain)]), (6, Any[("\ROT-02\\"", MIME type text/plain), (\"market-based\\"", MIME type
text/plain), (\"72973.6\", MIME type text/plain), (\"9932.52\", MIME type text/plain)]), (7,
Any[("\SAO-01\\"", MIME type text/plain), (\"location-based\\"", MIME type text/plain),
(\"1.06196e5\", MIME type text/plain), (\"2566.4\", MIME type text/plain)]), (8, Any[("\SAO-01\\"",
MIME type text/plain), (\"market-based\\"", MIME type text/plain), (\"1.06196e5\", MIME type
text/plain), (\"1691.82\", MIME type text/plain)]), (9, Any[("\SAO-02\\"", MIME type text/plain),
(\"location-based\\"", MIME type text/plain), (\"1.04235e5\", MIME type text/plain), (\"2519.02\",
MIME type text/plain)]), (10, Any[("\SAO-02\\"", MIME type text/plain), (\"market-based\\"", MIME
type text/plain), (\"1.04235e5\", MIME type text/plain), (\"4343.14\", MIME type text/plain)]), (11,
Any[("\SHA-03\\"", MIME type text/plain), (\"location-based\\"", MIME type text/plain),
(\"1.18108e5\", MIME type text/plain), (\"19028.5\", MIME type text/plain)]), (12, Any[("\SHA-03\\"",
MIME type text/plain), (\"market-based\\"", MIME type text/plain), (\"1.18108e5\", MIME type
text/plain), (\"25918.1\", MIME type text/plain)]], :objectid => \"949b1e160102e068\", :schema =>
Dict{Symbol, Any}(:names => Any[\"site\", \"method\", \"activity_GJ\", \"E\"], :types => [\"String\",
\"String\", \"Float64\", \"Float64\"])
```

```
md\"### Dual reporting is not a formality: the two figures answer different questions\"
```

Dual reporting is not a formality: the two figures answer different questions

```
begin
    plt_dual = bar([\"location based\", \"market based\"],
                    reshape([scope2.E_loc, scope2.E_mkt], 2, 1),
                    bar_position=:dodge,
                    label="", legend=false, ylabel=\"tCO₂e\",
                    title=\"Scope 2 of the portfolio (operational control)\",
                    color=[:seagreen :darkseagreen])
    save_figure(plt_dual, \"05_scope2_dual\"; saver=:savefig)
    println(\"E_Scope2 location : \", round(scope2.E_loc; digits=1), \" tCO₂e\")
    println(\"E_Scope2 market : \", round(scope2.E_mkt; digits=1), \" tCO₂e\")
    println(\"purchased electricity : \", round(scope2.C_elec; digits=1), \" MWh\")
    println(\"contractual coverage λ_cov : \", round(100 * scope2.λ_cov; digits=1), \" %\")
    println(\"Notebook 05 – both Scope 2 figures come from the same activities, so their
difference is attributable.\")
end
```

06_Scope3_Value_Chain

17 cells · executed in 21.15 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end

datadir (generic function with 1 method)
```

```
md"""
# 06 · Scope 3 – value chain

Scope 3 is where an inventory is made or lost: 15 categories, most of them outside
the company's own meters. The engine applies the hybrid data hierarchy of the
GHG Protocol:

| method | activity | factor | used for |
|---|---|---|---|
| supplier specific | supplier-reported | supplier factor | DQ 1 |
| average data | mass, energy, t·km, p·km | industry average | DQ 2–3 |
| spend based | deflated spend | EEIO factor | DQ 4, screening |

Screening is deliberate: the Pareto hotspots carrying ~80 % of the total are exactly
the ones that must be refined with better data – and the engine names them.
"""
```

06 · Scope 3 — value chain

Scope 3 is where an inventory is made or lost: 15 categories, most of them outside the company's own meters. The engine applies the **hybrid data hierarchy** of the GHG Protocol:

method	activity	factor	used for
supplier specific	supplier-reported	supplier factor	DQ 1
average data	mass, energy, t·km, p·km	industry average	DQ 2–3
spend based	deflated spend	EEIO factor	DQ 4, screening

Screening is deliberate: the Pareto hotspots carrying ~80 % of the total are exactly the ones that must be refined with better data — and the engine names them.

```
md"""### The 15 categories and the activity each one expects"
```

The 15 categories and the activity each one expects

```
begin
    rows = [(c, SCOPE3_CATEGORY_NAMES[c], r.key, string(r.dim), r.unit_hint)
             for r in values(SOURCE_RULES) for c in [r.cat]
             if r.scope == :scope3 && c != nothing]
    sort!(DataFrame(cat=[r[1] for r in rows], category=[r[2] for r in rows],
                    source_type=[r[3] for r in rows], dimension=[r[4] for r in rows],
                    unit=[r[5] for r in rows]), :cat)
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("1", MIME type text/plain), ("1 Purchased goods and
services", MIME type text/plain), ("purchased_goods_spend", MIME type text/plain),
("currency", MIME type text/plain), ("USD2024", MIME type text/plain)]), (2, Any[("1",
MIME type text/plain), ("1 Purchased goods and services", MIME type text/plain),
("purchased_goods_mass", MIME type text/plain), ("mass", MIME type text/plain), ("t",
MIME type text/plain)]), (3, Any[("2", MIME type text/plain), ("2 Capital goods", MIME type
text/plain), ("capital_goods", MIME type text/plain), ("currency", MIME type text/plain),
("USD2024", MIME type text/plain)]), (4, Any[("3", MIME type text/plain), ("3 Fuel- and
energy-related activities", MIME type text/plain), ("fuel_energy_upstream", MIME type
text/plain), ("energy", MIME type text/plain), ("GJ", MIME type text/plain)]), (5,
Any[("4", MIME type text/plain), ("4 Upstream transportation and distribution", MIME type
text/plain), ("upstream_transport", MIME type text/plain), ("transport_work", MIME type
text/plain), ("t.km", MIME type text/plain)]), (6, Any[("5", MIME type text/plain), ("5
Waste generated in operations", MIME type text/plain), ("waste", MIME type text/plain),
("mass", MIME type text/plain), ("t", MIME type text/plain)]), (7, Any[("6", MIME type
text/plain), ("6 Business travel", MIME type text/plain), ("business_travel_air", MIME
type text/plain), ("transport_work", MIME type text/plain), ("p.km", MIME type
text/plain)]), (8, Any[("6", MIME type text/plain), ("6 Business travel", MIME type
text/plain), ("business_travel_land", MIME type text/plain), ("transport_work", MIME type
text/plain), ("p.km", MIME type text/plain)]), (9, Any[("7", MIME type text/plain), ("7
Employee commuting", MIME type text/plain), ("employee_commuting", MIME type text/plain),
("transport_work", MIME type text/plain), ("p.km", MIME type text/plain)]), (10,
Any[("8", MIME type text/plain), ("8 Upstream leased assets", MIME type text/plain),
("upstream_leased", MIME type text/plain), ("currency", MIME type text/plain),
("USD2024", MIME type text/plain)]), "more", (17, Any[("15", MIME type text/plain), ("15
Investments", MIME type text/plain), ("investments", MIME type text/plain),
("currency", MIME type text/plain), ("USD2024", MIME type text/plain)]), :objectid =>
"75776a732ed7c257", :schema => Dict{Symbol, Any}(:names => Any["cat", "category", "source_type",
"dimension", "unit"], :types => ["Int64", "String", "String", "String", "String"])))
```

```
md"""
### Spend before deflation is not spend after it
```

```
An EEIO factor is expressed per unit of spend in its own base year, so a 2023
invoice is deflated to USD2024 before it meets the factor:
`s_spend = amount · CPI(base) / CPI(year)`.
"""
```

Spend before deflation is not spend after it

An EEIO factor is expressed per unit of spend *in its own base year*, so a 2023 invoice is deflated to USD2024 before it meets the factor: $s_spend = amount \cdot CPI(base) / CPI(year)$.

```
begin
    cpi = CSV.read(datadir("reference", "price_index.csv"), DataFrame)
    idx = PriceIndex(2024, Dict{Int{r.year} => Float64(r.cpi_usd) for r in eachrow(cpi)})
    DataFrame(year=cpi.year, cpi_usd=cpi.cpi_usd,
              in_USD2024_of_1M=[round(deflate(1.0e6, Int{r.year}, idx); digits=0) for r in
eachrow(cpi)])
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("2020", MIME type text/plain), ("100.0", MIME type
text/plain), ("1.219e6", MIME type text/plain)]), (2, Any[("2021", MIME type text/plain),
("104.7", MIME type text/plain), ("1.16428e6", MIME type text/plain)]), (3, Any[("2022", MIME
type text/plain), ("113.2", MIME type text/plain), ("1.07686e6", MIME type text/plain)]), (4,
Any[("2023", MIME type text/plain), ("117.9", MIME type text/plain), ("1.03393e6", MIME type
text/plain)]), (5, Any[("2024", MIME type text/plain), ("121.9", MIME type text/plain), ("1.0e6",
MIME type text/plain)]), (6, Any[("2025", MIME type text/plain), ("125.0", MIME type text/plain),
("975200.0", MIME type text/plain)]), (7, Any[("2026", MIME type text/plain), ("128.2", MIME type
text/plain), ("950858.0", MIME type text/plain)]), :objectid => "b04dfd7ae7fb2fe", :schema =>
Dict{Symbol, Any}(:names => Any["year", "cpi_usd", "in_USD2024_of_1M"], :types => ["Int64",
"Float64", "Float64"])]
```

```
md"### Two physical methods side by side: distance-based freight and waste treatment"
```

Two physical methods side by side: distance-based freight and waste treatment

```
begin
    lib = default_library()
    hgv = resolve_factor(lib, SOURCE_RULES["upstream_transport"]; item="hgv_diesel",
                        date=Date(2024, 12, 31))
    waste = resolve_factor(lib, SOURCE_RULES["waste"]; item="landfill_msw",
                        date=Date(2024, 12, 31))
    (Freight_E=emission(q"4.0e6 t·km", Quantity(hgv.value, hgv.unit)).val,
     freight_factor=describe_factor(hgv),
     Landfill_E=emission(q"620 t", Quantity(waste.value, waste.unit)).val,
     waste_factor=describe_factor(waste))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("Freight_E", ("428.0", MIME type text/plain)),
("freight_factor", ("\"hgv_diesel = 0.107 kgCO₂e/t·km [CarbonAccounting built-in 2026.1, GLOBAL,
DQ3, u_r=15%, DEFRA 2023 HGV (average laden)]\"", MIME type text/plain)), ("Landfill_E",
("289.54", MIME type text/plain)), ("waste_factor", ("\"landfill_msw = 0.467 kgCO₂e/kg
[CarbonAccounting built-in 2026.1, GLOBAL, DQ3, u_r=35%, US EPA WARM / DEFRA 2023 landfill]\"",
MIME type text/plain))], :type => :NamedTuple, :objectid => "90f3cf4eb337494e")
```

```
md"### From purchase-order text to an EEIO factor"
```

From purchase-order text to an EEIO factor

```

begin
    po = ["stainless steel plate (2 t)", "facility management Q3", "corrugated packaging",
          "cold-chain logistics", "IT consulting"]
    spend = [420_000.0, 96_000.0, 72_000.0, 210_000.0, 140_000.0]
    ef(s) = resolve_factor(lib, SOURCE_RULES["purchased_goods_spend"]; item=s,
                           date=Date(2024, 12, 31)).value
    DataFrame(purchase=po, sector=sector_hint(po), spend_USD2024=spend,
              EF_kg_per_USD=[ef(sector_hint(p)) for p in po],
              E_tCO₂e=[emission(Quantity(s, "USD2024"), Quantity(ef(sector_hint(p)),
                           "kgCO₂e/USD2024")).val for (p, s) in zip(po, spend)])
end

Dict{Symbol, Any}(:rows => Any[(1, Any[("\stainless steel plate (2 t)\", MIME type text/plain),
("\steel\", MIME type text/plain), ("420000.0", MIME type text/plain), ("0.85", MIME type
text/plain), ("357.0", MIME type text/plain)]), (2, Any[("\facility management Q3\", MIME type
text/plain), ("\services\", MIME type text/plain), ("96000.0", MIME type text/plain), ("0.035",
MIME type text/plain), ("3.36", MIME type text/plain)]), (3, Any[("\corrugated packaging\",
MIME type text/plain), ("\services\", MIME type text/plain), ("72000.0", MIME type text/plain),
("0.035", MIME type text/plain), ("2.52", MIME type text/plain)]), (4, Any[("\cold-chain
logistics\", MIME type text/plain), ("\transport_services\", MIME type text/plain),
("210000.0", MIME type text/plain), ("0.35", MIME type text/plain), ("73.5", MIME type
text/plain)]), (5, Any[("\IT consulting\", MIME type text/plain), ("\services\", MIME type
text/plain), ("140000.0", MIME type text/plain), ("0.035", MIME type text/plain), ("4.9", MIME
type text/plain)]), :objectid => "a619a39e0ad03748", :schema => Dict{Symbol, Any}(:names =>
Any["purchase", "sector", "spend_USD2024", "EF_kg_per_USD", "E_tCO₂e"], :types => ["String",
"String", "Float64", "Float64", "Float64"]))

```

```
md"### The value chain of the portfolio"
```

The value chain of the portfolio

```

begin
    files = ["procurement_2024.csv", "logistics_travel_2024.csv", "waste_2024.csv",
            "activity_erp_2024.csv", "electricity_utility_2024.csv"]
    records = ingest_files([datadir("raw", f) for f in files];
                          entity_default="AcmeIndustrial_SA").records
    scope3 = compute_scope3(records; date=Date(2024, 12, 31), consolidation=:operational,
                          hotspot_share=0.8)

    scope3
end

```

```

Dict{Symbol, Any}(:prefix => "Scope3Result", :elements => Any[(:rows, Dict{Symbol, Any}(:prefix
=> "CarbonAccounting.EmissionRow", :elements => Any[(1, ("procurement_2024.csv:2 |
purchased_goods_spend | corrugated packaging | 21119.600 USD2024 | spend based (EEIO,
deflated) | 0.7392 tCO2e | DQ4", MIME type text/plain)), (2, ("procurement_2024.csv:3 |
purchased_goods_spend | IT consulting | 206536.000 USD2024 | spend based (EEIO,
deflated) | 7.2288 tCO2e | DQ4", MIME type text/plain)), (3, ("procurement_2024.csv:4 |
purchased_goods_spend | facility management | 55172.700 USD2024 | spend based (EEIO,
deflated) | 1.9310 tCO2e | DQ4", MIME type text/plain)), (4, ("procurement_2024.csv:5 |
purchased_goods_spend | stainless steel plate | 1033649.100 USD2024 | spend based (EEIO,
deflated) | 878.6017 tCO2e | DQ4", MIME type text/plain)), (5, ("procurement_2024.csv:6 |
purchased_goods_spend | industrial chemicals | 38589.460 USD2024 | spend based (EEIO,
deflated) | 16.2076 tCO2e | DQ4", MIME type text/plain)), (6, ("procurement_2024.csv:7 |
purchased_goods_spend | cement bulk | 132752.000 USD2024 | spend based (EEIO,
deflated) | 86.2888 tCO2e | DQ4", MIME type text/plain)), (7, ("procurement_2024.csv:8 |
purchased_goods_spend | cement bulk | 212188.900 USD2024 | spend based (EEIO,
deflated) | 137.9228 tCO2e | DQ4", MIME type text/plain)), (8, ("procurement_2024.csv:9 |
purchased_goods_spend | polymer resin | 86315.600 USD2024 | spend based (EEIO,
deflated) | 47.4736 tCO2e | DQ4", MIME type text/plain)), (9, ("procurement_2024.csv:10 |
purchased_goods_spend | industrial chemicals | 43325.940 USD2024 | spend based (EEIO,
deflated) | 18.1969 tCO2e | DQ4", MIME type text/plain)), "more", (126, ("waste_2024.csv:19 |
waste | incineration | 268.000 t | average data (industry) |
5.6280 tCO2e | DQ3", MIME type text/plain))], :type => :Array, :prefix_short => "", :objectid =>
"68b4aeaa669fa722"), MIME type application/vnd.pluto.tree+object)), (:by_category, (Dict{Symbol,
Any}(:rows => Any[(1, Any[("1", MIME type text/plain), ("\"1 Purchased goods and services\"",
MIME type text/plain), ("4162.14", MIME type text/plain), ("0.332713", MIME type text/plain),
("42", MIME type text/plain), ("4.0", MIME type text/plain), ("0.474342", MIME type
text/plain)]), (2, Any[("4", MIME type text/plain), ("\"4 Upstream transportation and
distribution\"", MIME type text/plain), ("1349.18", MIME type text/plain), ("0.107851", MIME type
text/plain), ("18", MIME type text/plain), ("3.0", MIME type text/plain), ("0.206128", MIME type
text/plain)]), (3, Any[("5", MIME type text/plain), ("\"5 Waste generated in operations\"", MIME
type text/plain), ("1724.34", MIME type text/plain), ("0.13784", MIME type text/plain), ("18",
MIME type text/plain), ("3.0", MIME type text/plain), ("0.359212", MIME type text/plain)]), (4,
Any[("6", MIME type text/plain), ("\"6 Business travel\"", MIME type text/plain), ("582.3", MIME
type text/plain), ("0.0465479", MIME type text/plain), ("18", MIME type text/plain), ("3.0", MIME
type text/plain), ("0.191574", MIME type text/plain)]), (5, Any[("7", MIME type text/plain),
("\"7 Employee commuting\"", MIME type text/plain), ("4089.19", MIME type text/plain),
("0.326882", MIME type text/plain), ("18", MIME type text/plain), ("3.0", MIME type text/plain),
("0.23065", MIME type text/plain)]), (6, Any[("9", MIME type text/plain), ("\"9 Downstream
transportation and distribution\"", MIME type text/plain), ("602.539", MIME type text/plain),
("0.0481658", MIME type text/plain), ("12", MIME type text/plain), ("3.0", MIME type text/plain),
("0.203151", MIME type text/plain)]), :objectid => "6ce38e841290bf89", :schema => Dict{Symbol,
Any}(:names => Any["cat", "category", "E", "share", "rows", "DQ", "u_rel"], :types => ["Int64",
"String", "Float64", "Float64", "Int64", "Float64", "Float64"])), MIME type
application/vnd.pluto.table+object)), (:E_Scope3, ("12509.686021500002", MIME type text/plain)),
(:hotspots, (Dict{Symbol, Any}(:prefix => "String", :elements => Tuple{Int64, Any}[(1, ("\"1
Purchased goods and services\"", MIME type text/plain)), (2, ("\"7 Employee commuting\"", MIME
type text/plain)), (3, ("\"5 Waste generated in operations\"", MIME type text/plain)), (4, ("\"4

```

```
Upstream transportation and distribution\""", MIME type text/plain))], :type => :Array,
:prefix_short => "", :objectid => "970736b94331bcfb"), MIME type
application/vnd.pluto.tree+object))], :type => :struct, :prefix_short => "Scope3Result",
:objectid => "9595cdb4fb2efa64")
```

```
begin
  bycat = sort(scope3.by_category, :E, rev=true)
  bycat.category = [first(c, 28) for c in bycat.category]
  plt = bar(bycat.category, bycat.E, legend=false, xrotation=20, ylabel="tCO2e",
            title="Scope 3 by category (sorted)", color=:steelblue)
  save_figure(plt, "06_scope3_categories"; saver=savefig)
  bycat
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("1", MIME type text/plain), ("\"1 Purchased goods and
servic\"", MIME type text/plain), ("4162.14", MIME type text/plain), ("0.332713", MIME type
text/plain), ("42", MIME type text/plain), ("4.0", MIME type text/plain), ("0.474342", MIME type
text/plain)]), (2, Any[("7", MIME type text/plain), ("\"7 Employee commuting\"", MIME type
text/plain), ("4089.19", MIME type text/plain), ("0.326882", MIME type text/plain), ("18", MIME
type text/plain), ("3.0", MIME type text/plain), ("0.23065", MIME type text/plain)]), (3,
Any[("5", MIME type text/plain), ("\"5 Waste generated in operati\"", MIME type text/plain),
("1724.34", MIME type text/plain), ("0.13784", MIME type text/plain), ("18", MIME type
text/plain), ("3.0", MIME type text/plain), ("0.359212", MIME type text/plain)]), (4, Any[("4",
MIME type text/plain), ("\"4 Upstream transportation an\"", MIME type text/plain), ("1349.18",
MIME type text/plain), ("0.107851", MIME type text/plain), ("18", MIME type text/plain), ("3.0",
MIME type text/plain), ("0.206128", MIME type text/plain)]), (5, Any[("9", MIME type text/plain),
("\"9 Downstream transportation \"", MIME type text/plain), ("602.539", MIME type text/plain),
("0.0481658", MIME type text/plain), ("12", MIME type text/plain), ("3.0", MIME type text/plain),
("0.203151", MIME type text/plain)]), (6, Any[("6", MIME type text/plain), ("\"6 Business
travel\"", MIME type text/plain), ("582.3", MIME type text/plain), ("0.0465479", MIME type
text/plain), ("18", MIME type text/plain), ("3.0", MIME type text/plain), ("0.191574", MIME type
text/plain)]), :objectid => "616257b3e6f0b78d", :schema => Dict{Symbol, Any}(:names =>
Any["cat", "category", "E", "share", "rows", "DQ", "u_rel"], :types => ["Int64", "String",
"Float64", "Float64", "Int64", "Float64", "Float64"])]
```

```
md"""
### Hotspots and the method mix
```

```
The hotspots are the categories that together carry `hotspot_share` of the total –
the refinement list. The method mix shows how much of the Scope 3 figure rests on
spend-based proxies, i.e. on estimates that must be replaced before assurance.
"""
```

Hotspots and the method mix

The hotspots are the categories that together carry `hotspot_share` of the total — the refinement list. The method mix shows how much of the Scope 3 figure rests on spend-based proxies, i.e. on estimates that must be replaced before assurance.

```

begin
    mix = combine(groupby(emissions_dataframe(scope3.rows), :method),
                    :E => sum => :E, nrow => :rows)
    mix.share = mix.E ./ sum(mix.E)
    (hotspots=scope3.hotspots, method_mix=mix)
end

```

```

Dict{Symbol, Any}(:elements => Tuple{"hotspots", (Dict{Symbol, Any}(:prefix => "String",
:elements => Tuple{Int64, Any}[(1, ("\"1 Purchased goods and services\"", MIME type text/plain)),
(2, ("\"7 Employee commuting\"", MIME type text/plain)), (3, ("\"5 Waste generated in
operations\"", MIME type text/plain)), (4, ("\"4 Upstream transportation and distribution\"",
MIME type text/plain))], :type => :Array, :prefix_short => "", :objectid => "970736b94331bcfb"),
MIME type application/vnd.pluto.tree+object)), ("method_mix", (Dict{Symbol, Any}(:rows => Any[(1,
Any[("\"spend based (EEIO, deflated)\"", MIME type text/plain), ("4162.14", MIME type
text/plain), ("42", MIME type text/plain), ("0.332713", MIME type text/plain)]), (2,
Any[("\"average data (industry)\"", MIME type text/plain), ("4258.36", MIME type text/plain),
("66", MIME type text/plain), ("0.340405", MIME type text/plain)]), (3, Any[("\"proxy
estimate\"", MIME type text/plain), ("4089.19", MIME type text/plain), ("18", MIME type
text/plain), ("0.326882", MIME type text/plain)]], :objectid => "2851212e38b65ee7", :schema =>
Dict{Symbol, Any}(:names => Any["method", "E", "rows", "share"], :types => ["String", "Float64",
"Int64", "Float64"])), MIME type application/vnd.pluto.table+object))), :type => :NamedTuple,
:objectid => "e35104db720e4bfff")

```

```

begin
    println("E_Scope3      : ", round(scope3.E_Scope3; digits=1), " tCO₂e")
    println("categories used : ", nrow(scope3.by_category), " of 15")
    println("hotspots      : ", join(scope3.hotspots, " · "))
    println("Notebook 06 – every category carries a method and a data-quality class, and the
hotspots are named rather than assumed.")
end

```

07_SoftSensor_Neural_Net

20 cells · executed in 22.6 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topright, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
    const TAGS = ["load_pct", "stack_temp_C", "o2_pct", "flare_flow_m3h", "pressure_bar",
                  "feedstock_tph"]
    const p_CH₄ = CarbonAccounting.p_CH₄           # kg/m³ at normal conditions
end
```

0.7168

```
md"""
# 07 · A neural soft sensor for the annual analyzer campaign

The operational reality: the expensive measurement – a mobile analyzer, a lab
campaign, a certified stack test – happens **once a year for one week**,
`N_camp = 7 d · 24 h = 168` samples, while the plant produces continuous data every
hour. The soft sensor learns the mapping

     $\hat{y} = f_{\theta}(x)$        $x$  : continuous DCS/SCADA tags (load, temperature, O₂, flow, ...)
                                 $y$  : what the campaign measures (here: CH₄ mole fraction %)

and then predicts `y` for all 8 760 hours of the year – with a quantified
uncertainty instead of a smoothed annual average.
"""
```

07 · A neural soft sensor for the annual analyzer campaign

The operational reality: the expensive measurement — a mobile analyzer, a lab campaign, a certified stack test — happens **once a year for one week**, $N_{\text{camp}} = 7 \text{ d} \cdot 24 \text{ h} = 168$ samples, while the plant produces continuous data every hour. The soft sensor learns the mapping

$\hat{y} = f_{\theta}(x)$ x : continuous DCS/SCADA tags (load, temperature, O₂, flow, ...)
 y : what the campaign measures (here: CH₄ mole fraction %)

and then predicts y for all 8 760 hours of the year — with a quantified uncertainty instead of a smoothed annual average.

```
md"""
### The network, written as the equations


$$z^{(1)} = W^{(1)}a^{(1-1)} + b^{(1)} \quad a^{(1)} = g(z^{(1)}) \quad \hat{y} = W^{(L)}a^{(L-1)} + b^{(L)}$$


$$\delta^{(L)} = \partial L / \partial \hat{y} \quad \delta^{(1)} = (W^{(1+1)})^T \delta^{(1+1)} \odot g'(z^{(1)}) \quad \partial L / \partial W^{(1)} = \delta^{(1)}(a^{(1-1)})^T + \lambda_{L2} W^{(1)}$$


Trained with mini-batch Adam ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ) and early stopping on a
validation split. Two output heads make it heteroscedastic: it predicts  $\hat{y}$  and
 $\hat{\sigma}(x)$ , its own noise level, through the Gaussian NLL


$$L = \frac{1}{2}[(\mu - y)^2 e^{-s} + s], \quad s = \log \hat{\sigma}^2$$

"""
```

The network, written as the equations

$$z^{(1)} = W^{(1)}a^{(1-1)} + b^{(1)} \quad a^{(1)} = g(z^{(1)}) \quad \hat{y} = W^{(L)}a^{(L-1)} + b^{(L)}$$

$$\delta^{(L)} = \partial L / \partial \hat{y} \quad \delta^{(1)} = (W^{(1+1)})^T \delta^{(1+1)} \odot g'(z^{(1)}) \quad \partial L / \partial W^{(1)} = \delta^{(1)}(a^{(1-1)})^T + \lambda_{L2} W^{(1)}$$

Trained with mini-batch **Adam** ($\beta_1 = 0.9$, $\beta_2 = 0.999$) and early stopping on a validation split. Two output heads make it *heteroscedastic*: it predicts $\hat{y}$ and $\hat{\sigma}(x)$, its own noise level, through the Gaussian NLL

$$L = \frac{1}{2}[(\mu - y)^2 e^{-s} + s], \quad s = \log \hat{\sigma}^2$$

```

begin
    campaign = CSV.read(datadir("raw", "analyzer_campaign_2024.csv"), DataFrame)
    dcs = CSV.read(datadir("raw", "dcs_hourly_2024.csv"), DataFrame)
    X = Matrix(campaign[:, TAGS])                # campaign features
    y = Vector(campaign.ch4_fraction_pct)        # what the analyzer measured
    X_year = Matrix(dcs[:, TAGS])                # continuous data for the year
    (campaign=size(X), year=size(X_year),
     tags=DataFrame(tag=TAGS,
                    campaign_min=vec(minimum(X; dims=1)), campaign_max=vec(maximum(X;
dims=1)),
                    year_min=vec(minimum(X_year; dims=1)), year_max=vec(maximum(X_year;
dims=1))))
end

```

```

Dict{Symbol, Any}(:elements => Tuple[("campaign", (Dict{Symbol, Any}(:elements => Tuple{Int64,
Tuple}[(1, ("168", MIME type text/plain)), (2, ("6", MIME type text/plain))], :type => :Tuple,
:objectid => "9a58515f63a76dd7"), MIME type application/vnd.pluto.tree+object)), ("year",
(Dict{Symbol, Any}(:elements => Tuple{Int64, Tuple}[(1, ("8760", MIME type text/plain)), (2,
("6", MIME type text/plain))], :type => :Tuple, :objectid => "5bcf313ca7745b5e"), MIME type
application/vnd.pluto.tree+object)), ("tags", (Dict{Symbol, Any}(:rows => Any[(1,
Any[("\load_pct\\"", MIME type text/plain), ("26.4015", MIME type text/plain), ("85.1292", MIME
type text/plain), ("38.6551", MIME type text/plain), ("104.0", MIME type text/plain)]), (2,
Any[("\stack_temp_C\\"", MIME type text/plain), ("312.804", MIME type text/plain), ("349.21",
MIME type text/plain), ("314.414", MIME type text/plain), ("383.699", MIME type text/plain)]),
(3, Any[("\o2_pct\\"", MIME type text/plain), ("2.81724", MIME type text/plain), ("4.20234", MIME
type text/plain), ("2.09842", MIME type text/plain), ("5.07404", MIME type text/plain)]), (4,
Any[("\flare_flow_m3h\\"", MIME type text/plain), ("2949.5", MIME type text/plain), ("5710.87",
MIME type text/plain), ("3100.57", MIME type text/plain), ("6871.2", MIME type text/plain)]), (5,
Any[("\pressure_bar\\"", MIME type text/plain), ("2.40751", MIME type text/plain), ("2.7237",
MIME type text/plain), ("2.00612", MIME type text/plain), ("2.86971", MIME type text/plain)]),
(6, Any[("\feedstock_tph\\"", MIME type text/plain), ("11.0605", MIME type text/plain),
("17.5904", MIME type text/plain), ("11.4712", MIME type text/plain), ("20.0019", MIME type
text/plain)]), :objectid => "15e04b287fdc7914", :schema => Dict{Symbol, Any}(:names =>
Any["tag", "campaign_min", "campaign_max", "year_min", "year_max"], :types => ["String",
"Float64", "Float64", "Float64", "Float64"])), MIME type application/vnd.pluto.table+object)],
:type => :NamedTuple, :objectid => "d9322000e15a0c92")

```

```
md"### Train, and look at the convergence – not only at the final number"
```

Train, and look at the convergence — not only at the final number

```

begin
    net = train_soft_sensor(X, y; hidden=[16, 8], heteroscedastic=true,
                           epochs=1200, η=0.01, λ_L2=1e-4, seed=3)

    net
end

```

```
MLP(heteroscedastic, layers 6→16×8×2, 266 θ, nll, 115 epochs)
```

```
begin
    h = training_history(net)
    plt_train = plot(h.epoch, [h.train h.val], label=["train" "validation"],
                     xlabel="epoch", ylabel="loss (NLL, may be negative)", title="Convergence
(early stopping)")
    vline!(plt_train, [net.trained_epochs], label="best epoch", ls=:dash, color=:grey)
    save_figure(plt_train, "07_training_history"; saver=savefig)
    plt_train
end
```



```
md"""
### Scores that matter: error and honesty

RMSE and  $R^2$  say how close the predictions are; `coverage95` says whether the stated
uncertainty can be trusted. A model with 95 % coverage has error bars that mean
something – the property a compliance team actually needs.
"""
```

Scores that matter: error *and* honesty

RMSE and R^2 say how close the predictions are; coverage95 says whether the stated uncertainty can be trusted. A model with 95 % coverage has error bars that mean something — the property a compliance team actually needs.

```
begin
    ŷ = predict(net, X)
    iv = predict_interval(net, X)
    metrics_table(softsensor_metrics(y, ŷ; intervals=iv))
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("168", MIME type text/plain), ("0.0357626", MIME type
text/plain), ("0.0274398", MIME type text/plain), ("0.951101", MIME type text/plain),
("0.000697688", MIME type text/plain), ("0.97619", MIME type text/plain), ("0.0351208", MIME type
text/plain)]], :objectid => "8d87c739dab51cc1", :schema => Dict{Symbol, Any}(:names => Any["n",
"RMSE", "MAE", "R²", "bias", "coverage95", "ŷ_mean"], :types => ["Int64", "Float64", "Float64",
"Float64", "Float64", "Float64", "Float64", "Float64"])))
```

```
cross_validate_softsensor(X, y; k=5, epochs=600, seed=11)
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("1", MIME type text/plain), ("134", MIME type
text/plain), ("34", MIME type text/plain), ("0.0476642", MIME type text/plain), ("0.0376726",
MIME type text/plain), ("0.918382", MIME type text/plain)]], (2, Any[("2", MIME type text/plain),
("134", MIME type text/plain), ("34", MIME type text/plain), ("0.0486917", MIME type text/plain),
("0.0368909", MIME type text/plain), ("0.904745", MIME type text/plain)]], (3, Any[("3", MIME
type text/plain), ("134", MIME type text/plain), ("34", MIME type text/plain), ("0.0429147", MIME
type text/plain), ("0.0340811", MIME type text/plain), ("0.939403", MIME type text/plain)]], (4,
Any[("4", MIME type text/plain), ("135", MIME type text/plain), ("33", MIME type text/plain),
("0.0381933", MIME type text/plain), ("0.029796", MIME type text/plain), ("0.922544", MIME type
text/plain)]], (5, Any[("5", MIME type text/plain), ("135", MIME type text/plain), ("33", MIME
type text/plain), ("0.0449881", MIME type text/plain), ("0.0334754", MIME type text/plain),
("0.925781", MIME type text/plain)]], :objectid => "6d00bb00dc7410f7", :schema => Dict{Symbol,
Any}(:names => Any["fold", "n_train", "n_test", "RMSE", "MAE", "R²"], :types => ["Int64",
"Int64", "Int64", "Float64", "Float64", "Float64"])))
```

```
md"""
### An ensemble, so that uncertainty has two components
```

$$\sigma_{\text{total}}^2 = \sigma_{\text{epistemic}}^2 + \hat{\sigma}_{\text{aleatoric}}^2$$

The spread between ensemble members is the epistemic part – how much the answer depends on which week happened to be measured – while the heteroscedastic head gives the aleatoric part per hour.

```
"""
```

An ensemble, so that uncertainty has two components

$$\sigma_{\text{total}}^2 = \sigma_{\text{epistemic}}^2 + \hat{\sigma}_{\text{aleatoric}}^2$$

The spread between ensemble members is the epistemic part — how much the answer depends on which week happened to be measured — while the heteroscedastic head gives the aleatoric part per hour.

```
begin
    nets = train_ensemble(X, y; n_models=5, epochs=800, heteroscedastic=true, η=0.01)
    ivc = ensemble_interval(nets, X)
    lower = [[i.ŷ - i.lo for i in ivc]...]
    upper = [[i.hi - i.ŷ for i in ivc]...]
    plt_fit = plot(1:length(y), y, label="analyzer (campaign week)", color=:black, lw=2)
    plot!(plt_fit, 1:length(y), [i.ŷ for i in ivc], ribbon=(lower, upper), fillalpha=0.3,
          label="soft sensor ± 95 %", color=:steelblue)
    save_figure(plt_fit, "07_softsensor_fit"; saver=savefig)
    plt_fit
end
```



```
md"""
### Does one week represent the year?
```

Two tests on the tag matrix: the **envelope** per feature (fraction of annual hours inside the campaign range) and the **Mahalanobis** distance of each annual hour from the campaign cloud, compared with the χ^2 limit – which catches combinations of tags that are individually plausible but jointly unseen.

```
"""
```

Does one week represent the year?

Two tests on the tag matrix: the **envelope** per feature (fraction of annual hours inside the campaign range) and the **Mahalanobis** distance of each annual hour from the campaign cloud, compared with the χ^2 limit — which catches combinations of tags that are individually plausible but jointly unseen.

```

begin
    design = campaign_design(X, X_year; feature_names=TAGS,
                             activity=Vector(dcs.flare_flow_m3h))
    (coverage=round(design.coverage; digits=3),
     emission_weighted_risk=round(design.risk_share; digits=3),
     χ²_limit=round(design.chi2_limit; digits=1),
     recommendation=design.recommendation,
     envelope=design.features)
end

```

```

Dict{Symbol, Any}(:elements => Tuple[("coverage", ("0.634", MIME type text/plain)),
  ("emission_weighted_risk", ("0.389", MIME type text/plain)), ("χ²_limit", ("12.6", MIME type
text/plain)), ("recommendation", ("\"the one-week campaign does NOT represent the year (coverage
63.4 %, emission-weighted risk 38.9 %): extend or stratify the campaign across load states, or
carry a wider uncertainty on the extrapolated hours\"", MIME type text/plain)), ("envelope",
(Dict{Symbol, Any}(:rows => Any[(1, Any[("\"load_pct\"", MIME type text/plain), ("26.4015", MIME
type text/plain), ("85.1292", MIME type text/plain), ("38.6551", MIME type text/plain), ("104.0",
MIME type text/plain), ("0.684361", MIME type text/plain)]), (2, Any[("\"stack_temp_C\"", MIME
type text/plain), ("312.804", MIME type text/plain), ("349.21", MIME type text/plain),
("314.414", MIME type text/plain), ("383.699", MIME type text/plain), ("0.7371", MIME type
text/plain)]), (3, Any[("\"o2_pct\"", MIME type text/plain), ("2.81724", MIME type text/plain),
("4.20234", MIME type text/plain), ("2.09842", MIME type text/plain), ("5.07404", MIME type
text/plain), ("0.734247", MIME type text/plain)]), (4, Any[("\"flare_flow_m3h\"", MIME type
text/plain), ("2949.5", MIME type text/plain), ("5710.87", MIME type text/plain), ("3100.57",
MIME type text/plain), ("6871.2", MIME type text/plain), ("0.786301", MIME type text/plain)]),
(5, Any[("\"pressure_bar\"", MIME type text/plain), ("2.40751", MIME type text/plain), ("2.7237",
MIME type text/plain), ("2.00612", MIME type text/plain), ("2.86971", MIME type text/plain),
("0.963128", MIME type text/plain)]), (6, Any[("\"feedstock_tph\"", MIME type text/plain),
("11.0605", MIME type text/plain), ("17.5904", MIME type text/plain), ("11.4712", MIME type
text/plain), ("20.0019", MIME type text/plain), ("0.764041", MIME type text/plain)])), :objectid
=> "b6b2f99fadc3821c", :schema => Dict{Symbol, Any}(:names => Any["feature", "campaign_min",
"campaign_max", "annual_min", "annual_max", "covered_fraction"], :types => ["String", "Float64",
"Float64", "Float64", "Float64", "Float64"])), MIME type application/vnd.pluto.table+object))),
:type => :NamedTuple, :objectid => "accb07c544f71749")

```

```

begin
    needed = min_campaign_samples(0.35; δ=0.05)
    println("to halve the campaign-mean interval: N ≥ ", needed,
           " samples (a one-week campaign provides 168)")
end

```

```
md"### Annualise, and compare with the method it replaces"
```

Annualise, and compare with the method it replaces

```

begin
    scale = p_CH₄ / 100                                # volume-% of CH₄ → kg CH₄ per m³
    annual = annualize_with_softsensor(nets, X_year, Vector(dcs.flare_flow_m3h);
                                       GWP₁₀₀=GWP_AR6.CH₄_fossil, hours=1.0,
                                       target_scale=scale)

    week = 1:168
    ratio = annualize_ratio(Vector(dcs.flare_flow_m3h),
Vector(campaign.flare_flow_m3h[week]),
                                scale * sum(Vector(campaign.flare_flow_m3h[week]) .* y[week]) *
                                GWP_AR6.CH₄_fossil / 1000)
    (soft_sensor=annualization_statement(annual), incumbent=annualization_statement(ratio),
    difference_pct=round(100 * (annual.E - ratio.E) / ratio.E; digits=1))
end

```

```

Dict{Symbol, Any}(:elements => Tuple[("soft_sensor", ("\"soft sensor over annual continuous data:
E = 10541.9 tCO₂e [9527.6 - 11556.2] at 95 % coverage (u_r = 4.9 %)\"", MIME type text/plain)),
("incumbent", ("\"activity-ratio scaling of the campaign week: E = 11459.7 tCO₂e (activity ratio
62.80×, no interval - the method assumes a constant factor)\"", MIME type text/plain)),
("difference_pct", ("-8.0", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"5ee1deebfec5a979")

```

```

begin
    per_hour = annual.per_hour
    plt_emit = plot(per_hour.t, per_hour.E_t, label="soft-sensor emission",
color=:darkorange,
                xlabel="hour of 2024", ylabel="tCO₂e per hour", title="Annualised emission")
    plot!(plt_emit, 1:168, per_hour.E_t[1:168], label="campaign week", color=:steelblue,
lw=2)
    save_figure(plt_emit, "07_annualised_emission"; saver=savefig)
    plt_emit
end

```



```

begin
    println("campaign samples      : ", length(y), " hours")
    println("annual estimate       : ", round(annual.E; digits=1), " tCO₂e [",
            round(annual.lo; digits=1), " - ", round(annual.hi; digits=1), "]")
    println("relative uncertainty : ", round(100 * annual.u_rel; digits=1), " %")
    println("campaign coverage      : ", round(100 * design.coverage; digits=1), " % of annual
hours")
    println("Notebook 07 - the soft sensor reproduces the measured variable, states an honest
interval, and shows where the campaign does not represent the year.")
end

```

08_Anomaly_Detection

20 cells · executed in 24.85 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topright, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
    const TAGS = ["load_pct", "stack_temp_C", "o2_pct", "flare_flow_m3h", "pressure_bar",
                  "feedstock_tph"]
    const EPISODES = [40 * 24 .+ (1:30), 120 * 24 .+ (1:20), 210 * 24 .+ (1:24),
                      300 * 24 .+ (1:12)]
end
```

```
Dict{Symbol, Any}(:prefix => "UnitRange{Int64}", :elements => Tuple{Int64, Any}[(1, ("961:990",
MIME type text/plain)), (2, ("2881:2900", MIME type text/plain)), (3, ("5041:5064", MIME type
text/plain)), (4, ("7201:7212", MIME type text/plain))], :type => :Array, :prefix_short => "",
:objectid => "5c9c47d5377c7028")
```

```
md"""
# 08 · Anomaly detection in emissions

Three jobs, in this order: (1) find emission events the plant did not intend – leaks,
flare upset, uncombusted slip, a controlled variable drifting; (2) find *data*
problems before they become reporting problems; (3) alert without crying wolf – the
false-alarm rate has to be a design parameter.

| detector | statistic | catches |
|---|---|---|
| EWMA |  $z_t = \lambda \hat{r}_t + (1-\lambda)z_{t-1}$  | slow drifts |
| CUSUM |  $S_t^+/S_t^- = \max(0, S + \hat{r}_t \mp k\hat{\sigma})$  | small persistent shifts |
| Hotelling  $T^2$  |  $(x - \mu)^T \Sigma^{-1} (x - \mu)$  | multivariate outliers |
| PCA SPE (Q) |  $\|x - P P^T x\|^2$  | broken correlations |
| autoencoder |  $\|x - f_{\theta}(x)\|^2$  | non-linear structure |

Every limit is calibrated on the reference period, so  $ARL_0 = 1/\alpha$  is documented
rather than hoped for.
"""
```

08 · Anomaly detection in emissions

Three jobs, in this order: (1) find emission events the plant did not intend — leaks, flare upset, uncombusted slip, a controlled variable drifting; (2) find *data* problems before they become reporting problems; (3) alert without crying wolf — the false-alarm rate has to be a design parameter.

detector	statistic	catches
EWMA	$z_t = \lambda \hat{r}_t + (1-\lambda)z_{t-1}$	slow drifts
CUSUM	$S_t^+/S_t^- = \max(0, S + \hat{r}_t \mp k\hat{\sigma})$	small persistent shifts
Hotelling T^2	$(x - \mu)^T \Sigma^{-1} (x - \mu)$	multivariate outliers
PCA SPE (Q)	$\ x - P P^T x\ ^2$	broken correlations
autoencoder	$\ x - f_{\theta}(x)\ ^2$	non-linear structure

Every limit is calibrated on the reference period, so $ARL_0 = 1/\alpha$ is documented rather than hoped for.

```
md"""
### The data, and what was injected into it

The DCS file carries a year of hourly tags plus four deliberate episodes. A detector
that cannot find them is not worth deploying.
"""
```

The data, and what was injected into it

The DCS file carries a year of hourly tags plus four deliberate episodes. A detector that cannot find them is not worth deploying.

```
begin
    dcs = CSV.read(datadir("raw", "dcs_hourly_2024.csv"), DataFrame)
    X = Matrix(dcs[:, TAGS])
    DataFrame(episode=1:4, hours=[string(first(w), "-", last(w)) for w in EPISODES],
              length=[length(w) for w in EPISODES],
              injected=["0₂ sensor drifts high", "flow falls while load rises",
                       "pressure drops (leak)", "thermal spike"])
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("1", MIME type text/plain), ("\"961-990\"", MIME type
text/plain), ("30", MIME type text/plain), ("\"0₂ sensor drifts high\"", MIME type text/plain)]),
(2, Any[("2", MIME type text/plain), ("\"2881-2900\"", MIME type text/plain), ("20", MIME type
text/plain), ("\"flow falls while load rises\"", MIME type text/plain)]), (3, Any[("3", MIME type
text/plain), ("\"5041-5064\"", MIME type text/plain), ("24", MIME type text/plain), ("\"pressure
drops (leak)\"", MIME type text/plain)]), (4, Any[("4", MIME type text/plain), ("\"7201-7212\"",
MIME type text/plain), ("12", MIME type text/plain), ("\"thermal spike\"", MIME type
text/plain)]], :objectid => "9ef996acadf62d43", :schema => Dict{Symbol, Any}(:names =>
Any["episode", "hours", "length", "injected"], :types => ["Int64", "String", "Int64", "String"])))
```

```
md"""
### Choose a representative reference
```

Training the detectors on the first 2 000 hours (winter) makes the rest of the year look anomalous – a real failure mode of naive monitoring. The reference is therefore drawn from the whole year, with the known episodes removed.

Choose a *representative* reference

Training the detectors on the first 2 000 hours (winter) makes the rest of the year look anomalous — a real failure mode of naive monitoring. The reference is therefore drawn from the whole year, with the known episodes removed.

```
begin
    normal = setdiff(1:size(X, 1), vcat(EPISODES...))
    ref_idx = normal[1:6:end]
    (normal_hours=length(normal), reference_hours=length(ref_idx), of_year=size(X, 1))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("normal_hours", ("8674", MIME type text/plain)),
("reference_hours", ("1446", MIME type text/plain)), ("of_year", ("8760", MIME type
text/plain))], :type => :NamedTuple, :objectid => "1a54ea27bd35a462")
```

```
begin
    report = detect_anomalies(X; reference=X[ref_idx, :], α=0.01, k_pca=3,
                             ae_hidden=[10], ae_epochs=400, names=TAGS)
    report
end
```

```
AnomalyReport(570 event(s) from Hotelling T² + PCA SPE + Autoencoder SPE, target false-alarm rate
α = 0.01)
```

```
report.summary
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\Autoencoder SPE\\"", MIME type text/plain), ("\low\\"",
MIME type text/plain), ("49", MIME type text/plain), ("5.59361", MIME type text/plain)]), (2,
Any[("\Autoencoder SPE\\"", MIME type text/plain), ("\medium\\"", MIME type text/plain), ("62",
MIME type text/plain), ("7.07763", MIME type text/plain)]), (3, Any[("\Autoencoder SPE\\"", MIME
type text/plain), ("\high\\"", MIME type text/plain), ("129", MIME type text/plain), ("14.726",
MIME type text/plain)]), (4, Any[("\Hotelling T²\\"", MIME type text/plain), ("\low\\"", MIME
type text/plain), ("111", MIME type text/plain), ("12.6712", MIME type text/plain)]), (5,
Any[("\Hotelling T²\\"", MIME type text/plain), ("\medium\\"", MIME type text/plain), ("28", MIME
type text/plain), ("3.19635", MIME type text/plain)]), (6, Any[("\Hotelling T²\\"", MIME type
text/plain), ("\high\\"", MIME type text/plain), ("66", MIME type text/plain), ("7.53425", MIME
type text/plain)]), (7, Any[("\PCA SPE\\"", MIME type text/plain), ("\low\\"", MIME type
text/plain), ("67", MIME type text/plain), ("7.6484", MIME type text/plain)]), (8, Any[("\PCA
SPE\\"", MIME type text/plain), ("\medium\\"", MIME type text/plain), ("24", MIME type
text/plain), ("2.73973", MIME type text/plain)]), (9, Any[("\PCA SPE\\"", MIME type text/plain),
("\high\\"", MIME type text/plain), ("34", MIME type text/plain), ("3.88128", MIME type
text/plain)]), :objectid => "55409dec79f7a484", :schema => Dict{Symbol, Any}(:names =>
Any["detector", "severity", "n", "rate_per_1000"], :types => ["String", "String", "Int64",
"Float64"])])
```

```
md#### Did the suite find the injected episodes? Flagged hours, per window"
```

Did the suite find the injected episodes? Flagged hours, per window

```
begin
    flagged = falses(size(X, 1))
    for e in report.events
        (e.severity == :low) && continue
        (1 <= e.t <= length(flagged)) && (flagged[e.t] = true)
    end
    DataFrame(episode=1:4, window=[string(first(w), "-", last(w)) for w in EPISODES],
              flagged_hours=[count(flagged[w]) for w in EPISODES],
              window_hours=[length(w) for w in EPISODES],
              recall=[round(count(flagged[w]) / length(w); digits=2) for w in EPISODES])
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("1", MIME type text/plain), ("\961-990\\"", MIME type
text/plain), ("30", MIME type text/plain), ("30", MIME type text/plain), ("1.0", MIME type
text/plain)]), (2, Any[("2", MIME type text/plain), ("\2881-2900\\"", MIME type text/plain),
("9", MIME type text/plain), ("20", MIME type text/plain), ("0.45", MIME type text/plain)]), (3,
Any[("3", MIME type text/plain), ("\5041-5064\\"", MIME type text/plain), ("24", MIME type
text/plain), ("24", MIME type text/plain), ("1.0", MIME type text/plain)]), (4, Any[("4", MIME
type text/plain), ("\7201-7212\\"", MIME type text/plain), ("12", MIME type text/plain), ("12",
MIME type text/plain), ("1.0", MIME type text/plain)]), :objectid => "8219e9878d5e744d", :schema
=> Dict{Symbol, Any}(:names => Any["episode", "window", "flagged_hours", "window_hours",
"recall"], :types => ["Int64", "String", "Int64", "Int64", "Float64"])])
```

```
begin
    cases = events_dataframe(report.events)
    sel = cases[cases.severity .!= "low", :]
    first(select(sel, :t, :detector, :severity, :feature, :score, :limit, :note), 8)
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("34", MIME type text/plain), ("\"PCA SPE\"", MIME type text/plain), ("\"high\"", MIME type text/plain), ("\"correlation break\"", MIME type text/plain), ("2.24922", MIME type text/plain), ("1.04281", MIME type text/plain), ("\"correlation between tags broken (Q-statistic)\"", MIME type text/plain)]), (2, Any[("36", MIME type text/plain), ("\"Autoencoder SPE\"", MIME type text/plain), ("\"medium\"", MIME type text/plain), ("\"pressure_bar\"", MIME type text/plain), ("0.0596045", MIME type text/plain), ("0.0390714", MIME type text/plain), ("\"non-linear reconstruction error; dominant tag: pressure_bar\"", MIME type text/plain)]), (3, Any[("57", MIME type text/plain), ("\"Autoencoder SPE\"", MIME type text/plain), ("\"medium\"", MIME type text/plain), ("\"pressure_bar\"", MIME type text/plain), ("0.0596003", MIME type text/plain), ("0.0390714", MIME type text/plain), ("\"non-linear reconstruction error; dominant tag: pressure_bar\"", MIME type text/plain)]), (4, Any[("59", MIME type text/plain), ("\"Autoencoder SPE\"", MIME type text/plain), ("\"high\"", MIME type text/plain), ("\"pressure_bar\"", MIME type text/plain), ("0.14607", MIME type text/plain), ("0.0390714", MIME type text/plain), ("\"non-linear reconstruction error; dominant tag: pressure_bar\"", MIME type text/plain)]), (5, Any[("148", MIME type text/plain), ("\"Autoencoder SPE\"", MIME type text/plain), ("\"high\"", MIME type text/plain), ("\"pressure_bar\"", MIME type text/plain), ("0.211281", MIME type text/plain), ("0.0390714", MIME type text/plain), ("\"non-linear reconstruction error; dominant tag: pressure_bar\"", MIME type text/plain)]), (6, Any[("180", MIME type text/plain), ("\"Autoencoder SPE\"", MIME type text/plain), ("\"medium\"", MIME type text/plain), ("\"pressure_bar\"", MIME type text/plain), ("0.0640109", MIME type text/plain), ("0.0390714", MIME type text/plain), ("\"non-linear reconstruction error; dominant tag: pressure_bar\"", MIME type text/plain)]), (7, Any[("184", MIME type text/plain), ("\"Autoencoder SPE\"", MIME type text/plain), ("\"high\"", MIME type text/plain), ("\"pressure_bar\"", MIME type text/plain), ("0.0862676", MIME type text/plain), ("0.0390714", MIME type text/plain), ("\"non-linear reconstruction error; dominant tag: pressure_bar\"", MIME type text/plain)]), (8, Any[("186", MIME type text/plain), ("\"Autoencoder SPE\"", MIME type text/plain), ("\"medium\"", MIME type text/plain), ("\"pressure_bar\"", MIME type text/plain), ("0.0611705", MIME type text/plain), ("0.0390714", MIME type text/plain), ("\"non-linear reconstruction error; dominant tag: pressure_bar\"", MIME type text/plain)]], :objectid => "88223f07ddc5a426", :schema => Dict{Symbol, Any}(:names => Any[("t", "detector", "severity", "feature", "score", "limit", "note"), :types => ["Int64", "String", "String", "String", "Float64", "Float64", "String"]])
```

```
md"""
### Calibrating a threshold instead of guessing one

`calibrate_threshold(scores; α)` takes the empirical `1 - α` quantile of the
in-control scores: no distributional assumption, and the expected run length between
false alarms is `ARL₀ = 1/α`.
"""
```

Calibrating a threshold instead of guessing one

`calibrate_threshold(scores; α)` takes the empirical $1 - \alpha$ quantile of the in-control scores: no distributional assumption, and the expected run length between false alarms is $ARL_0 = 1/\alpha$.

```
begin
    f̂ = residual_frame(Vector(dcs.flare_flow_m3h),
                        Vector(dcs.load_pct) .* 38.0 .+ 2_200.0).f̂
    cal = calibrate_threshold(abs.(f̂); α=0.01)
    (α=cal.α, limit=round(cal.limit; digits=1), ARL₀_samples=cal.ARL₀,
     samples=cal.n, expected_false_alarms_per_year=round(8760 * cal.α; digits=1))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("α", ("0.01", MIME type text/plain)), ("limit", ("642.7",
MIME type text/plain)), ("ARL₀_samples", ("100.0", MIME type text/plain)), ("samples", ("8760",
MIME type text/plain)), ("expected_false_alarms_per_year", ("87.6", MIME type text/plain))],
:type => :NamedTuple, :objectid => "ee5432b680f3f138")
```

```
md"""
### The findings become audit-trail entries

`log_anomalies!` writes every medium/high finding as a proposed investigation, so
"we saw it, we looked at it" is provable at the next audit.
"""
```

The findings become audit-trail entries

`log_anomalies!` writes every medium/high finding as a *proposed investigation*, so "we saw it, we looked at it" is provable at the next audit.

```
begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("notebook-08")))
    analyst = Actor("u001", "A. Analyst", :analyst)
    (logged=log_anomalies!(ledger, analyst, report; context="hourly monitoring run"),
     chain=chain_statement(ledger))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("logged", ("343", MIME type text/plain)), ("chain",
("\nAUDIT CHAIN VERIFIED - 343 entries, head 926986226a5e7f5a...\n", MIME type text/plain))], :type
=> :NamedTuple, :objectid => "d31cf1e35e5f32f0")
```

```
md"""
### From alerts to emissions

The point of detection is action: how much CO2e was emitted during flagged intervals,
and how much of it exceeds the expected level?
"""
```

From alerts to emissions

The point of detection is action: how much CO₂e was emitted during flagged intervals, and how much of it exceeds the expected level?

```
begin
    scale = CarbonAccounting.p_CH₄ / 100          # volume-% → kg CH₄ per m³
    E_t = Vector(dcs.flare_flow_m3h) .* scale .* 1.4 .* GWP_AR6.CH₄_fossil ./ 1000
    pack = anomaly_report(report; per_hour=DataFrame(t=1:length(E_t), E_t=E_t),
                        ledger=ledger, actor=analyst)
    pack.impact
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("E_flagged", ("342.875", MIME type text/plain)),
("E_excess", ("23.063", MIME type text/plain)), ("share_flagged", ("0.0254596", MIME type
text/plain)), ("n_flagged", ("229", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"b386e1ceae29d5db")
```

```
begin
    plt = plot(1:length(E_t), E_t, label="tCO₂e per hour", color=:grey, lw=1,
              xlabel="hour of 2024", ylabel="tCO₂e", title="Injected episodes (shaded)")
    for (i, w) in enumerate(EPISODES)
        vspan!(plt, [first(w), last(w)], label=i == 1 ? "episodes" : "", color=:red,
alpha=0.15)
    end
    save_figure(plt, "08_anomalies"; saver=savefig)
    plt
end
```



```
begin
    println("events detected      : ", length(report.events), " from ",
            join(report.detectors, " + "))
    println("recall on injected    : ", join([round(count(flagged[w]) / length(w);
digits=2)
                                for w in EPISODES], ", "))
    println("emission at flagged hours: ", round(pack.impact.E_flagged; digits=1),
            " tCO₂e, of which ", round(pack.impact.E_excess; digits=1), " tCO₂e above
expectation")
    println("Notebook 08 – every injected episode is found, the limits are calibrated to a
false-alarm rate, and the alerts are logged and priced in tCO₂e.")
end
```

09_Audit_Trail_and_Security

18 cells · executed in 20.47 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 09 · Audit trails and security

Two guarantees, and nothing else:

**1. Who may do what.** A role-based permission matrix  $r_{ij}$  with segregation of duties –
the analyst who prepares a figure cannot approve it – and passwords handled by
PBKDF2-HMAC-SHA256.

**2. Nothing can be changed unnoticed.** Every state change is appended to a hash
chain


$$h_n = \text{SHA256}(h_{n-1} \parallel \text{canonical}(\text{entry}_n)), \quad h_0 = \text{SHA256}(\text{"genesis"})$$


so that altering any historical entry invalidates every subsequent hash. Batches are
sealed with a Merkle root and signed with HMAC-SHA256.
"""
```

09 · Audit trails and security

Two guarantees, and nothing else:

1. Who may do what. A role-based permission matrix r_{ij} with segregation of duties — the analyst who prepares a figure cannot approve it — and passwords handled by PBKDF2-HMAC-SHA256.

2. Nothing can be changed unnoticed. Every state change is appended to a hash chain

$$h_n = \text{SHA256}(h_{n-1} \parallel \text{canonical}(\text{entry}_n)), \quad h_0 = \text{SHA256}(\text{"genesis"})$$

so that altering any historical entry invalidates every subsequent hash. Batches are sealed with a Merkle root and signed with HMAC-SHA256.

```
md"""### The permission matrix  $r_{ij}$ """
```

The permission matrix r_{ij}

```
DataFrame(role=collect(keys(PERMISSIONS)),
           permissions=[join(sort([ACTION_NAMES[a] for a in PERMISSIONS[r]]), ", ")
                        for r in keys(PERMISSIONS)])
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("analyst", MIME type text/plain), ("calculate, export,
ingest, propose, read\"", MIME type text/plain)]), (2, Any[("admin", MIME type text/plain),
("admin, approve, calculate, export, ingest, propose, read, verify\"", MIME type text/plain)]),
(3, Any[("service", MIME type text/plain), ("calculate, ingest, read\"", MIME type
text/plain)]), (4, Any[("viewer", MIME type text/plain), ("read\"", MIME type text/plain)]),
(5, Any[("auditor", MIME type text/plain), ("export, read, verify\"", MIME type text/plain)]),
(6, Any[("approver", MIME type text/plain), ("approve, export, read\"", MIME type
text/plain)]), :objectid => "741285c0a2a82a4c", :schema => Dict{Symbol, Any}(:names =>
Any["role", "permissions"], :types => ["Symbol", "String"])
```

```
begin
    analyst = Actor("u001", "A. Analyst", :analyst)
    approver = Actor("u002", "B. Approver", :approver)
    auditor = Actor("u003", "C. Auditor", :auditor)
    (can_calculate=can(analyst, CalculateAction), can_approve=can(analyst, ApproveAction),
     denied=try
         authorize(analyst, ApproveAction; context="approval of the FY2024 inventory")
         "not denied – that would be a bug"
     catch e
         sprint(showerror, e)
     end,
     segregation=try
         check_segregation_of_duties(analyst, analyst)
         "allowed – that would be a bug"
     catch e
         first(sprint(showerror, e), 70)
     end)
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("can_calculate", ("true", MIME type text/plain)),
("can_approve", ("false", MIME type text/plain)), ("denied", ("\"AccessDenied: actor «A. Analyst»
(role analyst) may not approve – approval of the FY2024 inventory\"", MIME type text/plain)),
("segregation", ("\"ArgumentError: segregation of duties: «A. Analyst» may not approve the\"",
MIME type text/plain))], :type => :NamedTuple, :objectid => "9764332b870b87aa")
```

```
md#### Credentials: PBKDF2, and secrets kept out of the log"
```

Credentials: PBKDF2, and secrets kept out of the log

```
begin
    cred = hash_password("u001", "correct horse battery staple"; iterations=10_000)
    (algorithm="PBKDF2-HMAC-SHA256", iterations=cred.iterations,
hash_bytes=length(cred.hash),
    accepted=check_password(cred, "correct horse battery staple"),
    rejected=check_password(cred, "correct horse battery stapl"),
    redacted=redact("api_key=SECRET123 written by user@example.com"))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("algorithm", ("\"PBKDF2-HMAC-SHA256\"", MIME type
text/plain)), ("iterations", ("10000", MIME type text/plain)), ("hash_bytes", ("32", MIME type
text/plain)), ("accepted", ("true", MIME type text/plain)), ("rejected", ("false", MIME type
text/plain)), ("redacted", ("\"api_key=<redacted> written by <email>\"", MIME type text/plain))],
:type => :NamedTuple, :objectid => "74efe36d31dcadb3")
```

```
md"""
### The ledger: a calculation run writes itself into history

`compute_inventory` accepts a ledger and an actor; every engine writes its sub-total
with a *reason*, a source hash and the resulting value.
"""
```

The ledger: a calculation run writes itself into history

`compute_inventory` accepts a ledger and an actor; every engine writes its sub-total with a *reason*, a source hash and the resulting value.

```
begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("deployment-key-2026")))
    files = ["activity_erp_2024.csv", "electricity_utility_2024.csv", "procurement_2024.csv",
        "logistics_travel_2024.csv", "waste_2024.csv"]
    report = ingest_files([datadir("raw", f) for f in files];
        entity_default="AcmeIndustrial_SA", ledger=ledger, actor=analyst,
        context="FY2024 close - scheduled ingestion")
    inventory = compute_inventory(report.records; consolidation=:operational,
        date=Date(2024, 12, 31), ledger=ledger, actor=analyst)
    (entries=length(ledger), head=first(head_hash(ledger), 24),
    total=round(inventory.E_total; digits=1))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("entries", ("8", MIME type text/plain)), ("head",
("\"d055cbd037f3665e47c72c50\"", MIME type text/plain)), ("total", ("2.60724e5", MIME type
text/plain))], :type => :NamedTuple, :objectid => "af15e15ed16ee7a5")
```

```
begin
    trail = audit_trail_table(ledger)
    select(trail, :seq, :a_actor, :π_role, :action, :entity, :why, :after, :h_n)
end

Dict{Symbol, Any}(:rows => Any[[1, Any[("1", MIME type text/plain), ("\"A. Analyst\"", MIME type text/plain), ("\"analyst\"", MIME type text/plain), ("\"ingest\"", MIME type text/plain), ("\"activity_erp_2024.csv\"", MIME type text/plain), ("\"FY2024 close – scheduled ingestion\"", MIME type text/plain), ("\"175 records accepted, 2 quarantined\"", MIME type text/plain), ("\"80046d8b59a29780\"", MIME type text/plain)]], (2, Any[("2", MIME type text/plain), ("\"A. Analyst\"", MIME type text/plain), ("\"analyst\"", MIME type text/plain), ("\"ingest\"", MIME type text/plain), ("\"electricity_utility_2024.csv\"", MIME type text/plain), ("\"FY2024 close – scheduled ingestion\"", MIME type text/plain), ("\"247 records accepted, 2 quarantined\"", MIME type text/plain), ("\"ea2bfa81093dca31\"", MIME type text/plain)]], (3, Any[("3", MIME type text/plain), ("\"A. Analyst\"", MIME type text/plain), ("\"analyst\"", MIME type text/plain), ("\"ingest\"", MIME type text/plain), ("\"procurement_2024.csv\"", MIME type text/plain), ("\"FY2024 close – scheduled ingestion\"", MIME type text/plain), ("\"289 records accepted, 2 quarantined\"", MIME type text/plain), ("\"2ef69efd1607eb68\"", MIME type text/plain)]], (4, Any[("4", MIME type text/plain), ("\"A. Analyst\"", MIME type text/plain), ("\"analyst\"", MIME type text/plain), ("\"ingest\"", MIME type text/plain), ("\"logistics_travel_2024.csv\"", MIME type text/plain), ("\"FY2024 close – scheduled ingestion\"", MIME type text/plain), ("\"355 records accepted, 2 quarantined\"", MIME type text/plain), ("\"334fecdl49956d8d\"", MIME type text/plain)]], (5, Any[("5", MIME type text/plain), ("\"A. Analyst\"", MIME type text/plain), ("\"analyst\"", MIME type text/plain), ("\"ingest\"", MIME type text/plain), ("\"waste_2024.csv\"", MIME type text/plain), ("\"FY2024 close – scheduled ingestion\"", MIME type text/plain), ("\"373 records accepted, 2 quarantined\"", MIME type text/plain), ("\"cd7b6e70b34bcc58\"", MIME type text/plain)]], (6, Any[("6", MIME type text/plain), ("\"A. Analyst\"", MIME type text/plain), ("\"analyst\"", MIME type text/plain), ("\"calculate\"", MIME type text/plain), ("\"scope1\"", MIME type text/plain), ("\"scope 1 calculation run\"", MIME type text/plain), ("\"174781.393 tCO2e from 175 row(s)\"", MIME type text/plain), ("\"af13f01ccd7c03e0\"", MIME type text/plain)]], (7, Any[("7", MIME type text/plain), ("\"A. Analyst\"", MIME type text/plain), ("\"analyst\"", MIME type text/plain), ("\"calculate\"", MIME type text/plain), ("\"scope2\"", MIME type text/plain), ("\"scope 2 calculation run\"", MIME type text/plain), ("\"location 73433.265 tCO2e, market 86683.228 tCO2e, λcov = 19.6%\"", MIME type text/plain), ("\"4a96be43c480f1f2\"", MIME type text/plain)]], (8, Any[("8", MIME type text/plain), ("\"A. Analyst\"", MIME type text/plain), ("\"analyst\"", MIME type text/plain), ("\"calculate\"", MIME type text/plain), ("\"scope3\"", MIME type text/plain), ("\"scope 3 calculation run\"", MIME type text/plain), ("\"12509.686 tCO2e over 6 categor(y)ies; hotspots: 1 Purchased goods and services; 7 Employee commuting; 5 Waste generated in operations; 4 Upstream transportation and distribution\"", MIME type text/plain), ("\"d055cbd037f3665e\"", MIME type text/plain)]]], :objectid => "302256b600375f74", :schema => Dict{Symbol, Any}(:names => Any["seq", "a_actor", "π_role", "action", "entity", "why", "after", "h_n"], :types => ["Int64", "String", "String", "String", "String", "String", "String", "String"])))
```

```
md"""
### Verification, sealing and tamper detection
```

The chain is verified by re-computing every link; a manifest seals the current head and Merkle root with an HMAC. Then an attacker edits a stored entry – and the verifier stops exactly there.

```
"""
```

Verification, sealing and tamper detection

The chain is verified by re-computing every link; a manifest seals the current head and Merkle root with an HMAC. Then an attacker edits a stored entry — and the verifier stops exactly there.

```
begin
    ok, broken = verify_chain(ledger)
    manifest = seal(ledger)
    (chain_ok=ok, broken_at=broken, statement=chain_statement(ledger),
     merkle_root=first(manifest.merkle, 24), signature=first(manifest.signature, 24),
     seal_verifies=verify_seal(manifest, ledger))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("chain_ok", ("true", MIME type text/plain)), ("broken_at",
("nothing", MIME type text/plain)), ("statement", ("\"AUDIT CHAIN VERIFIED - 8 entries, head
d055cbd037f3665e...\"", MIME type text/plain)), ("merkle_root", ("\"9a3459854b6dc417af8c5e3c\"",
MIME type text/plain)), ("signature", ("\"7a68f95bf15136165c2e1048\"", MIME type text/plain)),
("seal_verifies", ("true", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"b8955af70fadbc35")
```

```
begin
    backup = joinpath(CarbonAccounting.builddir("notebook09"), "ledger_backup.json")
    save_ledger(ledger, backup)
    tamper!(ledger, 3; after="1 tCO2e (forged value, no corresponding source)")
    ok2, broken2 = verify_chain(ledger)
    (forged_entry=3, chain_ok=ok2, broken_at=broken2, detected=(!ok2 && broken2 == 3),
     seal_still_verifies=verify_seal(manifest, ledger))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("forged_entry", ("3", MIME type text/plain)), ("chain_ok",
("false", MIME type text/plain)), ("broken_at", ("3", MIME type text/plain)), ("detected",
("true", MIME type text/plain)), ("seal_still_verifies", ("true", MIME type text/plain))], :type
=> :NamedTuple, :objectid => "15455d121e165504")
```

```
begin
    restored = load_ledger(backup; hmac_key=ledger.hmac_key)
    (restored_entries=length(restored), statement=chain_statement(restored),
     verified=first(verify_chain(restored)))
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("restored_entries", ("8", MIME type text/plain)),
("statement", ("\"AUDIT CHAIN VERIFIED - 8 entries, head d055cbd037f3665e...\"", MIME type
text/plain)), ("verified", ("true", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"53c7df55f3ea113c")
```

```
md"""
### Immutability by design: corrections are entries, not edits
```

An inventory is corrected by appending an adjustment entry that references the original – the trail then shows both the mistake and the fix, which is what an assurance provider wants to see.

```
"""
```

Immutability by design: corrections are *entries*, not edits

An inventory is corrected by appending an adjustment entry that references the original — the trail then shows both the mistake and the fix, which is what an assurance provider wants to see.

```

begin
    reviewer = Actor("u004", "D. Reviewer", :approver)
    proposal = record!(ledger, analyst, "propose", "natural_gas 2024-06 site HAM-01";
        why="invoice correction: meter reading replaced by supplier
statement",
        before="-8.000 GJ (wrong sign)", after="+9.100 GJ (supplier
statement)",
        source_hash=sha256_hex("supplier statement 2024-06"))
    approval = record!(ledger, reviewer, "approve", "natural_gas 2024-06 site HAM-01";
        why="correction verified against the supplier statement",
        after="approved: +9.100 GJ")
    (proposal="entry $(proposal.seq) by $(proposal.actor_name) [$(proposal.role)]",
    approval="entry $(approval.seq) by $(approval.actor_name) [$(approval.role)]",
    prev_hash=first(approval.prev_hash, 24), statement=chain_statement(ledger))
end

```

```

Dict{Symbol, Any}(:elements => Tuple[("proposal", ("\"entry 9 by A. Analyst [analyst]\"", MIME
type text/plain)), ("approval", ("\"entry 10 by D. Reviewer [approver]\"", MIME type
text/plain)), ("prev_hash", ("\"8b327a5366c763cfafa8e5c0\"", MIME type text/plain)),
("statement", ("\"AUDIT CHAIN BROKEN at entry #3 - the ledger was modified outside the
application\"", MIME type text/plain))], :type => :NamedTuple, :objectid => "39651792bcb292ea")

```

```

begin
    println("ledger entries   : ", length(ledger))
    println("chain           : ", chain_statement(ledger))
    println("tamper detection : entry 3 forged → detected at 3, restored from the trusted
copy")
    println("access control   : the analyst cannot approve; segregation of duties enforced")
    println("Notebook 09 - every figure has a who, a when, a why and a source hash, and no
history can change unnoticed.")
end

```

10_Inventory_Rollup_Uncertainty_Disclosure

20 cells · executed in 33.12 s

```
begin
    using CarbonAccounting, CSV, DataFrames, Dates, Statistics, Printf
    using Plots
    default(fmt=:svg, legend=:topleft, size=(760, 420), dpi=120)
    datadir(parts...) = CarbonAccounting.datadir(parts...)
end
```

datadir (generic function with 1 method)

```
md"""
# 10 · The inventory: rollup, uncertainty, data quality and disclosure

Three things leave this notebook: a consolidated total, an uncertainty statement, and a submission package whose integrity can be verified by whoever receives it.


$$E_{\text{total}} = E_{\text{Scope1}} + E_{\text{Scope2}}(\text{location}) + E_{\text{Scope3}}$$


$$u_c^2 = \sum_g [ \sum_{i \in g} (E_i u_{\text{act},i})^2 + (A_g u_{\text{fac},g})^2 ] + 2p \sum_{\{g < h\}} A_g A_h u_{\text{fac},g} u_{\text{fac},h}$$


$$DQI_i = \sum_j w_j d_{ij} / \sum_j w_j$$

"""
```

10 · The inventory: rollup, uncertainty, data quality and disclosure

Three things leave this notebook: a **consolidated total**, an **uncertainty statement**, and a **submission package** whose integrity can be verified by whoever receives it.

```
E_total = E_Scope1 + E_Scope2(location) + E_Scope3
u_c^2    = Σ_g [ Σ_{i ∈ g} (E_i u_act,i)^2 + (A_g u_fac,g)^2 ] + 2p Σ_{g < h} A_g A_h u_fac,g u_fac,h
DQI_i    = Σ_j w_j d_ij / Σ_j w_j
```

```

begin
    ledger = AuditLedger(hmac_key=Vector{UInt8}(codeunits("notebook-10")))
    analyst = Actor("u001", "A. Analyst", :analyst)
    files = ["activity_erp_2024.csv", "electricity_utility_2024.csv", "procurement_2024.csv",
            "logistics_travel_2024.csv", "waste_2024.csv"]
    inventory = let
        recs = ingest_files([datadir("raw", f) for f in files];
                        entity_default="AcmeIndustrial_SA", ledger=ledger,
                        actor=analyst).records
        compute_inventory(recs; consolidation=:operational, date=Date(2024, 12, 31),
                        ledger=ledger, actor=analyst)
    end
    inventory
end

```

```

InventoryResult [operational]
E_Scope1      = 174781.393 tCO2e
E_Scope2 loc  = 73433.265 tCO2e
E_Scope2 mkt  = 86683.228 tCO2e
E_Scope3      = 12509.686 tCO2e
E_total       = 260724.344 tCO2e (445 rows)

```

```

md"""
### Consolidation @i changes which activities enter the boundary

Equity share weights every activity by ownership; the control approaches take all of
it. The comparison is therefore a boundary disclosure, not a sensitivity.
"""

```

Consolidation @_i changes *which* activities enter the boundary

Equity share weights every activity by ownership; the control approaches take all of it. The comparison is therefore a boundary disclosure, not a sensitivity.

```

begin
    recs = ingest_files([datadir("raw", f) for f in files];
                        entity_default="AcmeIndustrial_SA").records
    # the equity shares @_i come from the master data, keyed by site
    master_eq = CSV.read(datadir("reference", "master_data.csv"), DataFrame)
    shares = Dict{String{r.site} => Float64{r.equity_share} for r in eachrow(master_eq))
    eq = compute_inventory(recs; consolidation=:equity, shares=shares, date=Date(2024, 12,
31))
    fs = compute_inventory(recs; consolidation=:financial, date=Date(2024, 12, 31))
    DataFrame(approach=["operational control", "financial control", "equity share"],
              E_Scope1=[inventory.E_Scope1, fs.E_Scope1, eq.E_Scope1],
              E_Scope2_loc=[inventory.E_Scope2_loc, fs.E_Scope2_loc, eq.E_Scope2_loc],
              E_Scope3=[inventory.E_Scope3, fs.E_Scope3, eq.E_Scope3],
              E_total=[inventory.E_total, fs.E_total, eq.E_total])
end

Dict{Symbol, Any}(:rows => Any[(1, Any[("\operational control\\"", MIME type text/plain),
("1.74781e5", MIME type text/plain), ("73433.3", MIME type text/plain), ("12509.7", MIME type
text/plain), ("2.60724e5", MIME type text/plain)]), (2, Any[("\financial control\\"", MIME type
text/plain), ("1.74781e5", MIME type text/plain), ("73433.3", MIME type text/plain), ("12509.7",
MIME type text/plain), ("2.60724e5", MIME type text/plain)]), (3, Any[("\equity share\\"", MIME
type text/plain), ("1.3757e5", MIME type text/plain), ("50929.9", MIME type text/plain),
("10062.0", MIME type text/plain), ("1.98562e5", MIME type text/plain)]), :objectid =>
"db325ef513026d10", :schema => Dict{Symbol, Any}(:names => Any["approach", "E_Scope1",
"E_Scope2_loc", "E_Scope3", "E_total"], :types => ["String", "Float64", "Float64", "Float64",
"Float64"])]))

```

```

md"""
### Two routes to the uncertainty of the total

The GUM route propagates the sensitivity coefficients analytically, keeping the
activity part independent and the factor part correlated inside each factor group.
The Monte Carlo route draws each row's activity and factor and recomputes the
total – which also captures non-linearity and asymmetry. Agreement between the two is
the evidence that the model is nearly linear; disagreement is a finding.

Both routes run over `total_rows(inventory)`, i.e. Scope 1, the location-based
Scope 2 rows and Scope 3 – the rows that sum to `E_total`. The market-based Scope 2
rows sit in the same table but are a parallel report, not an addition.
"""

```

Two routes to the uncertainty of the total

The **GUM** route propagates the sensitivity coefficients analytically, keeping the activity part independent and the factor part correlated inside each factor group. The **Monte Carlo** route draws each row's activity and factor and recomputes the total — which also captures non-linearity and asymmetry. Agreement between the two is the evidence that the model is nearly linear; disagreement is a finding.

Both routes run over `total_rows(inventory)`, i.e. Scope 1, the *location-based* Scope 2 rows and Scope 3 — the rows that sum to `E_total`. The market-based Scope 2 rows sit in the same table but are a parallel report, not an addition.

```
begin
    rows_total = total_rows(inventory)
    gum = gum_uncertainty(rows_total)
    mc = monte_carlo_uncertainty(rows_total; n_MC=5_000, seed=42)
    (rows_in_total=length(rows_total), of_all=length(inventory.rows), gum, mc)
end
```

```
Dict{Symbol, Any}(:elements => Tuple[("rows_in_total", ("373", MIME type text/plain)), ("of_all",
("445", MIME type text/plain)), ("gum", ("GUM quadrature: E = 260724.344 tCO2e, uc = 96157.455
(36.88%), U95 = 192314.910, 95% CI = [68409.435, 453039.254] (n=0)", MIME type text/plain)),
("mc", ("Monte Carlo (n=5000): E = 260724.344 tCO2e, uc = 90264.508 (34.62%), U95 = 180529.016,
95% CI = [107012.580, 450314.720] (n=5000)", MIME type text/plain))], :type => :NamedTuple,
:objectid => "9dbdb214f64d698a")
```

```
uncertainty_table(gum, mc)
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\GUM quadrature (analytic)\", MIME type text/plain),
("2.60724e5", MIME type text/plain), ("96157.5", MIME type text/plain), ("0.368809", MIME type
text/plain), ("1.92315e5", MIME type text/plain), ("68409.4", MIME type text/plain),
("4.53039e5", MIME type text/plain), ("0", MIME type text/plain)]), (2, Any[("\Monte Carlo
simulation\"", MIME type text/plain), ("2.60724e5", MIME type text/plain), ("90264.5", MIME type
text/plain), ("0.346207", MIME type text/plain), ("180529.0", MIME type text/plain),
("1.07013e5", MIME type text/plain), ("4.50315e5", MIME type text/plain), ("5000", MIME type
text/plain)]), (3, Any[("\Monte Carlo, median\"", MIME type text/plain), ("2.60724e5", MIME type
text/plain), ("90264.5", MIME type text/plain), ("0.346207", MIME type text/plain), ("180529.0",
MIME type text/plain), ("1.07013e5", MIME type text/plain), ("4.50315e5", MIME type text/plain),
("5000", MIME type text/plain)]]], :objectid => "2662568d4592a7ca", :schema => Dict{Symbol, Any}
(:names => Any["method", "E", "uc", "urel", "U95", "lo95", "hi95", "n"], :types => ["String",
"Float64", "Float64", "Float64", "Float64", "Float64", "Float64", "Int64"])
```

```
begin
    plt = histogram(mc.samples, bins=60, legend=false, color=:steelblue, alpha=0.7,
                    xlabel="tCO2e", ylabel="Monte-Carlo trials",
                    title="Inventory uncertainty (n = $(length(mc.samples)) trials)")
    vline!(plt, [mc.E], color=:black, ls=:dash, label="central")
    vline!(plt, [mc.lo95, mc.hi95], color=:red, ls=:dot, label="95 % interval")
    save_figure(plt, "10_uncertainty_histogram"; saver=savefig)
    plt
end
```



```
md"""
### Where the quality sits, and the intensity metrics that survive growth
"""
```

Where the quality sits, and the intensity metrics that survive growth

```

begin
    master = CSV.read(datadir("reference", "master_data.csv"), DataFrame)
    production = sum(master.production_t)
    revenue = sum(master.revenue_usd)
    employees = sum(master.employees)
    (dqi=first(sort(dqi_table(inventory.rows), :DQ, rev=true), 5),
     intensity=intensity_metrics(inventory; production_t=production, revenue_usd=revenue,
                               employees=employees),
     denominators=(production_t=production, revenue_USD=revenue, employees=employees))
end

Dict{Symbol, Any}(:elements => Tuple[("dqi", (Dict{Symbol, Any}(:rows => Any[(1,
Any[("\purchased_goods_spend\"", MIME type text/plain), ("4162.14", MIME type text/plain),
("0.0119806", MIME type text/plain), ("4.0", MIME type text/plain), ("0.474342", MIME type
text/plain), ("\D - low, screening estimate to be refined\"", MIME type text/plain)]), (2,
Any[("\ldar\"", MIME type text/plain), ("199.392", MIME type text/plain), ("0.000573942", MIME
type text/plain), ("4.0", MIME type text/plain), ("0.636396", MIME type text/plain), ("\D - low,
screening estimate to be refined\"", MIME type text/plain)]), (3, Any[("\employee_commuting\"",
MIME type text/plain), ("4089.19", MIME type text/plain), ("0.0117706", MIME type text/plain),
("3.0", MIME type text/plain), ("0.23065", MIME type text/plain), ("\C - fair, industry average
or proxy\"", MIME type text/plain)]), (4, Any[("\waste\"", MIME type text/plain), ("1724.34",
MIME type text/plain), ("0.00496344", MIME type text/plain), ("3.0", MIME type text/plain),
("0.359212", MIME type text/plain), ("\C - fair, industry average or proxy\"", MIME type
text/plain)]), (5, Any[("\upstream_transport\"", MIME type text/plain), ("1349.18", MIME type
text/plain), ("0.00388358", MIME type text/plain), ("3.0", MIME type text/plain), ("0.206128",
MIME type text/plain), ("\C - fair, industry average or proxy\"", MIME type text/plain)]),
:objectid => "c86b2b732f0374f9", :schema => Dict{Symbol, Any}(:names => Any["source_type", "E",
"share", "DQ", "u_rel", "quality"], :types => ["String", "Float64", "Float64", "Float64",
"Float64", "String"])), MIME type application/vnd.pluto.table+object)), ("intensity",
(Dict{Symbol, Any}(:rows => Any[(1, Any[("\Scope 1 intensity\"", MIME type text/plain),
("0.0267659", MIME type text/plain), ("\tCO₂e/t\"", MIME type text/plain)]), (2, Any[("\Scope 2
intensity (location)\"", MIME type text/plain), ("0.0112455", MIME type text/plain),
("\tCO₂e/t\"", MIME type text/plain)]), (3, Any[("\Scope 3 intensity\"", MIME type text/plain),
("0.00191573", MIME type text/plain), ("\tCO₂e/t\"", MIME type text/plain)]), (4, Any[("\Total
intensity\"", MIME type text/plain), ("0.0399272", MIME type text/plain), ("\tCO₂e/t\"", MIME
type text/plain)]), (5, Any[("\Revenue intensity\"", MIME type text/plain), ("105.131", MIME
type text/plain), ("\tCO₂e/M\\$\"", MIME type text/plain)]), (6, Any[("\Per employee\"", MIME
type text/plain), ("120.706", MIME type text/plain), ("\tCO₂e/FTE\"", MIME type text/plain)]),
:objectid => "e6dd8d043a4470a", :schema => Dict{Symbol, Any}(:names => Any["metric", "value",
"unit"], :types => ["String", "Float64", "String"])), MIME type
application/vnd.pluto.table+object)), ("denominators", (Dict{Symbol, Any}(:elements =>
Tuple[("production_t", ("6.53e6", MIME type text/plain)), ("revenue_USD", ("2.48e9", MIME type
text/plain)), ("employees", ("2160", MIME type text/plain))], :type => :NamedTuple, :objectid =>
"822b04105a32c4e"), MIME type application/vnd.pluto.tree+object))), :type => :NamedTuple,
:objectid => "bcd06672965713ec")

```

```
md"""
### Year-on-year change needs a prior year computed the same way
```

The prior period is built by re-ingesting the same export with volumes 7 % lower — through the identical pipeline and with the **same factor set** (the same factor date), so a change is attributable to activity instead of to factor churn.

Year-on-year change needs a prior year computed the same way

The prior period is built by re-ingesting the same export with volumes 7 % lower — through the identical pipeline and with the **same factor set** (the same factor date), so a change is attributable to activity instead of to factor churn.

```
begin
    prior_dir = CarbonAccounting.builddir("notebook10", "prior")
    mkpath(prior_dir)
    src = CSV.read(datadir("raw", "activity_erp_2024.csv"), DataFrame; delim=';',
                    types=String)
    scaled = copy(src)
    scaled[!, "Verbrauch"] = [string(round(parse_number(x) * 0.93; digits=1)) for x in
src.Verbrauch]
    CSV.write(joinpath(prior_dir, "activity_erp_prior.csv"), scaled; delim=';')
    prior_recs = vcat(ingest_files([joinpath(prior_dir, "activity_erp_prior.csv");
                                entity_default="AcmeIndustrial_SA").records,
                        [r for r in recs if !occursin("activity_erp", r.source)])
    prior = compute_inventory(prior_recs; consolidation=:operational, date=Date(2024, 12,
31))
    compare_years(inventory, prior)
end
```

```
Dict{Symbol, Any}(:rows => Any[(1, Any[("\Scope 1\\"", MIME type text/plain), ("1.74781e5", MIME
type text/plain), ("1.62547e5", MIME type text/plain), ("12234.8", MIME type text/plain),
("7.52692", MIME type text/plain)]), (2, Any[("\Scope 2 (location)\\"", MIME type text/plain),
("73433.3", MIME type text/plain), ("73433.3", MIME type text/plain), ("0.0", MIME type
text/plain), ("0.0", MIME type text/plain)]), (3, Any[("\Scope 2 (market)\\"", MIME type
text/plain), ("86683.2", MIME type text/plain), ("86683.2", MIME type text/plain), ("0.0", MIME
type text/plain), ("0.0", MIME type text/plain)]), (4, Any[("\Scope 3\\"", MIME type text/plain),
("12509.7", MIME type text/plain), ("12509.7", MIME type text/plain), ("0.0", MIME type
text/plain), ("0.0", MIME type text/plain)]), (5, Any[("\Total\\"", MIME type text/plain),
("2.60724e5", MIME type text/plain), ("2.4849e5", MIME type text/plain), ("12234.8", MIME type
text/plain), ("4.92365", MIME type text/plain)]), :objectid => "177e1cf1726d486c", :schema =>
Dict{Symbol, Any}(:names => Any["component", "current", "prior", "Δ", "Δ_pct"], :types =>
["String", "Float64", "Float64", "Float64", "Float64"])]
```

```
md#### The disclosure table — every figure with its qualification"
```

The disclosure table — every figure with its qualification

```

begin
    disp = disclosure_table(inventory; prior=prior, uncertainty=mc,
                           intensity=intensity_metrics(inventory; production_t=production,
                                                         revenue_usd=revenue,
                                                         employees=employees))
    first(disp, 12)
end

```

```

Dict{Symbol, Any}(:rows => Any[[1, Any[("\Gross Scope 1 GHG emissions\\"", MIME type text/plain),
(1.74781e5, MIME type text/plain), ("\tCO₂e\\"", MIME type text/plain), ("\direct emissions,
operational consolidation\\"", MIME type text/plain), (1.62547e5, MIME type text/plain),
(7.52692, MIME type text/plain)]], (2, Any[("\Scope 2, location based\\"", MIME type
text/plain), (73433.3, MIME type text/plain), ("\tCO₂e\\"", MIME type text/plain), ("\grid-
average factors\\"", MIME type text/plain), (73433.3, MIME type text/plain), (0.0, MIME type
text/plain)]], (3, Any[("\Scope 2, market based\\"", MIME type text/plain), (86683.2, MIME type
text/plain), ("\tCO₂e\\"", MIME type text/plain), ("\contractual instruments, λ_cov = 19.6 %\\"",
MIME type text/plain), (86683.2, MIME type text/plain), (0.0, MIME type text/plain)]], (4,
Any[("\Scope 3, total value chain\\"", MIME type text/plain), (12509.7, MIME type text/plain),
("\tCO₂e\\"", MIME type text/plain), ("\categories 1...15, hybrid methods\\"", MIME type
text/plain), (12509.7, MIME type text/plain), (0.0, MIME type text/plain)]], (5, Any[("\ 1
Purchased goods and services\\"", MIME type text/plain), (4162.14, MIME type text/plain),
("\tCO₂e\\"", MIME type text/plain), ("\share 33.3 %, weighted DQ 4.0\\"", MIME type text/plain),
("NaN", MIME type text/plain), ("NaN", MIME type text/plain)]], (6, Any[("\ 4 Upstream
transportation and distribution\\"", MIME type text/plain), (1349.18, MIME type text/plain),
("\tCO₂e\\"", MIME type text/plain), ("\share 10.8 %, weighted DQ 3.0\\"", MIME type text/plain),
("NaN", MIME type text/plain), ("NaN", MIME type text/plain)]], (7, Any[("\ 5 Waste generated
in operations\\"", MIME type text/plain), (1724.34, MIME type text/plain), ("\tCO₂e\\"", MIME
type text/plain), ("\share 13.8 %, weighted DQ 3.0\\"", MIME type text/plain), ("NaN", MIME type
text/plain), ("NaN", MIME type text/plain)]], (8, Any[("\ 6 Business travel\\"", MIME type
text/plain), (582.3, MIME type text/plain), ("\tCO₂e\\"", MIME type text/plain), ("\share 4.7
%, weighted DQ 3.0\\"", MIME type text/plain), ("NaN", MIME type text/plain), ("NaN", MIME type
text/plain)]], (9, Any[("\ 7 Employee commuting\\"", MIME type text/plain), (4089.19, MIME
type text/plain), ("\tCO₂e\\"", MIME type text/plain), ("\share 32.7 %, weighted DQ 3.0\\"", MIME
type text/plain), ("NaN", MIME type text/plain), ("NaN", MIME type text/plain)]], (10, Any[("\
9 Downstream transportation and distribution\\"", MIME type text/plain), (602.539, MIME type
text/plain), ("\tCO₂e\\"", MIME type text/plain), ("\share 4.8 %, weighted DQ 3.0\\"", MIME type
text/plain), ("NaN", MIME type text/plain), ("NaN", MIME type text/plain)]], (11, Any[("\Total
GHG emissions\\"", MIME type text/plain), (2.60724e5, MIME type text/plain), ("\tCO₂e\\"", MIME
type text/plain), ("\Scope 1 + Scope 2 (location) + Scope 3\\"", MIME type text/plain),
(2.4849e5, MIME type text/plain), (4.92365, MIME type text/plain)]], (12, Any[("\Biogenic
CO₂ emitted (memo)\\"", MIME type text/plain), (0.0, MIME type text/plain), ("\tCO₂e\\"", MIME
type text/plain), ("\excluded from the total by convention, disclosed separately\\"", MIME type
text/plain), ("NaN", MIME type text/plain), ("NaN", MIME type text/plain)]], :objectid =>
"a9d4b77c2873dcb4", :schema => Dict{Symbol, Any}(:names => Any["item", "value", "unit", "note",
"prior_year", "change_pct"], :types => ["String", "Float64", "String", "String", "Float64",
"Float64"])))

```

```
md"""
### The submission package

Rows, contribution, Scope-3 categories, DQI, the disclosure table, the exact factor
library used, the audit ledger – and a manifest with the SHA-256 of every file
and the ledger head hash, so the recipient can verify that nothing changed after the
run.
"""
```

The submission package

Rows, contribution, Scope-3 categories, DQI, the disclosure table, the exact factor library used, the audit ledger — and a **manifest** with the SHA-256 of every file and the ledger head hash, so the recipient can verify that nothing changed after the run.

```
begin
    package = export_inventory(inventory; dir=CarbonAccounting.builddir("submission"),
                              ledger=ledger, library=default_library(), uncertainty=mc,
                              extra=Dict("reporting_year" => "2024",
                                          "consolidation" => "operational control"))
    (files=sort(collect(keys(package.files))),
     manifest_entries=sort(collect(keys(package.manifest))),
     ledger_verified=package.manifest["ledger_verified"],
     factor_library=package.manifest["factor_library"])
end
```

```
Dict{Symbol, Any}(:elements => Tuple{("files", (Dict{Symbol, Any}(:prefix => "String", :elements
=> Tuple{Int64, Any}[(1, ("\"audit_ledger.json\"", MIME type text/plain)), (2,
(\"\"calc_rows.csv\"", MIME type text/plain)), (3, ("\"contribution.csv\"", MIME type
text/plain)), (4, ("\"disclosure.csv\"", MIME type text/plain)), (5, ("\"dqi.csv\"", MIME type
text/plain)), (6, ("\"factors.csv\"", MIME type text/plain)), (7, ("\"manifest.json\"", MIME type
text/plain)), (8, ("\"scope3_categories.csv\"", MIME type text/plain))], :type => :Array,
:prefix_short => "", :objectid => "45c829748be31160"), MIME type
application/vnd.pluto.tree+object)), ("manifest_entries", (Dict{Symbol, Any}(:prefix => "String",
:elements => Tuple{Int64, Any}[(1, ("\"consolidation\"", MIME type text/plain)), (2,
(\"\"factor_library\"", MIME type text/plain)), (3, ("\"files\"", MIME type text/plain)), (4,
(\"\"generated_at\"", MIME type text/plain)), (5, ("\"ledger_broken_at\"", MIME type text/plain)),
(6, ("\"ledger_entries\"", MIME type text/plain)), (7, ("\"ledger_head\"", MIME type
text/plain)), (8, ("\"ledger_verified\"", MIME type text/plain)), (9, ("\"notes\"", MIME type
text/plain)), (10, ("\"sync_watermark\"", MIME type text/plain)), (11, ("\"totals\"", MIME type
text/plain))], :type => :Array, :prefix_short => "", :objectid => "a50b82d6ca698bcb"), MIME type
application/vnd.pluto.tree+object)), ("ledger_verified", ("true", MIME type text/plain)),
("factor_library", ("\"CarbonAccounting built-in 2026.1\"", MIME type text/plain))], :type =>
:NamedTuple, :objectid => "ade269c41debfd7a")
```

```
Markdown.parse(report_markdown(inventory; uncertainty=mc,
                                dqi=dqi_table(inventory.rows), prior=prior))
```

Inventory summary (operational consolidation)

component	tCO ₂ e	share
Scope 1	174781.4	67.0 %
Scope 2 (location based)	73433.3	28.2 %
Scope 2 (market based)	86683.2	33.2 %
Scope 3	12509.7	4.8 %

- **Total:** 260724.3 tCO₂e over 445 calculated rows
- **Biogenic CO₂ (memo):** 0.0 tCO₂e
- **Contractual instrument coverage:** 19.6 % of purchased electricity
- **Uncertainty:** Monte Carlo (n=5000): E = 260724.3 tCO₂e ± 180529.0 tCO₂e (U₉₅, k = 2), i.e. ±34.6 %; 95 % interval [107012.6, 450314.7] tCO₂e.
- **Weakest data quality:** purchasedgoodsspend (DQ 4.0, 1.2 % of emissions)

Change versus the prior year:

- Scope 1: +12234.8 tCO₂e (+7.5 %)
- Scope 2 (location): +0.0 tCO₂e (+0.0 %)
- Scope 2 (market): +0.0 tCO₂e (+0.0 %)
- Scope 3: +0.0 tCO₂e (+0.0 %)
- Total: +12234.8 tCO₂e (+4.9 %)

```
begin
    t = inventory_totals(inventory)
    println("E_Scope1      : ", round(t.scope1; digits=1), " tCO2e")
    println("E_Scope2 loc/mkt: ", round(t.scope2_loc; digits=1), " / ",
            round(t.scope2_mkt; digits=1), " tCO2e")
    println("E_Scope3      : ", round(t.scope3; digits=1), " tCO2e")
    println("E_total        : ", round(t.total; digits=1), " tCO2e")
    println("uncertainty     : ", uncertainty_statement(mc))
    println("chain          : ", chain_statement(ledger))
    println("Notebook 10 - the inventory is consolidated, uncertainty-stated, quality-rated,
disclosable and packaged with a verifiable manifest.")
end
```